Nafs-Aql: A Verifier-Gated Dual-Process Architecture on Small Qwen Models for CPU-Only Deployment

Dirac R&D Team

***Abstract*—We present Nafs-Aql, a hybrid System 1 / System 2 language-model architecture built from small, openly licensed Qwen models and designed to run on a CPU-only machine. The design borrows its vocabulary, as an explicit analogy, from the classical Islamic account of the self: the impulsive *nafs*, the arbitrating *'aql*, the integrating *qalb*, and the practices of self-accounting (*muhasaba*) and purification (*tazkiya*). We turn this inspiration into a formal model with explicit state spaces, operators, costs and losses, and prove the properties that drive the engineering choices: the gate between the fast and slow paths is a threshold rule on a calibrated error estimate; precision on verifiable tasks is bounded by the verifier's false-accept rate and not by model strength; unverified self-refinement converges to a fixed point that can be worse than the first draft, while a verifier-gated loop has risk linear in the false-accept rate; plurality voting cannot repair confidently wrong tasks; and a Beta-prior optimal-stopping rule yields the retry cap. Each result is mapped to a parameter or an algorithm of the specification. We verify the stack against current documentation: Qwen3 and Qwen3.5 are the only Qwen generations with open weights at or below 9B parameters; vLLM's CPU backend requires Linux and prefers AVX-512; Unsloth requires a GPU for training, so fine-tuning is scheduled on rented GPUs with GGUF export for CPU inference; and a Rust gateway (axum, tokio) fronts both vLLM and llama-server through the OpenAI-compatible API. On the reference machine (Intel i3-8100, AVX2, 32 GB) we measured 15.5 decoded tokens/s for Qwen3-1.7B in 4-bit, consistent with a memory-bandwidth roofline. We give the architecture, the implementation stack, the evaluation plan with statistically sized gates, and a summary of the companion specification and roadmap. Version 2.0 adds the implemented system and its measurements on a shared four-core CPU. With an exact verifier visible, the gate matched deliberate-only accuracy at about a third of its cost. Stage-level latency attribution shows that System 1 decode speed dominates the fast path, and an early-escalation rule removes the remainder of drafts the gate will reject anyway. A Qwen3.5-2B System 1 was 17 points more accurate but 3.7 times slower, so it was not adopted. These quality results are indicative: sample sizes are far below the release gates. Version 2.1 integrates mechanisms from two open agent harnesses: from Open Math Harness, Lean 4 as an exact verifier under a frozen-statement protocol and a best-first repair tree scored by the verifier, which provably reduces to the previous loop when verdicts are all-or-nothing; from OpenAI's Codex, the `apply_patch` format for code repairs and a bounded code agent that stops only when the exact verifier accepts. Version 2.2 rebuilds that agent on the harness mechanisms of Codex's agent core, adapted to small models on a CPU: an append-only transcript that keeps the server's prompt cache valid, compaction written by the harness, Codex's patch grammar as a per-step decoding grammar that leaves the reasoning span free, and a whole-file edit policy for short files. In an interleaved live A/B the new loop verified 4 of 10 System 1 tasks against 2 for the old one, with malformed steps falling from 23 of 50 to 1 of 41, and all 3 System 2 tasks in one call each, where the old loop verified none in 15 calls; with a smaller reasoning budget on the first step a verified System 2 fix of a small function took 33–59 s. Samples are small. Version 2.3 validates the whole system end to end on the final code (190 Rust tests, 26 security probes, 65 API and UI checks, 59 Rooms checks, 40 harness checks, 9 acceptance tests), finds and fixes three defects in the Lean integration, and reports that Qwen3-4B-Thinking cannot yet write valid Lean 4 proofs even for trivial statements, so Lean tasks abstain honestly rather than verify. Version 2.4 asks how far four cores can be pushed in parallel. Batched decode gains 2.7× at short context but only 1.5× at the 1,600 tokens of a repair prompt, so concurrent hedged sampling on System 2 does not pay (we implement and measure it, and keep it off), while cheap concurrent System 1 samples before escalating do: fusing the agreement samples with the draft cuts the time to the gate's decision by 24 % ($p = 0.0005$), and four verified rescue samples avoid escalations at a break-even of 106 s against 250–580 s each; one request with n choices shares the prefill (39.1 s against 124.5 s for four separate requests). Version 2.5 measures the engine's own flags, the switch between the two models and the two backends, and what Codex's agent core still had to offer: the flag matrix is inconclusive on a shared host and nothing is changed, the switch inside a request costs about 1.5 s and needs no freeze, the backend switch is made warm-first, failed model calls are retried with Codex's backoff, and a cascade in the code agent (System 1 candidates judged by the verifier first, System 2 only when none passes) verifies the same ten tasks in 26 % less wall time, six of them without a System 2 call. Version 2.6 integrates the owner's Rust CPU engine, Smiley Core, as a third backend, compares it with llama.cpp alone and through the whole gateway (1.13× decode; twelve checked prompts in 131 s against 303 s, faster in 10 of 12 pairs), and makes it the default; a failed first run of the code agent on it exposed a parser defect, now fixed, in how an unconstrained patch is read. Version 2.7 compares the code agent with the harnesses of Codex, Claude Code, Gemini CLI and the published agents (SWE-agent, Agentless, observation masking, DeepSeek-V3.2), closes the gaps that matter at this scale (a compile check before the verifier, rollback to the best version, a stall guard, project guidelines), and measures that they cost nothing: ten tasks on each model verify the same with the guards on and off, and the check costs 37 ms per edit. Version 2.8 assesses Smiley Core and the model switch: the engine is within 60–95 % of the machine's memory bandwidth, so the gains were in repeated work, not in more parallelism. Three decode threads beat four on a shared host; a prefix store cuts the cost of switching between kinds of request**

Manuscript prepared 18 October 2026 (version 2.0: design and implementation); version 2.1, 24 October 2026, adds the math and code harness (Section XI); version 2.2, 26 October 2026, adds the Codex-style agent harness (Section XII); version 2.3, 29 October 2026, reports the end-to-end validation of the whole system (Section XIII); version 2.4, 31 October 2026, adds parallel and concurrent inference (Section XIV); version 2.9, 5 November 2026, adds the multi-user engine (Section XIX) and the engine update with LittleBit (Section XX). Contact: Dirac R&D Team. Section IX reports measured results of the implemented system; elsewhere, performance figures are targets or estimates unless marked as measured.

from 5.8 s and 14 s of prefill to 0.1 s and 0.2 s with byte-identical output; and the reasoning budget's continuation no longer re-prefills the reasoning it has just computed. Version 2.9 serves several users from one pass over the weights (identical text to a solo run at every concurrency, $1.4\times$ the throughput of llama.cpp at two users) and then updates the engine to its latest revision with LittleBit, a 1-bit-class weight format: the integration is complete and tested, but post-training conversion of Qwen3-1.7B at 0.98 bits per weight answers 0 of 12 exact-answer prompts against 9 for the dense model, so it stays off until trained factors exist; the update also exposed a fourfold decode slowdown caused by a compiler inlining decision in an unchanged kernel, found by bisection and fixed.

***Index Terms*—dual-process reasoning, test-time compute, routing, verification, small language models, Qwen, CPU inference, Islamic philosophy of mind, Rust.**

I. Introduction

Reasoning-tuned language models answer hard questions by spending more computation at inference time: they sample long chains of thought, draw several candidates, and verify or vote among them [1]–[4]. The gain is real but expensive, and it is wasted on the many requests that a direct answer would have handled [5], [6]. This is the engineering form of a very old observation: cognition has a fast, automatic mode and a slow, effortful one, and competence lies in knowing when to switch [7]–[9].

The Islamic intellectual tradition developed a rich vocabulary for the same observation long before dual-process psychology. The Qur'an speaks of the *nafs* that "commands to evil" (Q 12:53), the *nafs* that reproaches itself (Q 75:2) and the *nafs* "at rest" (Q 89:27) [10]; al-Ghazali describes the heart (*qalb*) as a king whose vizier is the intellect (*'aql*) and whose unruly soldiers are appetite and anger [11]; and the Sufi disciplines of *muraqaba* (vigilance) and *muhasaba* (self-accounting) institutionalise the daily audit of one's own conduct [12], [13]. We use this vocabulary as a design metaphor. We make no claim about the metaphysics of the soul, and we say so once here and again in Section II-A: every mapping in this paper is an analogy between a classical *description of a faculty* and an *engineering component*.

A. Problem

The project that this report serves has a concrete constraint set: build a dual-process assistant from pretrained Qwen models, serve it on a CPU-only machine, fine-tune it with Unsloth, and expose it through a Rust web application with a chat room, a performance-test console and an evaluation dashboard. An existing draft design [14] targeted 27B–35B models on 80 GB GPUs and reviewed twelve formal results; it did not address CPU feasibility, the model roster, the serving stack, or a buildable specification. This report assesses that draft, keeps what is sound, corrects what is not, and extends it into a mathematically principled and feasible design.

B. Contributions

1) A faithful, sourced account of the classical concepts we borrow, with the disagreements among scholars noted and the analogy kept separate from theology (Section II-A).
2) A formal model of a verifier-gated dual-process system with seventeen stated results, each with explicit assumptions and an engineering consequence (Section III). The results correct two defects of the draft: its router rule was hand-tuned rather than derived, and its self-refinement loop was not shown to be stable.
3) A verified feasibility study of the stack as of October 2026: which Qwen variants exist below 9B parameters, which run in vLLM-CPU and llama.cpp, what Unsloth can and cannot do without a GPU, which Rust crates to use, and what a four-core AVX2 machine actually delivers (Sections V and V-D).
4) A component architecture with a termination guarantee, an assurance-level model for answers, and a self-model that is unbiased under routing (Section IV).
5) An evaluation plan whose gates are statistically sized, and a summary of the companion specification and roadmap (Sections VII and VIII).

C. What this report does not claim

No accuracy, calibration or cost result for Nafs-Aql has been measured. Where a number appears it is (a) a measurement of an existing model on the reference machine, clearly marked, (b) a value quoted from a cited paper, or (c) a target or estimate. The roadmap (Section VII) schedules every hypothesis for validation.

II. Background

A. The Islamic Account of the Self

1) Scope and disclaimer: We summarise the classical vocabulary we borrow, from primary texts in verified translations and from standard scholarship. The summary is descriptive. We attribute each concept to a specific author and text, note where scholars disagree, and do not present "the Islamic view of the soul" as one fixed scheme. The engineering mapping of Section IV is an analogy between faculty descriptions and software components; it is not a claim that the software has a soul, a heart or an intellect in the sense of the sources. We do not model the *ruh* (spirit), which the Qur'an places beyond human analysis (Q 17:85), and we do not map divine inspiration (*ilham*), unveiling (*mukashafa*) or the Active Intellect of the philosophers onto any component.

2) The nafs and its Qur'anic stages: Nafs most often means simply "self" or "person". Three verses attach a qualifying adjective and these became the seeds of the later Sufi "stations" of the soul [10], [15]: *al-nafs al-ammarah bi-l-su'*, the self that commands to evil (Q 12:53); *al-nafs al-lawwamah*, the self-reproaching self (Q 75:2); and *al-nafs al-mutma'innah*, the self at rest (Q 89:27), addressed in the next verse as "well pleased and well pleasing" (*radiyah*, *mardiyyah*, Q 89:28). Q 91:7–10 says the soul was "inspired" with its wickedness and its God-consciousness and that whoever purifies it (*zakkaha*) succeeds. Only the first three are fixed noun–adjective pairs in the text. The "inspired self" (*al-nafs al-mulhamah*) is a Sufi coinage from the verb in Q 91:8, and the "perfect" or "pure" self (*kamilah*, *safiyyah*, *zakiyyah*) is a later elaboration. Later manuals arrange seven stages, commonly *ammarah*, *lawwamah*,

mulhamah, *mutma'innah*, *radiyah*, *mardiyyah*, *kamilah*, but the order and the name of the highest rung vary by order and author [16]. A recent grounded-theory model of the soul for Islamic psychotherapy deliberately keeps only the three Qur'anic stages [17]. We follow the same restraint: our trace-state automaton (Fig. 3) uses the three Qur'anic stages as mnemonics and names its other states in plain engineering terms.

3) Al-Ghazali: heart, intellect, soul, spirit: In Book 21 of the *Ihya'*, "The Marvels of the Heart", al-Ghazali (d. 1111) distinguishes two senses of each of four words [11]. *Qalb* is the organ of flesh and also a spiritual subtlety that is the knowing, perceiving human being; *ruh* is the physicians' subtle vapour and also that same subtlety; *nafs* is the aggregate of appetite and anger, the blameworthy lower self of Q 12:53, and also the same subtlety named *mutma'innah*, *lawwamah* or *ammarah* according to its state; and *'aql* is knowledge of the real natures of things and also the perceiving subtlety itself. In their second sense the four words name one entity under different aspects. Al-Ghazali's political image is the one we borrow: the heart is a king, the intellect is its vizier and counsellor, appetite is its provisioner and anger its police; the kingdom is well ordered when the king consults the vizier and uses the other two without letting them rule. He also describes the heart as a mirror in which the forms of things are reflected, tarnished by vice and polished by remembrance and discipline; incoming thoughts (*khawatir*) that call to good are *ilham* and those that call to evil are *waswasa*. Books 22–23 give the practical programme of disciplining the soul [18]. Al-Ghazali adopts Avicennan faculty psychology in all of this [19]–[21], and treats error as a structural failure of the soul's faculties [22].

4) Muraqaba and muhasaba: Book 38 of the *Ihya'* is devoted to vigilance and self-accounting: the servant sets conditions on the self in the morning, watches it through the day (*muraqaba*), and audits it at night (*muhasaba*), then corrects and strives. The vocabulary goes back to al-Harith al-Muhasibi (d. 857), whose name derives from the practice [12]; al-Qushayri's *Risala* has chapters on both and a lexicon that distinguishes *nafs*, *qalb*, *ruh* and *sirr* as successively subtler faculties [13]. The engineering reading is direct: muhasaba is self-correction *under a standard*, which is what an evaluation and monitoring loop does, and *tazkiyat al-nafs* (purification) is the long-run change of disposition that such a loop is meant to produce.

5) Ibn Sina: internal senses and intellect: Avicenna (d. 1037) defines the soul as the first perfection of a natural body with organs and distinguishes the vegetative, animal and rational souls [23]–[26]. He systematises five internal senses located in the brain's ventricles [27]: the common sense and the retentive imagination in the front ventricle, the estimative faculty (*wahm*) and the compositive imagination in the middle, and memory (*dhakira*) in the rear. *Wahm* grasps non-sensible "intentions" such as the hostility of a wolf; it is the quick, particular judgement that animals and humans share and a frequent source of error [28]. The theoretical intellect passes from potentiality through habit and act to the "acquired" intellect, receiving intelligibles from a separate Active Intellect, a doctrine al-Ghazali criticised and later philosophers replaced [29], [30]. The "flying man" thought experiment argues that self-awareness is primitive and does not depend on the body; whether it is a demonstration or a pointer is disputed [31], [32].

6) Ibn Arabi, Suhrawardi, Mulla Sadra: Ibn Arabi (d. 1240) names the heart from *taqallub*, fluctuation: it follows what reason, from *'iqal* (the fetter), cannot encompass, and sound knowledge needs both of the heart's "two eyes", reason and imagination [33], [34]. Suhrawardi (d. 1191) holds that the soul knows itself and its states by presence, without an intervening form [35], [36]. Mulla Sadra (d. 1636) teaches substantial motion: the soul is "bodily in its origination, spiritual in its subsistence", intensifying through sensory, imaginal and intellectual stages, and in the act of knowing the knower and the known are one [37]–[39]. The draft design mapped substantial motion to distillation (slow reasoning becoming fast); we keep the mapping as an illustration of *gradual change of disposition* and nothing more.

7) Where scholars disagree: Whether *nafs* and *ruh* are one entity; whether *'aql* is a faculty, a body of knowledge or a substance; whether the soul is immaterial from the start (Avicenna), a subtle body (much of early kalam), or material in origin and immaterial in subsistence (Sadra); whether the Active Intellect is needed; how many stages the *nafs* has; and even what al-Ghazali himself believed [19], [40], [41]: all of these are open in the literature. Our analogy does not depend on any of them being settled.

8) Rules we follow: (1) Every borrowing is presented as an analogy and the components keep plain engineering names in code. (2) We use the faculty-psychology vocabulary (*nafs*, *'aql*, *qalb*, *wahm*, *dhakira*, *muhasaba*, *tazkiya*) and nothing from the vocabulary of revelation. (3) The stage ladder is not a maturity model or a benchmark scale. (4) Scripture is cited by chapter and verse in a named translation and hadith by collection and number; we quote no unsourced sayings. (5) The analogy section should be reviewed by a scholar of the tradition before publication.

B. Dual-Process Theory and Metareasoning

Dual-process accounts distinguish a fast, automatic, associative mode from a slow, effortful, rule-based one [7], [9], [42]. The contemporary formulation of Evans and Stanovich defines Type 1 processing by autonomy and Type 2 by its demand on working memory and its capacity for hypothetical thought, and treats the question of *when* Type 2 intervenes as central [8]. Two further lines matter for us. The *expected value of control* theory of Shenhav, Botvinick and Cohen models the allocation of cognitive control as a cost–benefit decision [43]; resource-rational analysis makes the same move for cognition in general [44]; and the metareasoning literature in artificial intelligence, from Russell and Wefald onward, asks how an agent should decide which computations to run [45]. Theorem 2 is the one-step instance of these ideas: deliberate when its expected value exceeds its cost.

C. Fast and Slow Language Models

1) Test-time compute: Chain-of-thought prompting elicits multi-step reasoning [1]; sampling several chains and voting improves accuracy, by $+17.9$ points on GSM8K in the original study [2]. Coverage, the chance that at least one of N samples is correct, grows roughly log-linearly in N across orders of magnitude, but without an automatic verifier the gain from voting or reward models plateaus after a few hundred samples [46]; trained verifiers and process-supervised reward models recover much of it [47], [48]. Allocating test-time compute per prompt according to difficulty is more than four times as efficient as uniform best-of-N and, in a FLOPs-matched comparison, can beat a model fourteen times larger on problems the small model can sometimes solve [3]. Reinforcement learning induces long reasoning traces with self-verification [4], [49], [50]; a thousand curated examples plus "budget forcing" already yields a strong reasoner [51]. Qwen3 ships both modes in one model with a thinking budget [52].

2) Self-correction: Prompting a model to critique and revise its own answer helps on some tasks [53], [54] but, without external feedback, intrinsic self-correction does not improve reasoning and can degrade it [55]; a survey concludes that self-correction works with reliable external feedback or large-scale fine-tuning [56]. Theorem 4 gives the simplest model in which this happens. Reasoning models also *over*think simple questions [5].

3) Deciding when to think: A rapidly growing literature trains or prompts models to choose between a direct answer and a reasoning trace per query [6], [57]–[61], with typical reports of 40–90% fewer reasoning tokens at equal or better accuracy on mathematics benchmarks. Recent work allocates budgets across sub-questions or across a batch of questions [62]–[67]. Earlier, planning-oriented work combined fast and slow modes in a single transformer [68], [69], and the AI community has framed the agenda explicitly in dual-process terms [70], [71].

4) Routing and cascades: Routing between a cheap and an expensive model on predicted difficulty [72], [73] or cascading on the weak model's answer consistency [74], [75] reduces cost by 40–98% at matched quality in the cited studies. The classical foundations are Chow's reject option [76], selective classification with risk guarantees [77] and learning to defer [78], [79]. Calibration of the routing signal is the crux [80]–[82]; conformal methods give distribution-free coverage when a held-out set is available [83].

5) Speculative and parallel decoding: Speculative decoding drafts with a cheap model and verifies in parallel with the target, preserving the target distribution [84], [85]; Medusa and EAGLE replace the draft model with extra heads [86], [87], and multi-token prediction heads serve the same purpose [88]. Learned per-token compute allocation [89], [90] is the architectural cousin of our per-request gate. In Nafs-Aql, speculative decoding is an *inside-the-model* accelerator, orthogonal to the System 1/System 2 split; llama.cpp's n-gram speculation needs no draft model and is the only variant we plan on the CPU baseline (Section V).

6) Autoregressive and non-autoregressive Qwen: Non-autoregressive generation began in translation [91], [92] and is now pursued as discrete diffusion [93]–[95]. LLaDA trains an 8B masked-diffusion model from scratch [96]; Dream 7B and Dream-Coder are initialised from Qwen2.5 weights [97]; DiffuCoder adapts Qwen2.5-Coder [98]; block diffusion interpolates between the two paradigms [99], and SDAR and TraDo convert Qwen3 base models (1.7B–30B-A3B) into block-diffusion reasoners [100], [101]; Fast-dLLM v2 converts Qwen2.5 with about a billion tokens [102]; training-free caching and parallel decoding give large throughput gains [103]; commercial diffusion models report thousands of tokens per second on GPUs [104]; see the survey [105]. Jacobi-style parallel decoding of ordinary autoregressive models is a third route [106], [107]. **Verification result:** as of October 2026 there is no official non-autoregressive Qwen model from the Qwen team; every Qwen-based diffusion model is a third-party adaptation, and only the Dream family runs in llama.cpp's diffusion example on CPU. Diffusion decoding is therefore *not* on the critical path of Nafs-Aql; it is an optional System 1 accelerator evaluated in Phase 5, and only if a CPU-capable runtime exists for a Qwen3-derived checkpoint by then.

TABLE I
QWEN GENERATIONS AND OPEN TEXT MODELS ≤ 9B (VERIFIED ON HUGGING FACE, 2026-10-18)

Generation	Released	Open $\leq$9B	Architecture, notes
Qwen3	Apr 2025; 2507 refresh	0.6B, 1.7B, 4B, 8B; 4B-Instruct-2507, 4B-Thinking-2507	Dense GQA (`Qwen3ForCausalLM`); thinking switch; official GGUF; Apache-2.0
Qwen3.5	Feb–Mar 2026	0.8B, 2B, 4B, 9B	Hybrid 3:1 Gated DeltaNet : gated attention; vision encoder built in; 262K context; no official GGUF; Apache-2.0
Qwen3.6	Apr 2026	none (27B, 35B-A3B)	Same hybrid family
Qwen3.7	May 2026	none (API only)	Closed
Qwen3.8	Aug 2026	none (27B, 2.4T-A95B, Flash-Next)	Hybrid; new `qwen4_exp` for Flash-Next

7) Qwen generations: Table I summarises the Qwen generations available in October 2026. Only Qwen3 [52] and Qwen3.5 [108] have open weights at or below 9B parameters; Qwen3.6 and Qwen3.8 start at 27B dense or 35B-A3B [109]–[111], and Qwen3.7 has no open weights. The draft design's roster (Qwen3.6-27B and Qwen3.6-35B-A3B) is therefore real but outside any CPU budget considered here (about 16–22 GB at 4 bits).

III. MATHEMATICAL FORMULATION

This section turns the inspiration of Section II into a formal model. Nothing here depends on the metaphysical reading: the objects are policies, verifiers, costs and losses. Each result ends with its *engineering consequence*, which the specification (Section VII) turns into a parameter or an algorithm. Results are labelled by the strength of their assumptions: **[exact]** holds under the stated model without further hypotheses; **[strong]** needs an assumption that must be checked empirically; **[conjecture]** is a design hypothesis scheduled for validation in the roadmap.

A. Objects

Definition 1 (Task, answer, correctness)**.** A *task* is $x \in \mathcal{X}$ (a query together with its conversational context). An *answer* is $y \in \mathcal{Y}$. Correctness is an unobserved indicator $\kappa(x, y) \in \{0, 1\}$. A task is *verifiable* if a deterministic procedure can evaluate κ (unit tests, numeric or symbolic equality, schema validity); otherwise it is *non-verifiable*.

Definition 2 (Fast and slow paths)**.** The *fast path* (System 1, the analogue of the impulsive *nafs*) is a stochastic policy $\pi_1(y \mid x)$ with expected cost $c_1(x)$ and correctness probability $p_1(x) = \mathbb{P}_{y\sim\pi_1}[\kappa(x, y) = 1]$. The *slow path* (System 2, deliberation) is a policy $\pi_2(y \mid x, b)$ parametrised by a *thinking budget* $b \in \mathbb{N}$ (maximum reasoning tokens), with cost $c_2(x, b)$ and correctness $p_2(x, b)$. Costs are measured in a common unit (seconds of CPU time or tokens weighted by a tariff).

Definition 3 (Verifier)**.** A *verifier* is a map $V : \mathcal{X}\times\mathcal{Y} \to \{0, 1\}$ with true-accept rate $r = \mathbb{P}[V = 1 \mid \kappa = 1]$ and false-accept rate $\varphi = \mathbb{P}[V = 1 \mid \kappa = 0]$. A verifier is *exact* if $\varphi = 0$ and $r = 1$ on its domain. Soft verifiers (rubric judges, consensus) have $\varphi > 0$ and must publish measured (r, φ).

Definition 4 (Gate and trace state)**.** After the fast path has run, the *controller* (the *'aql* analogue) observes a feature vector $s = \sigma(x, \text{trace}_1) \in \mathcal{S}$ (token log-probabilities, agreement among cheap samples, verifier outcome, domain class) and chooses, with probability $\gamma(s) \in [0, 1]$, to *escalate* to the slow path. The lifecycle of one request is a finite automaton over the *trace states* $\mathcal{Z} = \{\mathsf{A}, \mathsf{L}, \mathsf{M}, \mathsf{T}, \mathsf{X}\}$ of Fig. 3: A (an unverified draft), L (the draft has been flagged by the gate or a verifier), M (a deliberate retry conditioned on evidence), T (an accepted answer) and X (abstention with an explicit uncertainty notice). The labels are mnemonic for the four classical stages of the *nafs* (Section II-A); the automaton itself is an ordinary bounded retry loop.

Definition 5 (Losses and objective)**.** A wrong answer costs $L > 0$, an abstention costs $L_a \in (0, L)$, and compute is priced by $\lambda > 0$ per cost unit. For a controller $\theta = (\gamma, b, N, R)$ (gate, thinking budget, samples per round, retry rounds) the design objective is

$$\mathcal{J}(\theta) = \mathbb{E}_x\big[L\,\mathbb{P}[\text{wrong} \mid x, \theta] + L_a\,\mathbb{P}[\text{abstain} \mid x, \theta] + \lambda\,\mathbb{E}[\text{cost} \mid x, \theta]\big]. \tag{1}$$

Lemma 1 (Termination)**.** [exact] *With at most N slow-path samples per round and at most R retry rounds, every request makes at most $1 + N(R{+}1)$ model calls and at most $1 + N(R{+}1)$ verifier calls before reaching* T *or* X*.*

Proof. Each transition of the automaton either consumes one of the $N(R{+}1)$ slow samples or terminates; the fast path is called once. □ □

Consequence. Every request has a hard upper bound on latency (NFR-001 in the specification); the gateway MUST enforce N and R as configuration limits rather than letting the model decide when to stop.

B. Precision of a verified answer

Theorem 1 (Verified-return precision)**.** [exact] *Let a candidate be correct with prior probability p and let it be accepted by a verifier with rates (r, φ). Then*

$$\rho(p) \;=\; \mathbb{P}[\textit{wrong} \mid \textit{accepted}] = \frac{\varphi(1-p)}{\varphi(1-p) + r\,p}, \tag{2}$$

and $\rho(p) \le \varepsilon$ if and only if $\varphi \le \dfrac{\varepsilon\, r\, p}{(1-\varepsilon)(1-p)}$. If k verifiers are applied and their acceptances are conditionally independent given κ, the same holds with $\varphi \leftarrow \prod_i \varphi_i$ and $r \leftarrow \prod_i r_i$.

Proof. Bayes' rule with $\mathbb{P}[\text{accepted}] = \varphi(1-p) + rp$. Solving $\rho \le \varepsilon$ for φ gives the bound. Conditional independence factorises the joint acceptance probabilities. □ □

Consequence. The precision of the system on verifiable tasks is bounded by the verifier's false-accept rate, not by the strength of either model. Every verifier in the hub MUST publish (r, φ) measured by mutation testing on a held-out set (FR-030), and the per-verifier acceptance threshold is 1.0 for exact verifiers. For hard tasks ($p = 0.1$) a 1% precision target requires $\varphi \le 1.1 \times 10^{-3}$: only exact verifiers qualify, which is why consensus (Theorem 6) is a flag and not a verifier.

C. The gate is a threshold rule

Let $e_1(s) = \mathbb{P}[\text{fast answer wrong} \mid s]$, $e_2(s) = \mathbb{P}[\text{final answer wrong after escalation} \mid s]$ and $\Delta c(s) = \mathbb{E}[\text{extra cost of escalation} \mid s]$.

Theorem 2 (Optimal gate)**.** [exact]

(i) The gate minimising (1) *escalates if and only if*

$$L\,\big(e_1(s) - e_2(s)\big) \;>\; \lambda\,\Delta c(s), \tag{3}$$

with ties broken arbitrarily.

(ii) Under a budget constraint $\mathbb{E}[\gamma(s)\Delta c(s)] \le B$ instead of the price λ, the optimal gate escalates if and only if $\big(e_1(s) - e_2(s)\big)/\Delta c(s) > \lambda^\star$, where $\lambda^\star$ is the smallest multiplier that meets the budget; randomisation is needed only on the level set $\{s : (e_1 - e_2)/\Delta c = \lambda^\star\}$.

(iii) If $e_1 - e_2$ is non-decreasing in a scalar score s and Δc is constant, the rule is a single threshold $s > \tau$.

Proof. (i) Conditional on s, keeping costs $Le_1(s)$ and escalating costs $\lambda\Delta c(s) + Le_2(s)$; the objective is an expectation over s of a pointwise choice, so pointwise minimisation is optimal. (ii) The Lagrangian $\mathbb{E}[\gamma(s)(Le_2 + \lambda\Delta c - Le_1)]$ is linear in γ; by the Neyman–Pearson lemma the optimal γ is an indicator of a likelihood-ratio set with randomisation on the boundary. (iii) Immediate from (i). □ □

Consequence. The controller is implemented as a calibrated estimator $\hat{e}_1(s)$ (logistic or isotonic regression on the fast-path features), a per-domain estimate $\hat{e}_2$, and the comparison (3). The only tunable is the cost ratio $\lambda\Delta c/L$ (or equivalently the budget B), which the operator sets; the threshold is *derived*, not hand-tuned (FR-010–FR-013). The rule coincides with

Chow's optimal reject rule [76] and with the expected-value-of-control account of cognitive control [43], with "reject" read as "deliberate".

Proposition 1 (Price of miscalibration)**.** [exact] *If the plug-in gate uses estimates with* $\sup_s |\hat{e}_1(s) - e_1(s)| \le \epsilon_1$ *and* $\sup_s |\hat{e}_2(s) - e_2(s)| \le \epsilon_2$*, its objective exceeds that of the optimal gate by at most* $L(\epsilon_1 + \epsilon_2)$*.*

Proof. At any s the two gates disagree only if $|L(e_1 - e_2) - \lambda\Delta c| \le L(\epsilon_1+\epsilon_2)$, and the loss from a wrong decision equals that absolute difference. Taking expectations over s gives the bound. □ □

Consequence. The calibration target of the gate (NFR-020) is set from the regret the operator tolerates: a maximum calibration error of 0.05 on $\hat{e}_1$ bounds the excess loss by $0.05L$ per request.

Proposition 2 (Cost–quality frontier)**.** [exact] *The set of achievable pairs (expected cost, expected error) over all gates* γ *is convex, and its lower-left boundary is traced by the family of threshold gates of Theorem 2 as* λ *varies.*

Proof. Both coordinates are linear in γ, so mixtures of gates achieve mixtures of pairs; the boundary of a convex set is supported by linear functionals, each of which is minimised by a threshold gate by Theorem 2. □ □

Consequence. The user-facing "speed–care" control in the chat UI is a single scalar λ (FR-014); no other knob can reach a better point.

D. Deliberation: sampling, stopping and retries

Let a slow-path round draw one candidate and verify it; let $a = q\,r + (1-q)\varphi$ be the probability that a draw is accepted, where q is the probability that a draw is correct.

Theorem 3 (Coverage and cost of sequential verified sampling)**.** [exact] *With at most* M *draws and stopping at the first acceptance, (i) the probability that some candidate is accepted is* $1-(1-a)^M$*; (ii) the expected number of draws is* $E(M) = \big(1-(1-a)^M\big)/a$*; (iii) the conditional error of the accepted candidate is* $\rho(q)$ *of Theorem 1, independent of* M*; (iv) if* a *varies across tasks with mean* $\bar{a}$*, the coverage is at most* $1-(1-\bar{a})^M$*.*

Proof. (i)–(ii) are the truncated geometric distribution. (iii) Each accepted candidate, whichever draw produced it, is wrong with probability $\rho(q)$ by Bayes' rule. (iv) $u \mapsto (1-u)^M$ is convex, so Jensen's inequality gives $\mathbb{E}[(1-a)^M] \ge (1-\bar{a})^M$. □ □

Consequence. More samples buy *coverage*, never *precision*: the number of draws is chosen from a coverage target and the per-task heterogeneity of a must be measured, not averaged (Phase 0 of the roadmap).

Proposition 3 (Optimal number of draws under uncertainty about a)**.** [strong: Beta prior, monotone stopping] *Suppose the acceptance probability of a task is unknown with prior* $a \sim \mathrm{Beta}(\alpha,\beta)$*, each draw costs* c_p*, and failing to produce an accepted candidate costs* L_a*. After* m *consecutive rejections the posterior mean is* $a_m = \alpha/(\alpha+\beta+m)$*. The policy that draws again if and only if* $a_m L_a > c_p$ *is optimal, so the optimal cap is*

$$M^\star = \left\lceil \frac{\alpha L_a}{c_p} - \alpha - \beta \right\rceil \tag{4}$$

(the number of $m \ge 0$ *with* $\alpha/(\alpha+\beta+m) > c_p/L_a$*), when the right-hand side is positive, and* $M^\star = 0$ *otherwise.*

Proof. A further draw costs c_p and, with posterior-predictive probability a_m, saves L_a; so the one-step look-ahead rule continues iff $a_m L_a > c_p$. Because every continuation happens only after a rejection and a_m is decreasing in m, once the rule says "stop" it says "stop" in all subsequent states: the problem is monotone, and in the monotone case the one-step look-ahead rule is optimal [112]. The draws made are exactly those indexed by the $m \ge 0$ with $m < \alpha L_a/c_p - \alpha - \beta$, whose count is the ceiling. □ □

Consequence. The retry limit R and the samples per round N (with $M = N(R+1)$) are derived per domain from the Phase-0 fit of (α,β) and the operator's cost ratio c_p/L_a (FR-021). The Beta model is a modelling choice; the roadmap validates it against the empirical distribution of acceptance rates (hypothesis H3).

E. Self-monitoring: why the loop must be verifier-gated

The classical *muhasaba* (self-accounting) is a loop in which a draft is re-examined and amended. Let u be the probability that one unverified refinement turns a wrong answer into a right one and d the probability that it turns a right answer into a wrong one.

Theorem 4 (Unverified refinement converges to a fixed point)**.** [strong: homogeneous Markov refinement] *If* q_t *is the probability that the* t*-th draft is correct and* $q_{t+1} = q_t(1-d)+(1-q_t)u$*, then* $q_t = q^\star + (q_0 - q^\star)(1-u-d)^t$ *with* $q^\star = u/(u+d)$*. In particular* q_t *decreases monotonically in* t *whenever* $q_0 > q^\star$*.*

Proof. The recursion is affine with fixed point $q^\star$ and contraction factor $1-u-d \in (-1,1)$ for $u+d \in (0,2)$. □ □

Consequence. Iterating self-critique without external evidence can only help tasks whose first draft is worse than the fixed point; for a strong first draft it *harms*. This is consistent with the empirical finding that language models do not reliably self-correct without external feedback [55], [56]. Nafs-Aql therefore never accepts an unverified refinement: on non-verifiable tasks the loop is capped at $R = 1$ (FR-022).

Theorem 5 (Verifier-gated loop is stable)**.** [exact] *In a loop that accepts a candidate only when a verifier with rates* (r,φ) *passes it, and whose* t*-th candidate is correct with probability* q_t*, the probability of terminating with an accepted wrong answer is at most* $\varphi\,\mathbb{E}[T]$*, where* T *is the number of candidates examined, and the error conditional on acceptance lies in* $[\min_t \rho(q_t), \max_t \rho(q_t)]$*. Neither bound depends on the refinement dynamics* (u,d)*.*

Proof. A wrong candidate at round t is accepted with probability φ; summing $\mathbb{P}[\text{reach } t]\,(1-q_t)\varphi \le \varphi\mathbb{P}[\text{reach } t]$ over t gives $\varphi\,\mathbb{E}[T]$. Conditional on acceptance at round t the error is $\rho(q_t)$ by Theorem 1; the overall conditional error is a mixture of these values. □ □

Consequence. With a verifier in the loop the risk is linear in φ and in the expected number of rounds, both of which the controller bounds (Lemma 1); the refinement prompt can be changed freely without re-deriving the safety case.

F. Consensus has a floor

Theorem 6 (Plurality voting converges to the modal answer)**.** [exact] *Let N answers be drawn i.i.d. from $\pi(\cdot \mid x)$ over a finite answer set with a unique mode $y^\circ(x)$. Then the plurality vote converges almost surely to $y^\circ(x)$ as $N \to \infty$, so the limiting accuracy of self-consistency is $\mathbb{P}_x[\,y^\circ(x)$ is correct$\,]$. For finite N, if the correct answer has mass q and the strongest wrong answer has mass $\bar{q} < q$, the probability that the wrong answer ties or beats the correct one is at most $\exp\big(-N(q-\bar{q})^2/2\big)$.*

Proof. The empirical frequencies converge almost surely to $\pi(\cdot \mid x)$ (strong law), so the arg-max converges to the unique mode. For the finite bound let $D_i = \mathbb{1}[y_i = y^\star] - \mathbb{1}[y_i = \bar{y}] \in [-1,1]$ with mean $q-\bar{q}$; Hoeffding's inequality [113] gives $\mathbb{P}[\sum_i D_i \le 0] \le \exp(-2N(q-\bar{q})^2/4)$. □ □

Consequence. Agreement among samples cannot repair tasks on which the model is confidently wrong; the mass of such tasks is a floor that no N removes. Consensus is therefore recorded as an *assurance level* (`consensus`) distinct from `verified` (FR-031), is never used to write training data without a human check, and the number of samples for a consensus flag is set from the margin $q-\bar{q}$ measured in Phase 0.

G. Independence of the two paths

Proposition 4 (Anchoring and collision)**.** [exact] *If the slow path does not see the fast draft, the probability that both produce the same wrong answer is the collision probability $\sum_{y:\kappa(x,y)=0} \pi_1(y \mid x)\pi_2(y \mid x)$. Passing the draft to the slow path can only add anchoring on top of this value. Certifying an adoption rate below 5% at confidence 95% with zero observed adoptions needs at least $\lceil \ln 0.05 / \ln 0.95 \rceil = 59$ trials; with observed adoptions the Clopper–Pearson interval [114] applies.*

Consequence. The slow path is *draft-blind* by default (FR-023); draft-conditioned deliberation is an ablation, and the coupling test (hypothesis H5) is a release gate. Because both paths may share weights (the single-model profile of Section IV-E), the collision term is not small a priori and is measured.

H. Honest self-knowledge under routing

Proposition 5 (Unbiased capability estimates need audits)**.** [exact] *Slow-path correctness is observed only on escalated tasks, so the naive estimate is $\mathbb{P}[$slow correct $\mid$ escalated$]$, a biased estimate of p_2. If a random fraction $\epsilon > 0$ of the kept tasks is also run through the slow path, then with escalation rate g, $\hat{p}_2 = g\,\hat{p}_{2|\text{esc}} + (1-g)\,\hat{p}_{2|\text{audit}}$ is unbiased, with standard error at most $\frac{1}{2}\sqrt{\big(g+(1-g)/\epsilon\big)/n}$ over n requests.*

Proof. Inverse-probability weighting with known strata weights g and $1-g$ and sampling probabilities 1 and ϵ; the variance of each stratum mean is at most $1/(4\,n_{\text{stratum}})$ with $n_{\text{esc}} = ng$ and $n_{\text{audit}} = n(1-g)\epsilon$. □ □

Consequence. The self-model (the *muhasaba* ledger) is built only from traces plus a 5% audit stream (FR-040); capabilities with no audited data are reported as *unmeasured*, never as zero. With $n = 10^4$, $g = 0.3$ and $\epsilon = 0.05$ the standard error is at most 0.019.

I. Estimation and gate sample sizes

Proposition 6 (Sample sizes)**.** [exact]

(i) (Threshold from a quantile.) If τ is set as the empirical $(1-g^\star)$-quantile of n i.i.d. scores, the realised escalation rate is within ε of $g^\star$ with probability at least $1-\delta$ whenever $n \ge \ln(2/\delta)/(2\varepsilon^2)$ [115].

(ii) (Non-inferiority.) To show that the cascade loses at most m points of accuracy against the slow-only baseline with power $1-\beta$ at level α, a paired design with discordance rate δ needs $n \ge (z_\alpha + z_\beta)^2\,\delta/m^2$ pairs.

(iii) (Binomial gates.) Certifying a rate below ε with zero failures at confidence $1-\delta$ needs $n \ge \ln\delta/\ln(1-\varepsilon)$ trials.

Consequence. The Phase-0 baseline is at least $4\,600$ tasks ($\varepsilon = 0.02$, $\delta = 0.05$), the one-point non-inferiority gate needs about $6\,200$ pairs at $\delta = 0.1$ (or 700 for a three-point margin), and every release gate in the roadmap states its sample size. With $n = 500$ the escalation rate is known only to ± 6 points.

J. Distillation (tazkiya) without self-deception

Proposition 7 (Label noise and verifier-inflated gains)**.** [exact]

(i) If a fraction η of training labels is wrong, then for every hypothesis h, $|\text{err}_{noisy}(h) - \text{err}_{clean}(h)| \le \eta$; with verifier-filtered traces $\eta = \rho$ of Theorem 1.

(ii) If accuracy is measured with a verifier of rates (r, φ), the measured rate is $\varphi + (r-\varphi)\,t$ where t is the true rate; a measured gain Δ corresponds to a true gain $\Delta/(r-\varphi)$, and training on verifier-accepted outputs can raise φ and inflate the measurement.

(iii) A paired promotion gate with discordance δ resolving a gain of ε needs $n \ge z^2\delta/\varepsilon^2$ pairs per domain.

Proof. (i) The two error events differ only on examples whose label is wrong. (ii) Expand $\mathbb{P}[V=1] = r\,t + \varphi(1-t)$. (iii) Normal approximation to the paired binomial. □ □

Consequence. Promotion of a distilled fast-path adapter requires a confidence-bound gain on a verifier that was never used for data collection, plus a per-domain non-regression bound and human sign-off (FR-050–FR-053).

K. What a CPU can serve

Proposition 8 (Decode and prefill ceilings)**.** [exact given the roofline model] *For a dense decoder whose active weights occupy W bytes and whose key–value cache at context length ℓ occupies $K(\ell)$ bytes, the steady-state decode rate on a machine with memory bandwidth β is at most $\beta/(W + K(\ell))$ tokens per second, and the prefill rate on a machine with peak throughput F FLOP/s and P active parameters is at most $F/(2P)$ tokens per second.*

Proof. Each decoded token must stream every active weight and the cache from memory at least once; each prefill token requires at least $2P$ floating-point operations. □ □

Consequence. On the reference machine (dual-channel DDR4-2400, $\beta \approx 38\,\text{GB/s}$; AVX2, four cores) a 4-billion-parameter model in 4-bit GGUF ($W \approx 2.5\,\text{GB}$) cannot exceed about 15 tokens/s, a 1.7B model about 35 tokens/s, and a 1000-token prefill of the 4B model takes seconds to tens of seconds. These ceilings fix the model sizes, the thinking-budget cap, the context cap and the prompt-cache requirement (NFR-002–NFR-006). Measured values on the reference machine are reported in Section V-D and are below the ceilings, as expected.

Proposition 9 (Thinking budget)**.** [strong: concave returns] *If $p_2(x, b)$ is concave and non-decreasing in b and the cost is linear, $c_2(x, b) = c_2^0 + \kappa_{tok}\, b$, the budget minimising $L(1 - p_2(x, b)) + \lambda c_2(x, b)$ is the unique $b^\star$ with $L\,\partial_b p_2(x, b^\star) = \lambda\kappa_{tok}$ (or the boundary if no interior solution exists).*

Consequence. The thinking budget is a per-domain configuration value estimated from the Phase-0 accuracy-versus-budget curve (hypothesis H4); when the budget is exhausted the controller forces the answer by closing the reasoning span [51] rather than returning nothing.

L. End-to-end identity

Proposition 10 (Cascade difference)**.** [exact] *Let ρ_1 be the error on tasks the gate keeps and $\bar{e}_2^{\,kept}$ the error the slow path alone would make on those tasks. If the slow path behaves identically on escalated tasks in both systems, $\mathbb{P}[\textit{cascade wrong}] - \mathbb{P}[\textit{slow-only wrong}] = (1 - g)\big(\rho_1 - \bar{e}_2^{\,kept}\big)$.*

Consequence. Non-inferiority within m requires $\rho_1 \le \bar{e}_2^{\,\text{kept}} + m/(1 - g)$; $\bar{e}_2^{\,\text{kept}}$ is observable only through the audit stream of Proposition 5, which is therefore part of the safety case.

M. Summary of engineering consequences

Table II lists each result with the parameter or algorithm it determines; the specification's traceability matrix repeats the mapping by requirement ID.

TABLE II
FORMAL RESULTS AND WHAT THEY DETERMINE

Result	Engineering consequence
Lemma 1	Hard caps N, R; latency bound
Thm. 1	Verifier (r, φ) publication; exact-verifier threshold 1.0
Thm. 2	Gate = calibrated $\hat{e}_1$, $\hat{e}_2$, cost ratio; no hand-tuned τ
Prop. 1	Calibration-error target of the gate
Prop. 2	Single "speed–care" scalar in the UI
Thm. 3	N from coverage target; measure heterogeneity of a
Prop. 3	R, N from Beta fit and c_p/L_a
Thm. 4	No unverified refinement; $R = 1$ on non-verifiable tasks
Thm. 5	Loop risk $\le \varphi\,\mathbb{E}[T]$; prompt changes need no new safety case
Thm. 6	Consensus is an assurance flag; N from margin
Prop. 4	Draft-blind slow path; coupling test gate
Prop. 5	5% audit stream; "unmeasured" label
Prop. 6	Phase-0 size $\ge 4\,600$; gate sample sizes
Prop. 7	Promotion gate: held-out verifier, confidence bounds
Prop. 8	Model sizes, context cap, prompt cache, budget cap
Prop. 9	Per-domain thinking budget; budget forcing
Prop. 10	Audit stream is part of the safety case

IV. Architecture and Design

A. Design Principles

Four principles follow from Section III. (1) *The gate is derived, not tuned*: the controller compares a calibrated error estimate with a cost ratio (Theorem 2). (2) *Acceptance needs evidence*: no answer is marked verified unless an exact verifier accepted it; consensus and judge scores are assurance levels, not proofs (Theorems 1 and 6). (3) *Every loop is bounded*: N, R and the thinking budget are configuration limits enforced by the gateway (Lemma 1). (4) *Self-knowledge is audited*: the self-model is built from traces plus a random audit stream and reports unmeasured capabilities as such (Proposition 5).

B. Concept-to-Component Mapping

Fig. 1 gives the mapping. The orchestrator (`nafs-core`) plays al-Ghazali's king: it owns the request and consults the controller. The controller (Aql Gate) plays the vizier: it arbitrates between the fast draft and deliberation on a cost–risk basis. The fast path (System 1) is the impulsive draft; the slow path (System 2) is deliberation; the Verifier Hub supplies external evidence; the Muhasaba Ledger performs the nightly accounting; Tazkiya Distillation is the long-run change of disposition. The gate's cheap features play the role of Avicenna's estimative faculty, a fast particular judgement that is useful and error-prone, which is exactly why it is calibrated and audited.

C. Request Lifecycle

Fig. 2 shows one request and Fig. 3 its state automaton. The Context Manager assembles the prompt from the conversation and any retrieved memory with provenance. System 1 produces a draft in non-thinking mode together with k cheap samples in one request (the `n` parameter of the OpenAI-compatible API, so the prompt is prefilled once). The Aql Gate

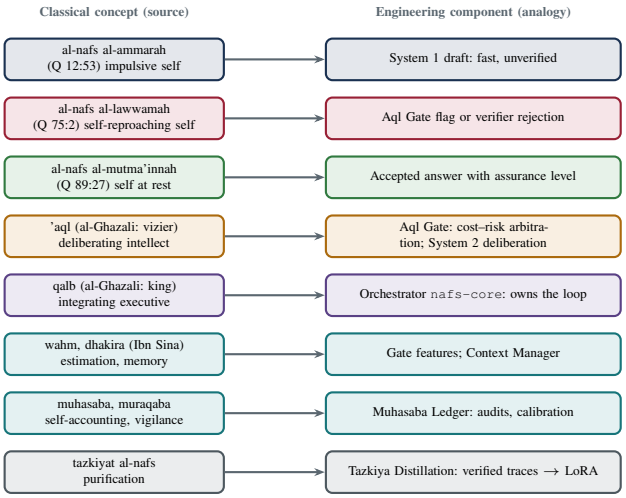


Fig. 1. Concept-to-component mapping. Every arrow is an analogy; component names in code are the engineering names on the right.

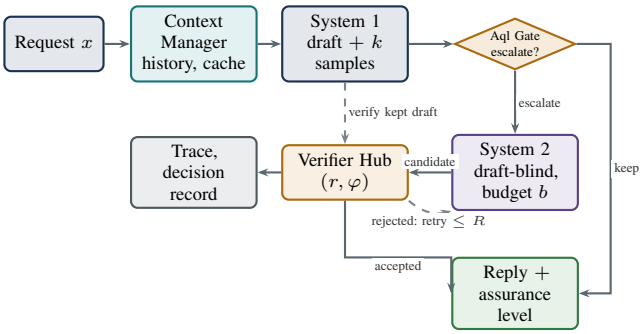
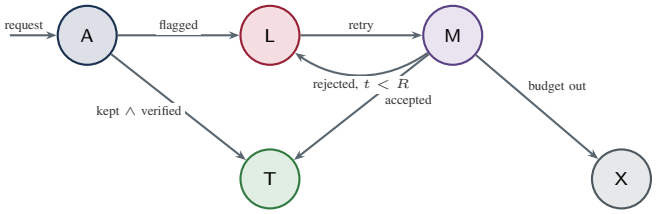


Fig. 2. Request lifecycle. Solid arrows are the main path; dashed arrows are the verification and retry edges.

computes features: mean and minimum token log-probability of the draft, agreement among the k samples, the domain class, and, for verifiable tasks, the verifier outcome on the draft. A calibrated model maps the features to $\hat{e}_1$; the gate escalates if $L(\hat{e}_1 - \hat{e}_2) > \lambda \Delta c$. A verifiable task whose draft fails verification is never eligible for the keep branch (this closes the draft's defect 10, where the router pseudo-code disagreed with its prose). On escalation, System 2 runs draft-blind in thinking mode with budget b and sequential sampling: each candidate is verified and the first accepted one is returned; a rejected candidate's evidence (failing test, counterexample, schema error) is fed into the next round, up to R rounds. If the budget is exhausted the reply is an explicit abstention with the best candidate and an uncertainty notice. Every reply carries an *assurance level*: `verified` (exact verifier passed), `consensus` (soft agreement only), `unverified` (kept fast draft on a non-verifiable task) or `abstained`. The decision record of the gate and the trace of the loop are written for the Muhasaba Ledger.

D. Components

- **Rust Gateway (`nafs-gateway`).** HTTP API (OpenAPI 3.1), SSE streaming of tokens and of controller events, WebSocket chat rooms, authentication, rate limits, Prometheus metrics.
- **Orchestrator and Aql Gate (`nafs-core`).** Implements the automaton, the gate, sequential verified sampling, budget forcing, timeouts and fallbacks; emits the decision record.
- **System 1 and System 2 backends.** Two OpenAI-compatible model servers (or one server with two modes in the single-model profile), reached over HTTP with retry and backpressure.
- **Verifier Hub (`nafs-verify`).** Exact verifiers: code sandbox (subprocess with CPU, memory, wall-clock and network limits), numeric and symbolic mathematics (reference-answer equality, SymPy via a sidecar), JSON Schema validation, format checks. Soft verifiers: rubric judge (the System 2 model in non-thinking mode at temperature 0) and sample consensus. Each verifier publishes (r, φ) from mutation tests.
- **Context Manager (`nafs-memory`).** Conversation store (SQLite), prompt assembly within a context cap, prompt-cache affinity hints for the model server, episodic trace store (JSONL), and a retrieval index built with Qwen3-Embedding-0.6B when retrieval is enabled.
- **Muhasaba Ledger.** Offline job that aggregates traces and the 5% audit stream into per-domain calibration tables (escalation rate, $\hat{e}_1$ calibration error, verifier acceptance rates, latency quantiles), keyed by model and adapter hash; feeds the dashboard and the gate's recalibration.
- **Eval Harness (`nafs-eval`) and performance console (`nafs-bench`).** Dataset loaders, metric definitions, paired statistical tests, load generation, and report export.
- **Web UI (`nafs-web`).** Chat room with the assurance level and the reasoning mode visible per message, a performance-test console, and an evaluation dashboard; server-rendered HTML with htmx for streaming updates.
- **Tazkiya Distillation (Python, GPU).** Dataset builder from verified traces, Unsloth LoRA training, promotion gate, GGUF export.



A draft (fast path); L flagged by gate or verifier; M deliberate retry conditioned on verifier evidence; T accepted (assurance level attached); X abstain with uncertainty notice.

Fig. 3. Trace-state automaton of one request. The three Qur'anic stage names are used as mnemonics for A, L and T; the automaton is a bounded retry loop (Lemma 1).

E. Model Profiles

Table III defines three profiles. Profile A is the reference: two dense Qwen3 checkpoints, one per role, chosen because dense grouped-query attention has the most mature CPU kernels, official GGUF files exist, and both are on vLLM's

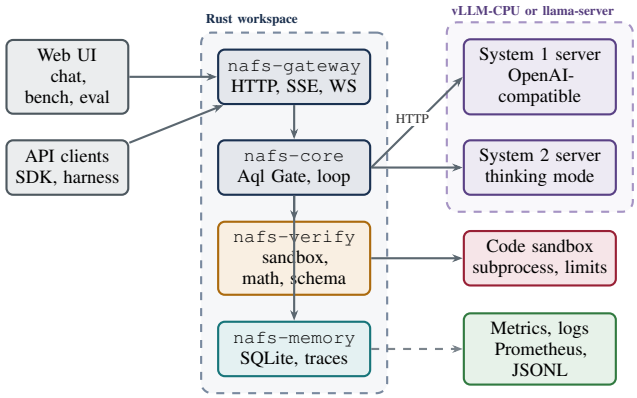


Fig. 4. System architecture, single-node CPU baseline. The GPU scale-out variant replaces the model servers with vLLM on CUDA and keeps every other component unchanged.

TABLE III
MODEL PROFILES (HUGGING FACE IDS VERIFIED 2026-10-18; SIZES ARE GGUF FILE SIZES)

Profile	System 1	System 2
A (ref.)	`Qwen/Qwen3-1.7B`, GGUF Q4_K_M 1.3 GB (`unsloth/Qwen3-1.7B-GGUF`) or Q8_0 1.8 GB (`Qwen/Qwen3-1.7B-GGUF`); thinking off	`Qwen/Qwen3-4B-Thinking-2507`, GGUF Q4_K_M 2.5 GB (`unsloth/Qwen3-4B-Thinking-2507-GGUF`); 262K context
B (min.)	`Qwen/Qwen3-4B` Q4_K_M 2.5 GB (`Qwen/Qwen3-4B-GGUF`), `enable_thinking=false`	same weights, `enable_thinking=true`
C (upgr.)	`Qwen/Qwen3.5-2B` Q4_K_M 1.3 GB (`unsloth/Qwen3.5-2B-GGUF`)	`Qwen/Qwen3.5-4B` Q4_K_M 2.7 GB (`unsloth/Qwen3.5-4B-GGUF`)

Judge: the System 2 model in non-thinking mode. Embeddings: `Qwen/Qwen3-Embedding-0.6B-GGUF` Q8_0. All Apache-2.0. GPU scale-out: the same IDs in BF16 or AWQ under vLLM CUDA, with `Qwen/Qwen3-8B` as the System 2 option.

validated CPU list. Profile B serves one switchable Qwen3-4B for both roles, halving resident memory at the price of a slower fast path; the gate then decides *whether to think*, exactly the decision studied by the adaptive-thinking literature. Profile C is the upgrade path to Qwen3.5, which is stronger and multimodal but uses hybrid linear attention whose CPU kernels are newer and, on AVX2 without AVX-512-BF16 or AMX, unaccelerated in vLLM; Phase 0 of the roadmap benchmarks it against Profile A on the target machine before any commitment. The upgrade decision is a go/no-go gate with a measurable criterion.

F. Deployment Topology

The CPU baseline is a single node: one gateway process, one or two model servers pinned to the physical cores (the gateway reserves none, since the gateway's own CPU use is small and bursty), a sandbox runner, SQLite and JSONL on local disk. The model servers are either `llama-server` (macOS or any x86) or vLLM-CPU (Linux, AVX-512 recommended), both behind the same OpenAI-compatible API, so the Rust code has one backend trait and two thin implementations. The optional GPU scale-out variant moves the model servers to vLLM on CUDA with both models resident, raises N and the thinking budget, and enables EAGLE or multi-token-prediction speculative decoding; the gateway, verifiers, memory and UI are unchanged.

G. What Changed from the Draft

The draft's twelve validation results are retained where exact (precision, coverage, cascade cost, router threshold, consensus, value-score algebra, coupling, audit, measurement floors, distillation noise, capacity, cascade identity) and are restated here with explicit assumptions. We added the miscalibration bound, the frontier proposition, the optimal-stopping cap, the two refinement-loop theorems, the plurality limit, the roofline and the thinking-budget condition. We removed the mapping of the Active Intellect to "external verifiers" and of *ilham* to any component, following the guidance in Section II-A; we replaced the GPU roster with CPU profiles; and we turned the draft's ten defects into requirements with tests in the specification.

V. IMPLEMENTATION STACK

A. Model Serving

vLLM. vLLM [116] is the specified serving layer for the Linux deployment profile. Its x86 CPU backend requires Linux, supports FP32, FP16 and BF16 with AWQ, GPTQ and INT8 W8A8 quantisation, recommends AVX-512 and offers "limited features" on AVX2; pre-built CPU wheels and the `vllm/vllm-openai-cpu` image exist. Qwen3-1.7B, 4B and 8B are on its validated CPU model list. The CPU backend implements hybrid Gated-DeltaNet models, but the accelerated path needs AMX or AVX-512-BF16; elsewhere it falls back to a generic path. At the time of writing the latest published release is v0.31.0 (v0.32.0 is a tag, not a release); the specification pins 0.31.0. vLLM exposes OpenAI-compatible chat completions with `logprobs` and `top_logprobs`, the `n` parameter, a `qwen3` reasoning parser that separates `reasoning_content`, and `chat_template_kwargs` for `enable_thinking`; these are exactly the hooks the gate needs. On the macOS Intel reference machine vLLM runs only inside a Linux container.

llama.cpp. `llama-server` (release v0.7.0, October 2026) is the mandated fallback and the primary engine on the reference machine: GGUF weights, a mature AVX2 path, OpenAI-compatible chat completions with SSE streaming and `logprobs`, `-reasoning-format` to separate reasoning, `-reasoning-budget` for budget forcing, n-gram speculative decoding without a draft model (`-spec-type ngram-simple`), and explicit thread pinning. Both engines present the same HTTP contract, so the Rust gateway has a single backend trait.

Ollama is convenient for demonstrations but its OpenAI endpoint does not return log-probabilities, which the gate requires; it is therefore not a supported backend.

B. Fine-Tuning

Unsloth (2026.10.6) trains with LoRA and QLoRA two to five times faster than baseline PEFT and exports merged

TABLE IV
RUST CRATES (VERSIONS ON CRATES.IO, 2026-10-18)

Crate	Version	Role
tokio	1.53.3	async runtime
axum	0.8.9	HTTP routing, SSE, WebSocket
tower, tower-http	0.5.3, 0.7.1	middleware: timeouts, limits, CORS, compression
hyper	1.12.0	HTTP/1.1 and HTTP/2
reqwest	0.13.5	client to model servers (streaming)
eventsource-stream	0.2.3	SSE parsing of upstream streams
serde, serde_json	1.0.229, 1.0.154	(de)serialisation
schemars	1.2.3	JSON Schema from types
jsonschema	0.59.0	schema verifier
utoipa, utoipa-swagger-ui	6.0.0, 10.0.1	OpenAPI 3.1 generation and docs UI
sqlx	0.9.0	SQLite access, migrations
tracing, tracing-subscriber	0.1.44, 0.3.23	structured logs and spans
opentelemetry, tracing-opentelemetry	0.33.1, 0.34.0	trace export
metrics, metrics-exporter-prometheus	0.24.6, 0.18.3	metrics
askama	0.16.1	HTML templates
clap, figment	4.6.7, 0.10.19	CLI and layered configuration
thiserror, anyhow	2.0.21, 1.0.104	error taxonomy
uuid, chrono	1.28.0, 0.4.45	identifiers, time
governor	0.10.4	rate limiting
criterion, goose	0.8.2, 0.18.1	micro-benchmarks, load tests

weights to GGUF directly. Its core trainer requires a torch accelerator: the device-selection code raises "You need a GPU" on CPU-only hosts, and the documented CPU support covers inference and data preparation, not training. The feasible path is therefore: collect and filter traces on the CPU machine, train on a rented GPU (a 16 GB T4 covers QLoRA on Qwen3-1.7B and 4B [117], [118], and BF16 LoRA on Qwen3.5 up to 4B; Unsloth advises against 4-bit training for Qwen3.5), export GGUF, and run the promotion gate and all evaluation on CPU. Unsloth's default hyperparameters (rank 16–32, $\alpha = r$, learning rate 2×10^{-4}, one to three epochs, effective batch 16, dropout 0, all attention and MLP projections) are the starting point of the training specification.

C. Rust Web Application

Table IV lists the crates and the versions verified on crates.io on 2026-10-18. **axum** on **tokio** and **tower** is the mainstream, well-maintained choice for an async HTTP service with SSE (`axum::response::Sse`) and WebSockets built in; **reqwest** with `eventsource-stream` consumes the model servers' SSE; **serde** and **schemars** give JSON (de)serialisation and JSON-Schema generation from the same Rust types that the OpenAPI document is generated from with **utoipa**; **sqlx** with SQLite gives compile-time-checked queries without a database server; **tracing** with OpenTelemetry export and **metrics-exporter-prometheus** provide observability; **askama** templates with **htmx** keep the front end server-rendered and dependency-light (a single-page framework such as Leptos is an option for a later phase, not a requirement); **goose** provides load generation for the performance console and **criterion** micro-benchmarks. The toolchain is pinned to the stable release current at project start (1.96 on the reference machine; 1.99 is current upstream).

TABLE V
MEASURED THROUGHPUT ON THE REFERENCE MACHINE (LLAMA.CPP, 4 THREADS; MEAN ± S.D. OVER 3 RUNS)

Model (GGUF)	Size	Prefill tok/s	Decode tok/s
Qwen3-0.6B Q8_0	604 MiB	180.6 ± 2.4	32.6 ± 1.8
Qwen3-1.7B Q4_K_M	1.26 GiB	94.1 ± 9.5	15.5 ± 3.4
Qwen3-4B Q4_K_M	2.32 GiB	38.2 ± 0.6	6.7 ± 0.8
Qwen3-4B-Thinking-2507 Q4_K_M	2.32 GiB	38.4 ± 1.5	7.0 ± 0.9

D. Reference Machine and Measured Feasibility

The reference machine is an Intel Core i3-8100 (four cores, no simultaneous multithreading, AVX2 and FMA, no AVX-512), 32 GB of DDR4, macOS 15.4.1. We built llama.cpp from source (commit `797de63`, 2026-10-17) and ran `llama-bench` with four threads, 512-token prefill and 128-token decode, three repetitions. Table V reports the results. The Qwen3-1.7B decode rate of 15.5 tokens/s against a 1.26 GiB weight file implies an effective bandwidth of about 20 GB/s, roughly half the theoretical 38.4 GB/s of dual-channel DDR4-2400, which is in line with the roofline of Proposition 8 and with published AVX2 data points. We also confirmed, through Ollama, that Qwen3-1.7B generates in both thinking and non-thinking modes on this machine; a 512-token thinking cap was exhausted on a simple arithmetic question before the answer was produced, which is the empirical reason the controller forces the answer when the budget is reached (Proposition 9). We also exercised `llama-server` through its OpenAI-compatible API with Qwen3-1.7B: per-token log-probabilities are returned with `logprobs`; the `n` parameter is capped by the number of server slots, so the gate's k cheap samples are issued as k parallel requests against k slots (four 12-token samples completed in 3.0 s with 47 of 48 prompt tokens served from the prompt cache); `enable_thinking` toggles the mode per request; `-reasoning-budget 256` closed the reasoning span after 256 tokens and the model then produced an answer; and SSE streaming delivered chunked deltas. These are the exact hooks the specification relies on. We did not run vLLM on the reference machine because the Docker daemon was not running; the specification's vLLM configuration is taken from its documentation and is marked as untested on this machine.

From these measurements and the roofline we derive the CPU-baseline targets of the specification: a fast-path first token under two seconds for prompts up to 1 000 tokens with prompt caching, a fast-path reply of 150 tokens in about ten seconds, and a slow-path reply with a 1 024-token thinking budget on the 4B model in about two to three minutes; the latter is acceptable for a demonstration and for batch evaluation, and is the reason the roadmap keeps interactive use of System 2 optional until the GPU variant exists.

VI. FEATURES AND CAPABILITIES

- **Dual-process chat.** Every reply shows which path produced it, the assurance level, the thinking budget used and the cost; the user can move a single "speed–care" control, which sets λ (Proposition 2).

- **Verified answers on code, mathematics and structured output**, with precision bounded by the verifier (Theorem 1), and evidence-fed retries (Theorem 5).
- **Calibrated abstention.** When the budget is exhausted the system says so, returns the best candidate with an uncertainty notice, and never upgrades consensus to verification (Theorem 6).
- **Honest self-description.** A per-domain capability table from audited traces, with unmeasured domains marked as such (Proposition 5).
- **Performance console.** Load tests against the gateway with latency quantiles, throughput, escalation rate and cost per request, run from the browser.
- **Evaluation dashboard.** Benchmark runs with paired comparisons against the fast-only and slow-only baselines, confidence intervals, routing accuracy and calibration plots, and the go/no-go status of each roadmap gate.
- **Gradual distillation.** Verified slow-path traces become training data for the fast path under a statistically sized promotion gate with human sign-off (Proposition 7).
- **Backend portability.** The same gateway runs against llama-server on a laptop and vLLM on a server, CPU or GPU.

Honest limits: non-verifiable domains never exceed the `consensus` assurance level; the fast and slow paths share a model family, so their errors are correlated and the collision term of Proposition 4 is measured, not assumed small; on the reference machine the slow path is not interactive.

VII. Specification and Roadmap Summary

The companion *Technical Specification* and *Development Roadmap* are standalone documents built from the same sources as this report; this section summarises them so that the report is self-contained.

A. Specification

The specification uses RFC 2119 keywords and gives every requirement an identifier: functional (FR), non-functional (NFR), interface (IF), model (MR), training (TR), security (SR), operations (OR) and evaluation (ER). The requirement catalogue has 99 entries; each is traced to the result or design decision that motivates it, to the component that implements it and to the test that verifies it. Key normative content:

- **Gate (FR-010–FR-014).** The controller MUST compute the decision of Theorem 2 from a calibrated estimator and a configured cost ratio, MUST never keep a verifiable draft that failed verification, and MUST write a decision record with the features, estimates and thresholds used.
- **Deliberation (FR-020–FR-024).** Sequential verified sampling with configured caps N, R and budget b per domain derived from the Phase-0 fit; draft-blind by default; budget forcing on exhaustion; abstention with notice.
- **Verification (FR-030–FR-033).** Exact verifiers accept at threshold 1.0; every verifier publishes (r, φ); assurance levels are attached to every reply; consensus never writes training data.
- **Self-model (FR-040–FR-042).** A 5% audit stream; unmeasured capabilities reported as unmeasured; statistics keyed by model and adapter hash.
- **Distillation (TR-001–TR-008).** Data schema, filtering, Unsloth LoRA configuration, GPU path, promotion gate with confidence bounds and held-out verifier, human sign-off, GGUF export.
- **Targets (NFR-001–NFR-012).** Hard latency bound from Lemma 1; fast-path first token under two seconds and 150-token reply within twelve seconds at p95 on the reference machine; resident memory under 8 GB for Profile A; reproducible builds; structured telemetry. All targets are labelled as targets.
- **Interfaces (IF-001–IF-010).** OpenAPI 3.1 for the gateway with SSE streaming and a `reasoning_mode` field on every reply; JSON Schemas for the request, reply, decision record and trace; the backend contract to vLLM and llama-server with retry, timeout and backpressure policy.

Machine-readable artefacts accompany the document: `openapi.yaml`, JSON Schemas, a sample vLLM launch configuration and llama-server command, a Cargo workspace skeleton and a sample configuration file, all validated with standard tools.

B. Roadmap

Fig. 5 shows the seven phases and their gates; effort and dates are estimates. Phase 0 (foundations, weeks 0–3) delivers the serving baseline on the reference machine and the $4\,600$-task measurement run. Phase 1 (CPU MVP, weeks 3–8) delivers the gateway, both paths behind a fixed-rule gate, the chat UI and the minimal verifier hub, which is the smallest build that demonstrates the idea end to end. Phase 2 (gate and loop, weeks 8–13) replaces the fixed rule with the calibrated gate and adds the verified retry loop and the audit stream; its exit gate is the non-inferiority test at three points. Phase 3 (fine-tuning, weeks 12–18) runs the first distillation cycle on a rented GPU under the promotion gate. Phase 4 (evaluation suite and performance console, weeks 15–21) completes the dashboard and the load tests. Phase 5 (hardening and optional GPU scale-out, weeks 20–25) adds security hardening, the vLLM CUDA profile and the Qwen3.5 upgrade decision. Phase 6 (release and publication, weeks 25–28) delivers the one-point non-inferiority gate, documentation and the research write-up. Phase 7 (CPU inference performance, added after the first six phases were implemented) attributes latency to stages, tunes the engine under interleaved A/B tests, adds early escalation, and prepares EAGLE-3 speculative heads and repacked weight formats. The MVP is specified with acceptance tests; every phase has measurable entry and exit criteria, a risk register with early-warning signals, and a research-to-engineering table that schedules hypotheses H1–H7.

VIII. Evaluation Plan and Metrics

No result in this section has been measured; the section defines what will be measured, how, and what counts as success. The full definitions, formulas and test plans are in the specification (Section 10 there).

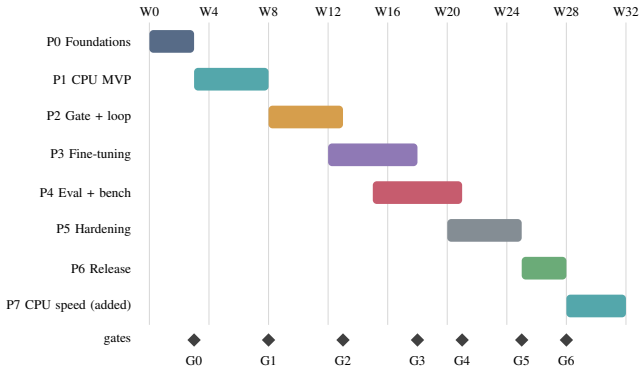


Fig. 5. Roadmap timeline (weeks; estimates). Diamonds are decision gates G0–G6 with go/no-go criteria defined in the roadmap document.

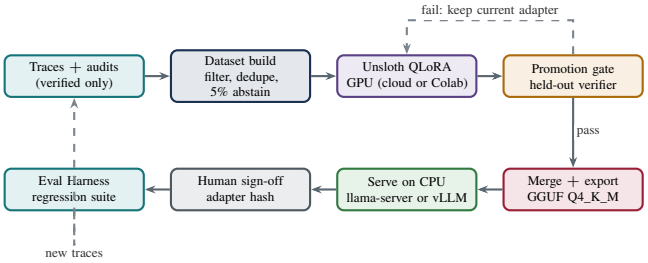


Fig. 6. Training and serving pipeline. Training runs on a rented GPU; everything else runs on CPU.

A. Metrics

- **Quality.** Exact-match or test-pass accuracy on verifiable benchmarks; pairwise win rate against the slow-only baseline by a held-out judge on non-verifiable prompts; abstention rate and accuracy among non-abstained answers (selective accuracy at coverage).
- **Routing.** Escalation rate g; routing accuracy, defined as the fraction of requests on which the gate's decision agrees with the oracle decision of Theorem 2 computed from labelled outcomes; the area under the cost–accuracy curve as λ varies (Proposition 2).
- **Cost and latency.** Tokens and CPU-seconds per request by path; time to first token and total latency at p50, p95 and p99; throughput under load.
- **Calibration.** Expected calibration error of $\hat{e}_1$ with five bins and $n \geq 2\,900$ (the noise floor of a ten-bin estimate at $n = 500$ is 0.034, above the target), and Brier score.
- **Verifier quality.** (r, φ) per verifier from mutation tests, with Clopper–Pearson intervals [114].
- **Faithfulness.** Agreement between the reasoning trace and the answer on perturbation tests [119], [120], and adoption rate of a wrong fast draft by the slow path (Proposition 4).
- **Robustness.** Accuracy under paraphrase and under prompt injection probes [121]–[123].

B. Benchmarks

GSM8K [47] and MATH [124] (mathematics, exact verifiers), HumanEval [125] and MBPP [126] (code, test verifiers), a JSON-Schema task set generated from public schemas

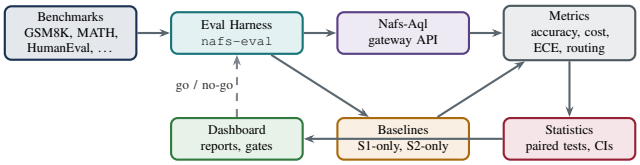


Fig. 7. Evaluation workflow. The harness drives the system and the two baselines through the same API and the dashboard shows gate status.

(format verifier), MMLU and GPQA [127], [128] (knowledge, label verifier), TruthfulQA and SimpleQA [129], [130] (factuality), BIG-Bench Hard [131] and LiveCodeBench [132] as contamination-resistant checks, and MT-Bench style judged conversations [133]. Licences are MIT, Apache-2.0 or CC-BY-4.0 (GPQA is gated). Public routing data from RouteLLM [73] seeds the gate's calibration set.

C. Protocol

Phase 0 runs the fast-only and slow-only baselines on at least 4 600 tasks across domains (Proposition 6) to estimate q, (r, φ), $e_1(s)$, $e_2(s)$, the cost ratio and the discordance rate. Every later comparison is paired on the same tasks, reports the difference with a 95% confidence interval, and uses a non-inferiority margin of three points (about 700 pairs) during development and one point (about 6 200 pairs) for release. Gates for rates that must be below a threshold use the zero-failure binomial rule or the Clopper–Pearson upper bound. Drift in the self-model is monitored with a CUSUM chart [134] and a negative control. Multiple comparisons across five domains use a Bonferroni-corrected $z = 2.326$.

D. Hypotheses Scheduled for Validation

H1: the calibrated gate achieves at least 70% routing accuracy against the oracle at $g \leq 0.3$ (Phase 2). H2: at matched accuracy the cascade costs at most 60% of the slow-only baseline on the Phase-0 suite (Phase 2). H3: per-task acceptance rates are adequately fitted by a Beta distribution per domain, judged by a goodness-of-fit test (Phase 2). H4: slow-path accuracy is concave in the thinking budget on each domain (Phase 0). H5: the draft-blind slow path adopts a wrong fast draft in fewer than 5% of trials at 95% confidence over at least 200 trials (Phase 2). H6: one round of distillation yields a lower-confidence-bound gain above zero on a held-out verifier without a per-domain regression beyond one point (Phase 3). H7: Qwen3.5 hybrid models decode at no less than 70% of the speed of same-size dense Qwen3 models on the target CPU (Phase 0, gate for Profile C).

IX. Implementation and Measured Results

This section reports the system as built and what it measured, as of 18 October 2026. Phases 0–6 of the roadmap are implemented on the CPU reference machine, and a Phase 7 on CPU inference performance was added (Section IX-D). GPU training and GPU scale-out were deferred. Every number below is generated by `scripts/gen_publication.py` from stored runs and evidence files. The machine was shared

with other workloads (load average 6–18), so absolute speeds are lower bounds and small effects are within noise. Sample sizes are far below the release gates of Section VIII, so the results are indicative.

A. The system as built

The Rust workspace has eight crates: the gateway (axum), the orchestrator (`nafs-core`), verifiers with a sandbox, memory (SQLite, BM25 retrieval), evaluation and statistics, load testing, the shared types, and an embedded web interface. The gateway serves 50 documented endpoints (OpenAPI 1.5) and an OpenAI-compatible chat API. System 1 is Qwen3-1.7B and System 2 Qwen3-4B-Thinking-2507, both in Q4_K_M on llama-server; a vLLM-CPU profile runs the same API in a Linux VM or container.

The web interface (Fig. 8) has seven pages:

- **Chat:** streaming answers with the automaton strip (A $\rightarrow$ L $\rightarrow$ M $\rightarrow$ T/X), assurance badges, a one-line reason from the gate, an inspector with the decision record and a per-stage latency waterfall, and "Both" trials in which a person compares the System 1 and System 2 answers;
- **Lab:** experiment presets, paired statistics, the roadmap gates and a pooled policy map;
- **Muhasaba:** gate mode and calibration, verifier quality, the capability ledger, drift and alerts;
- **Tazkiya:** distillation readiness, the review queue and the model registry;
- **Rooms**, **Performance** and **System**.

B. The laboratory

The Lab treats each way of combining the two systems as a *policy* and runs them on identical tasks:

- fast only and deliberate only;
- the gate with the verifier visible, and the gate routing on its own estimate ("blind");
- a draft-conditioned ablation;
- thinking-budget and λ sweeps.

Paired differences carry bootstrap intervals, exact McNemar tests and Bonferroni correction. The *policy map* (Fig. 9) pools every stored run and, for each domain and each price of latency λ, names the policy with the lowest expected loss $L(1-\mathrm{acc})+\lambda c$. Over the 6 stored runs the best policy is, per domain: code: the gate with the verifier for $\lambda \leq 0.1$, System 1 alone for $\lambda \geq 0.2$; math: the gate with the verifier for $\lambda \leq 0.1$, System 1 alone for $\lambda \geq 0.2$; structured: the gate with the verifier at every λ. Below a price of about 0.1 per minute of waiting, deliberation pays for its time on code and mathematics. Above it, the fast path wins, because the verifier already catches most of its errors.

C. Policies, verifiers and model choice

Table VI shows one scaled run of five policies (run `7ffd4442`). With the verifier visible, the gate matched deliberate-only accuracy at 0.36 of its cost. With the verifier hidden, the uncalibrated gate routed correctly on only 50% of items, because cheap samples agreed with wrong drafts: agreement is a flag, not evidence (Theorem 6).

TABLE VI
POLICIES ON A MIXED TASK SET (RUN `7FFD4442`)

System	n	Acc.	95% CI	Esc.	Wall (s)
fast_only	6	67%	[22%, 96%]	0%	13
nafs_aql	6	100%	[54%, 100%]	33%	96
nafs_blind	6	67%	[22%, 96%]	17%	133
slow_conditioned	6	100%	[54%, 100%]	100%	319
slow_only	6	100%	[54%, 100%]	100%	264

TABLE VII
VERIFIER QUALITY: r FROM CORRECT ANSWERS, φ FROM SEEDED MUTANTS

Verifier	Correct	r	Mutants	φ	φ upper
code_tests	364	0.995	754	0.109	0.133
json_schema	24	1.000	53	0.000	0.067
judge	23	1.000	22	0.045	0.228
math_numeric	2000	1.000	2800	0.000	0.001

Table VII gives the verifiers' measured true- and false-accept rates (runs `de66e962`, `f7991b65`, `50ff0d27`, `fd07e0a8`, `dc7168b0`). Mathematics meets the $\varphi \leq 0.01$ target. The code φ is an upper bound inflated by behaviourally equivalent mutants.

The Profile C decision (MR-005) compared Qwen3.5-2B with Qwen3-1.7B as System 1 on 30 tasks, in alternating order and with the production prompt (Table VIII). Qwen3.5-2B was more accurate (+17 points, 95% CI [+7, +30], $n = 30$), but about 3.7 times slower per task under load. The adoption rule requires latency within 1.3 times, so Profile A stays the default. The accuracy gain is recorded for a faster host.

D. CPU inference performance (Phase 7)

Where the time goes. Every reply now records its stages:

- retrieval, the System 1 draft, the cheap samples, verification, System 2 deliberation and the judge;
- for each model stage, the gateway queue wait and llama-server's own prefill time, decode time and KV-cache reuse.

Over 514 replies with stage timings, the fast path spends essentially all of its time in the System 1 draft, and the deliberate path in System 2 deliberation (Table IX, Fig. 10); verification and gating are negligible. The prompt cache already serves most prompt tokens (Table X), so prefill is small. *Decode speed is the bottleneck*, and it falls several-fold when other workloads share the four cores: 2.6–4.3 tokens/s in service, against 22.9 in an idle llama-bench run.

Techniques and their measured effect. Each change was accepted only after an interleaved A/B on the reference machine (NFR-031):

- *Skipping cheap samples when an exact verifier decides* (Theorem 1 makes them irrelevant): median draft time fell from 13.4 s to 7.9 s.
- *Prompt-cache warm-up* at start-up, with llama-server's host-memory prompt cache: a cold first request had 9.8 s time to first token, a warm one about 1 s.
- *Verifier warm-up*: the first verification after start-up fell from 0.45 s to 3 ms.

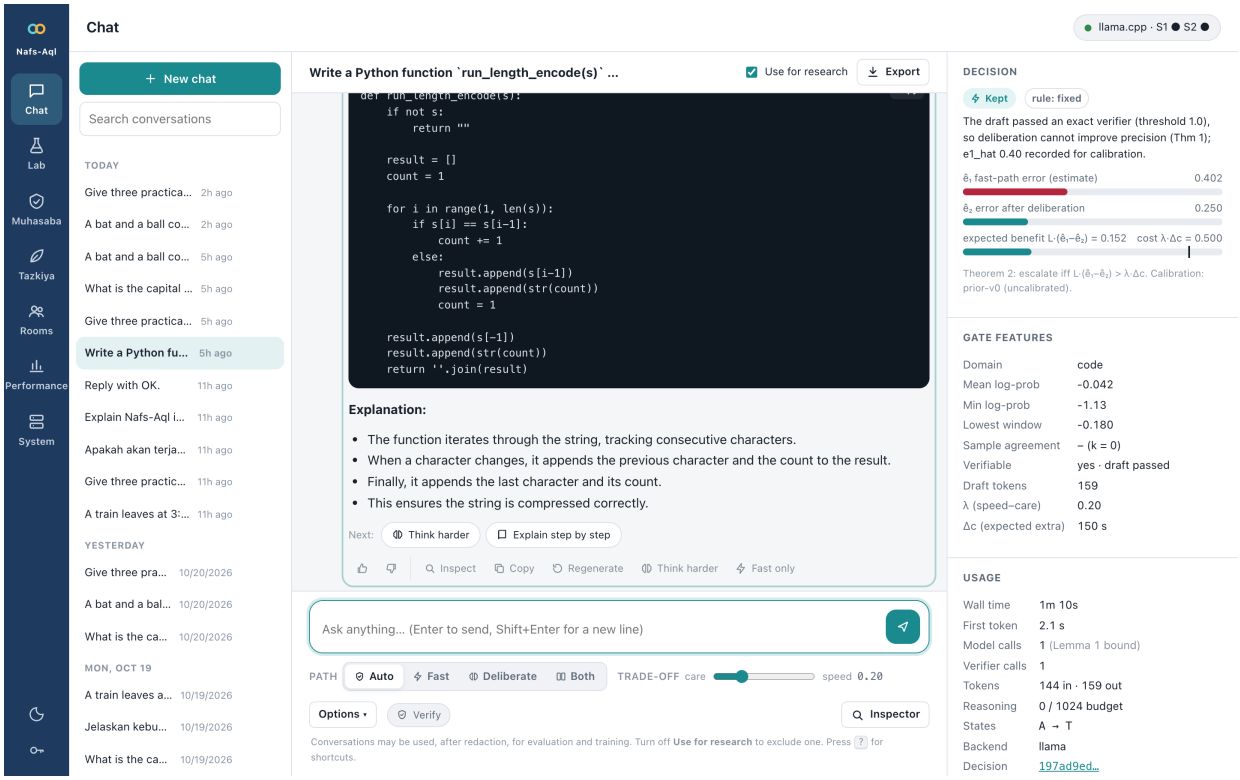


Fig. 8. Chat page (measured system). A code answer passed the exact verifier on the draft, so the gate kept it (Theorem 1); the inspector shows the decision, the gate features and the usage.

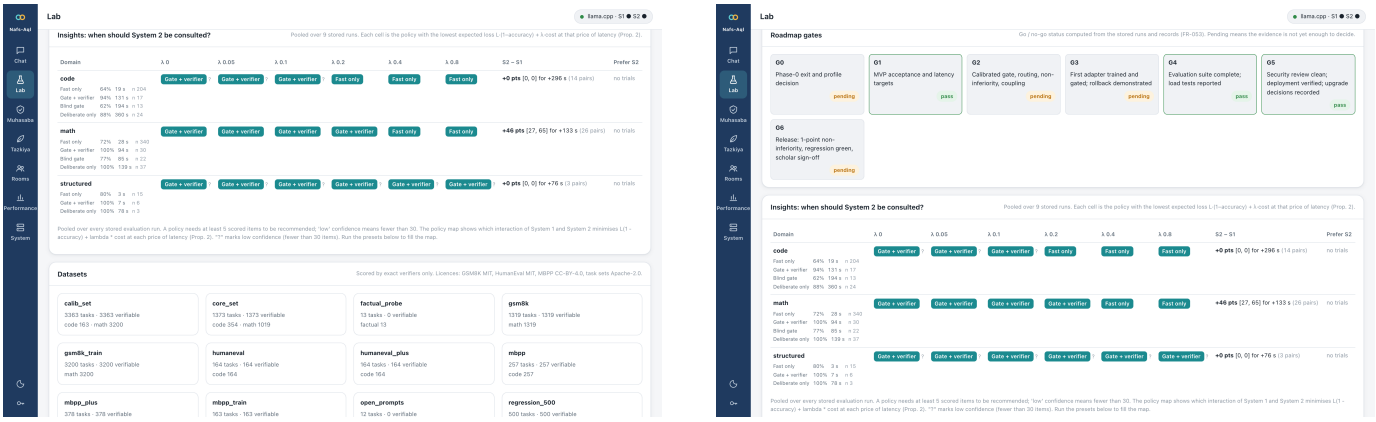


Fig. 9. Lab page: the pooled policy map, answering "when should System 2 be consulted?" per domain and price of latency (left); roadmap gates G0–G6 evaluated from stored evidence (right).

TABLE VIII
PROFILE C DECISION (SYSTEM 1 CANDIDATES, 30 TASKS)

System 1	All	Math	Code	JSON	Wall (s)	tok/s
Qwen3-1.7B (Profile A)	67%	56%	75%	83%	24.0	4.3
Qwen3.5-2B (Profile C)	83%	81%	75%	100%	88.9	1.9

TABLE IX
LATENCY BY STAGE (SHARE OF WALL TIME)

Path (replies)	Stage	Share	p50 (s)	p95 (s)
fast (475)	System 1 draft	98%	13.6	56.1
fast (475)	verify draft	2%	0.2	0.9
deliberate (39)	System 2 deliberation	95%	120.6	424.1
deliberate (39)	judge	3%	40.0	44.3
deliberate (39)	System 1 draft	2%	9.2	59.2
deliberate (39)	probe	1%	1.5	6.4

- *Early escalation* (FR-080): under the fixed rule, an open-ended draft whose running mean log-prob is already below the threshold minus a margin is stopped and handed to System 2, instead of being generated to the end, in the spirit of cost-aware cascades [74], [75]. In the agreement study, on 40 open-ended prompts, the configured rule ($N = 32$, margin 0.04) agreed with the full-draft decision on 100%

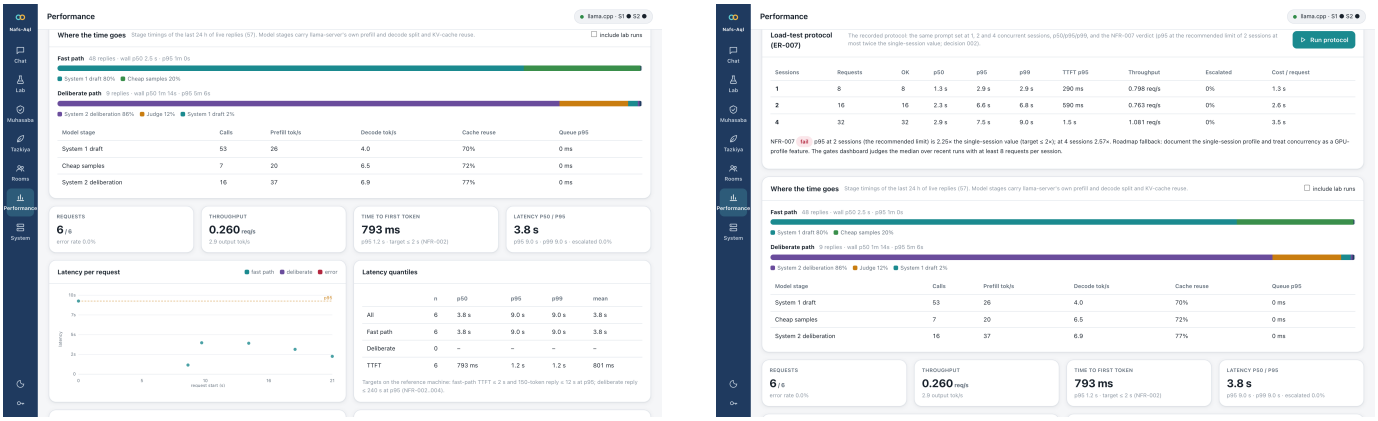


Fig. 10. Performance page: where the time goes, from stage timings, with the engine's prefill and decode speed and cache reuse (left); the load-test protocol at 1, 2 and 4 sessions with the NFR-007 verdict (right).

TABLE X
ENGINE SPEED PER MODEL STAGE (LLAMA-SERVER TIMINGS)

Model stage	Calls	Prefill tok/s	Decode tok/s	Cache reuse
System 1 draft	485	53	7.0	60%
cheap samples	32	13	4.2	80%
System 2 deliberation	45	16	3.4	95%

TABLE XII
LOAD-TEST PROTOCOL, FAST PATH (RUN `6c462074`)

Sessions	p50 (s)	p95 (s)	p99 (s)	req/s
1	3.8	7.2	7.2	0.25
2	2.7	4.9	7.2	0.68
4	3.6	9.0	12.0	0.76

TABLE XI
EARLY-ESCALATION AGREEMENT STUDY (OPEN-ENDED PROMPTS)

N	Margin	Agreement	Caught	False	Saved tok/esc.
32	0.00	80%	10/10	8	82
32	0.02	95%	5/10	2	92
32	0.04	100%	3/10	0	116
32	0.15	100%	0/10	0	0
48	0.00	90%	9/10	4	77
48	0.02	98%	4/10	1	117
48	0.04	100%	2/10	0	154
48	0.15	100%	0/10	0	0

of prompts, caught 3 of 10 escalations early and saved 116 draft tokens per early escalation (Table XI).

- *Sequential, uncertainty-gated samples*: the two agreement samples had run concurrently with the draft, and three sequences on four cores tripled the factual fast path (12.6 s median, 1.4 s to first token). They are now drawn after the draft and only when its mean log-prob lies within 0.10 of the threshold; the path is 4.3 s and 0.14 s, the same as drawing no samples.
- *Four System 1 slots*: four sessions no longer queue behind three; batched decode makes the fourth slot cheap.
- *No gain*, so not adopted:
  - speculative decoding [84] with a 0.6B draft model (6.5 against 7.3 tokens/s), because the draft model competes for the same cores and memory bandwidth;
  - n-gram lookup;
  - flash attention and a q8_0 KV cache: a sequential run suggested a 30% gain, which an interleaved repeat showed to be an ordering effect of the shared load.

Concurrency. The load-test protocol (latest run `6c462074`, Table XII) gave a 4-session p95 of 1.25 times the single-session value. The gate judges NFR-007 on the median over recent eligible runs (median ratio 2.15 over 3 run(s) (1.25, 2.15, 2.48); fallback: document the single-session profile), because single runs on the shared host ranged from 1.25 to 2.48.

Adaptive reasoning exit (FR-082). The deliberate path is almost entirely System 2 reasoning, so the second Phase 7 change shortens the reasoning itself, in the spirit of the 2026 early-exit methods (DTSR, DEER, Dynasor-CoT) but with the design's own stopping signal. System 2 reasons in chunks on llama-server's raw continuation endpoint, which keeps the prefix cached. At a reflection signal ("Wait", "Alternatively", ...), with a minimum gap of 64 tokens and at the latest every 256 tokens, the reasoning is closed provisionally and the model is asked for its answer in a few tokens. On a verifiable task the exact verifier judges that probe and, if it passes, the reasoning stops; on factual tasks two consecutive agreeing probes stop it. The final answer is then generated after the reasoning exactly as on the standard path, so the exit changes only how much reasoning is generated, and the exit reason is recorded with the reply. A probe costs 0.6–0.8 s. In the paired Lab comparison (run `e7607cbb`), with the exit: accuracy 100%, mean wall 123 s; without: 100%, 220 s ($n = 11$ GSM8K items; paired Δ accuracy +0 points, 95% CI [+0, +0]).

EAGLE-3 speculative heads [87] for both models (Angel-Slim, Apache-2.0) were converted and measured in interleaved A/B runs (Table XIII). On a GPU these heads give 1.7× because verifying a draft is nearly free there. On four cores the verification pass costs real compute, and at the measured acceptance of about half the drafted tokens neither system gained: System 1 1.04× greedy and 0.95× at the production temperature, System 2 0.99× on reasoning prompts; eight draft

TABLE XIII
EAGLE-3 SPECULATIVE DECODING, INTERLEAVED A/B ON THE REFERENCE CPU

Model	Variant	Sampling	Decode tok/s	Speed-up	Drafted tokens accepted
System 1	plain	greedy	17.3	1.00	–
System 1	plain	production	18.0	1.00	–
System 1	eagle3_n4	greedy	18.0	1.04	52%
System 1	eagle3_n4	production	17.2	0.95	49%
System 1	eagle3_n8	greedy	16.0	0.93	46%
System 1	eagle3_n8	production	15.4	0.86	42%
System 2	plain	greedy	8.3	1.00	–
System 2	plain	production	7.6	1.00	–
System 2	eagle3_n4	greedy	8.2	0.99	46%
System 2	eagle3_n4	production	7.5	0.99	46%
System 2	eagle3_n8	greedy	7.3	0.87	37%
System 2	eagle3_n8	production	6.9	0.91	37%

TABLE XIV
WEIGHT FORMATS, QWEN3-1.7B, LLAMA-BENCH PP128/TG64

Weights (Qwen3-1.7B)	Prefill tok/s	Decode tok/s
Qwen3-1.7B-Q4_K_M	81	15.3
Qwen3-1.7B-Q4_0 (own)	44	12.2
Qwen3-1.7B-IQ4_NL (own)	70	16.0
Qwen3-1.7B-Q4_K_M (own)	72	12.8

tokens were slower than no speculation on both. The heads stay available as an opt-in (`EAGLE=1`) for a host with more cores. Weight formats with AVX2 repack kernels (Q4_0, IQ4_NL) remain to be measured the same way.

Weight formats. Q4_0 and IQ4_NL use llama.cpp's AVX2 repack kernels, which read the weights in an interleaved layout; Table XIV compares them with the production Q4_K_M on the same model (interleaved `llama-bench` runs, 4 threads, host load recorded in the log).

A smaller judge? The rubric judge is the largest fixed cost on open questions once the adaptive exit is in place (26–40 s per answer on System 2). Scoring with System 1 instead would be about three times faster, so it was measured with the rates harness on the same labelled items: System 2 judge: $r = 1.00$, $\varphi = 0.045$ (upper 0.23); System 1 judge: $r = 0.78$, $\varphi = 0.27$ (upper 0.50); 23 correct and 22 seeded wrong answers. A judge that accepts a quarter of the seeded wrong answers is not a soft verifier in the sense of Theorem 1, so the judge stays on System 2; the backend remains a configuration option for hosts with a larger small model.

vLLM backend. The second serving profile, vLLM's CPU backend, now runs in Docker from the team's `carstenz/vllm-smoke` image (vLLM 0.30, x86 CPU build) with Qwen3-1.7B serving both roles from the host's Hugging Face weights, and the gateway switches between the two backend sets at run time. Two facts shaped the recipe. First, the Docker VM exposes AVX2 only, where oneDNN has no BF16 kernels: in BF16 the engine loaded but never returned a token, so the model runs in FP32 (6.8 GB of weights). Second, the VM's memory cap matters: at 6 GiB only 0.18 GiB remained for the KV cache and at 10 GiB with a 2 048-token context the engine was OOM-killed during its profiling pass, so the context, the prefill batch and the KV cache are kept small. The gateway-side check, which drives every request path the orchestrator depends on against the backend, reports no result yet: on the day of the work the host had under 1 GB free and the Docker VM did not start at the 7.5 GiB the stand-in needs. On four shared cores the three-vCPU VM is a correctness and portability profile, not a speed contest with llama-server.

E. Operations and the gates

Monitoring and safety:

- **Drift and alerts** (Fig. 11): a two-sided CUSUM per domain on calibration error and fast-path accuracy, with a shuffled negative control. Five alert rules run over live traffic only; evaluation and load-test replies are tagged as lab traffic.
- **Security:** 26/26 security tests pass; 0 open high-severity findings. The suite found one high-severity issue, now fixed: sandboxed code could signal other processes.
- **Deployment checks:** 11 passed, 0 failed, 1 skipped (tool or host not available).

Table XV lists the roadmap gates as evaluated from stored evidence. The gates that remain pending need release-scale runs (thousands of paired items), GPU training, or the scholar review of the analogy text.

X. PHASE 8: QUANTUM CONSCIOUSNESS, NEUROSCIENCE AND SEQUENTIAL EVIDENCE

Phase 8 (20 October 2026) asked whether quantum theories of consciousness and the neuroscience of decision making offer improvements to the inference speed or the reasoning accuracy of Nafs-Aql. Each candidate idea had to pass four tests before any code was written: it must be established rather than hypothesised; a classical, CPU-only pipeline must inherit something from it beyond its classical description; it must yield a result with an engineering consequence in the sense of Section III; and the consequence must be measurable on the reference machine with the exact verifiers. The full review, with sources, is `docs/research/quantum_neuro_review.md`; this section records the results.

A. Quantum consciousness: a negative result

Orchestrated objective reduction [135] places consciousness in quantum computations within microtubules. Tegmark's estimate of the decoherence times of the relevant superpositions is 10^{-20}–10^{-13} s against dynamical timescales of 10^{-3}–10^{-1} s [136]; the rebuttal of [137] lengthens the former by several orders of magnitude without reaching the latter. Recent measurements of collective ultraviolet emission from tryptophan networks in microtubules [138] show a modest quantum-yield enhancement and do not demonstrate computation. Two results close the question for Nafs-Aql.

Proposition 11 (Decoherence separation)**.** [literature] *With $\tau_{\mathrm{dec}} \leq 10^{-4}$ s on the most favourable published estimate and $\tau_{\mathrm{dyn}} \geq 10^{-3}$ s, any superposition of neural states has decohered into a mixture over the interval of a decision. The description of cognition at that timescale is a classical stochastic process.*

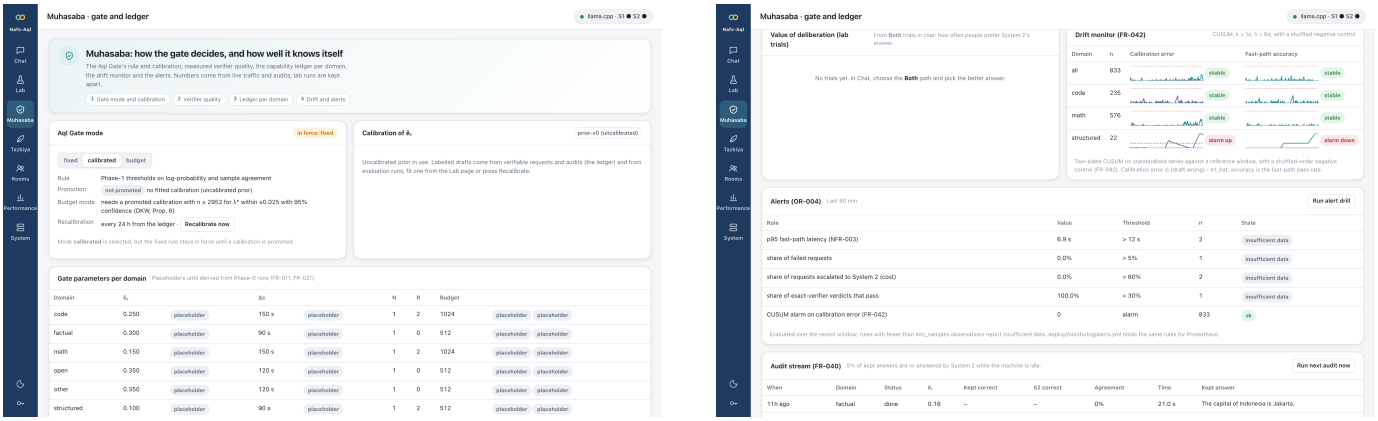


Fig. 11. Muhasaba page: gate mode, promotion status and calibration (left); drift monitor and alert rules over live traffic (right).

TABLE XV
ROADMAP GATES EVALUATED FROM STORED EVIDENCE (18 OCTOBER 2026)

Gate	Status	Evidence
G0	pending	H7: pass; P0-T04: pending
G1	fail	MVP: fail; NFR-002/004: fail
G2	pending	ECE: pending; H1: pending; ER-002: pending; ER-004: pending
G3	pending	TR-007: pending; OR-005: pending
G4	fail	P4: pass; ER-007: pass; NFR-007: fail
G5	pass	SR-006/SR-002: pass; OR-003: pass; MR-005: pass
G6	pending	ER-002: pending; ER-008: pass; SR-005: pending

Consequence. The object is the one Section III already models: stochastic policies π_1, π_2 and a finite automaton over trace states.

Proposition 12 (No transferable algorithm)**.** [exact] *A classical machine that follows the state of n two-level systems stores 2^n complex amplitudes; in double precision the reference machine's 32 GB hold $n \leq 30$. No theorem separates BQP from BPP, so no speed-up is inherited by a classical algorithm from the quantum character of a process.*

Proof. $\dim \mathbb{C}^{2^n} = 2^n$ and $2^{30} \times 16$ bytes $= 17$ GB; the second claim is the open status of the separation. □ □

Consequence. Even an established Orch OR would contribute nothing computable to a CPU inference loop (A-055). Quantum *cognition*, the use of Hilbert-space probability as a descriptive model of judgement [139], [140], has one structural lesson: non-commuting measurements produce order effects, confirmed by the parameter-free QQ equality on about seventy surveys [141], [142] and reported in language models [143]. Read for Nafs-Aql, an order effect is anchoring of the deliberate answer on the draft, which the draft-blind slow path (FR-023) removes and the coupling test (ER-004) measures (Proposition 4). The design needed no change.

B. Neuroscience: the sequential form of the gate

Three results of decision neuroscience have exact mathematical forms: evidence is accumulated to a bound [144], whose optimal form is Wald's sequential probability ratio test [145], [146]; under a cost of time or a deadline the optimal bound collapses [147], [148]; and the quality of a confidence signal is its accuracy–confidence association measured by ROC statistics [149], with the all-or-none ignition of a global workspace as the same threshold picture at system level [150]. Theorem 2 is the one-step instance; three mechanisms of Nafs-Aql read a *stream* of evidence and stop on a bound, and each had a hand-set rule where a theorem exists. The language-model literature reached the same rules independently [151]–[153].

Proposition 13 (ROC dominance)**.** [exact] *If the ROC curve of a confidence signal s' for predicting a wrong draft dominates that of s at every escalation rate, the best threshold gate on s' has an objective* (1) *no larger than the best threshold gate on s, for every L, λ and Δc.*

Proof. At a fixed escalation rate the compute term is equal and the error term is L times the wrong drafts not escalated, which dominance makes no larger; minimising over the rate preserves the inequality. □ □

Consequence. The gate's signal is chosen by a stored ROC study against exact labels (ER-021), not by convention.

Proposition 14 (Exact early escalation on a running minimum)**.** [exact] *Let $w(n)$ be the lowest mean log-probability over complete windows of W consecutive tokens among the first n tokens of a draft. Then w is non-increasing in n, so for the rule "escalate iff $w(N) < \theta$" the stopping time $T = \min\{n : w(n) < \theta\}$ gives exactly the full draft's decision, and no rule on the same stream escalates such a draft earlier without a non-zero disagreement rate.*

Proof. A minimum over a growing set; $w(N) \le w(T) < \theta$; before T every window is $\ge \theta$. □ □

Consequence. Early escalation on the lowest-window signal needs no margin, minimum token count or agreement study (FR-083). It is a decision, not a prediction.

Proposition 15 (Wald's bound on the mean rule)**.** [exact under i.i.d. Gaussian tokens] *Any sequential test of $\mu = \tau$ against $\mu = \tau - \delta$ on tokens with variance σ^2 and errors (α, β) has* $\mathbb{E}_1[N] \ge \big[(1-\beta)\ln\frac{1-\beta}{\alpha} + \beta\ln\frac{\beta}{1-\alpha}\big] \big/ \frac{\delta^2}{2\sigma^2}$.

Measured. On the 40 open-ended drafts of the early-escalation study the per-token spread is $\sigma \approx 0.28$ and the margin is $\delta = 0.04$: the bound is 260 tokens at $\alpha = \beta = 0.05$, against drafts of 47–291 tokens (median 133), and the first tokens of a draft are systematically more confident than the rest. *Consequence.* The mean-based early escalation of P7-T03 is an empirical heuristic that cannot be made more aggressive by a sequential test; it is kept at its measured setting and labelled as such (A-056).

Proposition 16 (Posterior stopping with a collapsing bound)**.** [exact for the rule; strong for optimality] *With k probed answers clustered into categories, counts v and a uniform Dirichlet prior, the posterior probability that the plurality answer is the majority answer is* $1 - I_{1/2}(v_{\text{top}}+1,\, k - v_{\text{top}} + 1) = 1 - 2^{-k-1}\sum_{j=v_{\text{top}}+1}^{k+1}\binom{k+1}{j}$. *Under a per-probe cost and a hard budget the Bayes-optimal rule stops at the first k with this probability at or above a bound $\theta(t_k)$ that is non-increasing in the budget used [147].*

Values. One probe 0.75; two agreeing 0.875; $[2,1]$ 0.6875; $[3,1]$ 0.8125. With θ falling linearly from 0.85 to 0.70 the rule reproduces the "two consecutive agreeing probes" rule early in the budget, never stops on one probe, votes over all probes (Theorem 6) rather than the last two, and lets a $[3,1]$ split stop once about 45 % of the budget is used (FR-084). Verifiable tasks are untouched: the exact verifier is the stopping signal (Theorem 1).

Proposition 17 (Optimal probe interval)**.** [exact under an exponential model] *If the reasoning length at which the answer becomes available is exponential with mean m tokens, a probe costs c_p seconds and decoding runs at v tokens per second, the periodic interval minimising probe cost plus overshoot is $\Delta^\star = \sqrt{2c_p v m}$, with* $\text{cost}(\Delta)/\text{cost}(\Delta^\star) = \frac{1}{2}(\Delta/\Delta^\star + \Delta^\star/\Delta)$.

Measured. Idle: $c_p \approx 0.7$ s, $v \approx 8$, $m \approx 300$ give $\Delta^\star \approx 58$ tokens; the configured gap of 64 costs 0.5 % more than the optimum (1 % under the shared load). The schedule is validated and unchanged (A-059). This is the quantitative content of the "quantum Zeno" intuition that measuring often is costly; here a probe has no back-action, only a price.

C. Measurements

All measurements were taken on the shared reference machine (load average 5–11, other sessions compiling); accuracy and log-probabilities do not depend on load, wall-clock figures do.

TABLE XVI
WHICH CONFIDENCE SIGNAL OF A SYSTEM 1 DRAFT PREDICTS A WRONG DRAFT (150 GSM8K DRAFTS, 31 WRONG)

Signal	AUROC [95 % CI]	recall / precisio
lowest token log-prob	**0.646** [0.539, 0.746]	0.23 / 0
mean top-1/top-2 margin	0.614 [0.493, 0.729]	0.23 / 0
lowest 16-token window of the margin	0.611 [0.510, 0.715]	0.06 / 0
lowest 16-token window log-prob [153]	0.581 [0.488, 0.698]	0.13 / 0
negative mean top-5 entropy	0.573 [0.453, 0.679]	0.16 / 0
mean log-prob (the fixed rule)	0.564 [0.449, 0.672]	0.13 / 0
bottom-10 % tokens	0.555 [0.459, 0.675]	0.10 / 0
tail (last 32 tokens)	0.492 [0.366, 0.631]	0.19 / 0

Confidence-signal study (ER-021). 150 GSM8K test items were drafted by System 1 under the production prompt and sampling (temperature 0.2, top-p 0.8, no thinking) and scored by the exact numeric verifier: 31 drafts are wrong (accuracy 0.793). Table XVI gives, for each candidate signal, the AUROC for predicting a wrong draft (lower signal flags the draft; 400-resample bootstrap interval) and the recall and precision when the least confident 10 % and 20 % of drafts are escalated.

Three facts follow. First, *the fixed rule is blind on mathematics*: at the production threshold (-0.175 at the default λ) the mean-log-prob rule escalates 0 of 150 drafts and catches 0 of 31 wrong ones, because mathematics drafts have a mean log-prob of about -0.03 whether right or wrong; on the unverified mathematics path every wrong draft was kept, and because such drafts never enter the uncertainty band the agreement samples were never drawn either. Second, *token confidence is a weak metacognitive signal* for this model on this task: the best signal is the single lowest token, and the window signal's interval includes chance, so no ROC dominates (Proposition 13) and the signal that Proposition 14 makes exact to act on is not the one worth acting on. Third, *Theorem 2 forbids escalating on any of them at the default price*: escalation pays only if $\mathbb{P}[\text{wrong} \mid \text{escalated}] > \hat{e}_2 + \lambda\Delta c/(L \cdot \text{cost unit}) = 0.15 + 0.2 \cdot 150/60 = 0.65$, and the highest precision reached at an escalation rate of at least 10 % is 0.47. By Proposition 10 the expected accuracy of a gate escalating the 20 % least confident drafts by the lowest token is 0.847–0.867 (System 2 accuracy 0.9–1.0) against 0.793, for about 30 s of mean latency per question, which the default price rates as a loss. *Decision:* the window signal is recorded on every decision and read by the calibrated gate, but its fixed-rule threshold stays off (A-057): the mechanism is exact, the measured signal is not strong enough to use it. The repository keeps this negative result next to the positive ones.

Agreement with cheap samples. The gate's second signal, agreement between the draft and $k = 2$ cheap System 1 samples, was never computed on mathematics because the samples are drawn only inside the uncertainty band of the mean log-prob. For the first 60 drafts the samples were drawn exactly as production draws them and compared with the draft's final number (12 of 60 drafts wrong; 54 s per item on the loaded machine). Agreement has AUROC 0.759 [0.618, 0.901] and, with ties broken by the lowest token, 0.823 [0.690, 0.940], against 0.720 for the lowest token and 0.611

for the window on the same drafts. The rule "any disagreement" (the configured threshold at the default care) escalates 25 % of drafts, catches 8 of the 12 wrong ones (precision 0.53) and raises the expected cascade accuracy from 0.800 to 0.908–0.933. This is what Theorem 6 predicts: a wrong low-temperature draft is usually one whose answer holds no majority at temperature 0.8. *Decision (implemented, FR-085):* unverified mathematics drafts always get their two samples (`gate.samples_on_unverified_domains`), whatever their token confidence says (A-060). Its precision is still below the 0.65 bar of Theorem 2 at the default price, so the calibrated gate, once promoted, will price it rather than apply it unconditionally.

Exit rules on factual prompts (FR-084). A paired Lab run on six factual prompts (run `73cd5bba`, 512-token budget, shared machine) compared the consistency rule with the posterior rule. Both gave the same factual content on every prompt; reasoning tokens were 357 against 382 on average (wall 93 s against 101 s), two prompts exhausting the budget under both rules. Where they diverged most, the probes disagreed first and then agreed: the consistency rule stopped at the two consecutive agreeing probes (183 tokens) while the posterior rule saw a $[2, 1]$ split below its bound and continued to $[3, 1]$ (318 tokens), as Proposition 16 prescribes. That caution costs tokens on this set, so by the criterion fixed in advance (A-058) the posterior rule stays an option and the consistency rule remains the default; the comparison is indicative ($n = 6$, no exact labels).

D. What changed

The crate `nafs-core` gains a `sequential` module (the window signal and its streaming tracker, the probe posterior with the exact beta tail, the collapsing bound, Wald's bound and the probe interval, each with tests of the property its proposition states). Every gate decision now records `low_window_logprob`; the calibrated gate reads it as an eighth feature; the fixed rule applies it for the measured domains and stops the draft at the crossing; the adaptive exit offers the posterior rule per request and in the Lab (`slow_exit_posterior` against `slow_exit_consistency`). Nothing was added for the quantum-consciousness ideas; Propositions 11–12 are the record of why.

XI. Phase 10: A Math and Code Harness from Open Math Harness and Codex

Two open agent harnesses were assessed for what Nafs-Aql could take from them (24 October 2026): Open Math Harness [154], the runtime of Open Math OS for mathematics research, and the Rust core of OpenAI's Codex CLI [155], a coding agent. Both are Apache-2.0 and both are large (about 1.1×10^5 and, for Codex's core crate alone, 2.6×10^5 lines of Rust), so the question was not whether to embed them but which mechanisms carry over to a single-process gateway that serves a 1.7B and a 4B model on four CPU cores. The assessment, the integration plan and the research behind it are in `docs/research/harness_integration.md`; this section records the design and its measurements.

A. What the two harnesses are

Open Math Harness is a capability microkernel (an in-memory kernel with capabilities, IPC and lifetimes) under which every service and agent is a confined Linux process. Its mathematics lives in three services. The *Lean service* decides proofs with Lean 4 [156] under a protocol in which the statement is frozen on the service side, the candidate is refused on text alone for forbidden tokens (`sorry`, `axiom`, `native_decide`, and every command that runs code during elaboration) or for not reproducing the statement byte for byte, and Lean then checks a file assembled from the *frozen* preamble, the candidate's body, an unguessable marker, `#print axioms` and the fully elaborated (`pp.all`) type, which must equal the pristine statement's character for character. The *discovery* service keeps candidate proofs in a tree, scores each node from Lean's verdict alone (1 solved, 0.9 complete but rejected, $0.5{+}0.4\,C/(C{+}L)$ for a partial proof with C credited and L open steps, 0.05 for errors, 0 for invalid), and continues the best nodes with a reserved slot for the best repair prospect. *Ultramath* runs eight members at maximum reasoning effort on distinct registered directions with a shared board.

Codex is a submission/event-queue agent whose turn loop samples the model, executes the tool calls of the stream, records their outputs and samples again until a reply carries no call. The parts that travel are its leaves: the `apply_patch` language with a lenient parser and a four-pass fuzzy matcher (exact, trailing whitespace, surrounding whitespace, Unicode punctuation folded to ASCII); execution results reported as exit code, wall time and *middle-truncated* output; and a macOS Seatbelt profile whose paths are passed as `-D` parameters instead of being written into the policy text. Codex speaks only the Responses API and has no fallback for models without native tool calls.

B. What was integrated

Table XVII maps each gap in Nafs-Aql to its source. The new crate `nafs-harness` holds the derived code as pure logic: it starts no process and touches no file; Lean, Python and the models are reached through closures and a trait that the gateway supplies, so every rule is unit-tested without either. Derived files carry their upstream path and revision and say how they were modified; the crate's `NOTICE` carries both projects' notices.

Three things were deliberately left out. The kernel, platform and launcher of Open Math Harness need Linux and bubblewrap, and Nafs-Aql is one process with an OS sandbox per untrusted run. Ultramath's eight members at maximum effort would take tens of minutes per question at 6–8 tokens/s; its cheapest idea, telling a repair what was already tried, is kept. Codex's Responses-API client, tool router, approvals and compaction assume a frontier model with native tool calls and a context window as the binding limit; here the model is small and the binding limit is the wall-clock cap, so the code agent speaks fenced text and resets its context every step.

TABLE XVII
INTEGRATED MECHANISMS (FR-090 TO FR-093).

Gap in Nafs-Aql	Source	What was taken
Maths checkable only against a reference answer	OMH Lean service	Lean verifier (FR-090): textual rules, frozen preamble, marker-fenced axioms and `pp.all` type, run in the OS sandbox
A retry always repairs the *last* candidate	OMH discovery	Candidate tree, scoring protocol, best-first repair with "already tried" siblings (FR-091)
All-or-nothing code verdicts	OMH partial-proof credit	Per-assert credit from the sandbox runner, protected by the run's nonce
Full-budget retries regenerate whole programs (A-067)	Codex `apply_patch`	Patch repair: a retry may answer with a patch to `solution.py` (FR-092)
Evidence keeps only the tail of the output	Codex truncation	Head and tail with an explicit marker
No agent that runs its own code	Codex turn loop, OMH prover agent	Code agent, `POST /v1/harness/code` (FR-093)
Path written into the sandbox policy	Codex Seatbelt	`-D` policy parameters

C. Lean as an exact verifier

A request may carry `verification.lean`: a frozen statement whose proof is `sorry`. The verifier takes the answer's last `lean` block, applies Open Math Harness's rules, and runs Lean inside the same sandbox as model-written Python (no network, no fork, no writes outside the scratch directory, no reads of user data except the toolchain). A partial proof that elaborates is graded *sorries* (score 0.5) and a proof is *verified* only when the strict protocol accepts it. The pristine type is computed once per statement and cached. By Theorem 1 the precision of an accepted proof is bounded by the verifier's false-accept rate; for Lean that rate is the kernel's soundness plus the textual rules, and the protocol's marker makes the candidate's own printed output unreadable as a report. Requests are refused (400) when no toolchain is configured, so a Lean task can never be labelled *verified* by a fallback.

D. Best-first repair

Every verified attempt becomes a node; the next attempt repairs the node a policy chooses and sees that node's own evidence and, as data, what earlier repairs of it did. Nafs-Aql explores one candidate at a time, so Open Math Harness's policy needs two changes: with a single slot its reserved repair slot would always beat the score ordering, so the repair prospect (fewest failures) breaks ties instead; and equal scores go to the latest node (the prover's own `pick_best` rule) rather than the shallowest. A node stays continuable until it has F children (default 2), which lets the search return to a parent when a repair made things worse.

Proposition 18 (Conservative extension)**.** [exact] *If every verdict is all-or-nothing (every failed candidate scores 0.05 and has one failure), the best-first policy repairs the latest candidate at every round, exactly as the previous loop did, for any fanout $F \geq 1$ along a chain of repairs.*

Proof. Along a chain each new node is a child of the previous one and has no children of its own, so it is eligible. All failed nodes tie on score and failures; the tie goes to the largest id, which is the latest node. The root scores 0 and is chosen only when no eligible node scores more, which a valid failed candidate (0.05) prevents. □ □

Consequence. The policy is on by default (`repair_policy = "best"`): it changes behaviour only when per-assert credit, Lean's graded verdicts or an invalid patch distinguish candidates, and the existing orchestrator tests pass unchanged.

E. Patch repair and when it pays

Let a whole program cost W answer tokens, a patch P, and let a patch fail to apply with probability f, after which the next round regenerates (cost W). To first order the expected answer tokens of a repair are $P + fW$, so a patch pays when

$$f < 1 - P/W. \tag{5}$$

For a one-line fix in a 15-line function $P/W \approx 0.3$, so patches help as long as fewer than about 70 % fail to apply. The literature does not settle f for 4B models: diffs save tokens, but weaker models fail diff formats more often than whole files [157]. Patch repair therefore accepts whole code as well, applies patches with Codex's fuzzy matcher, never runs a patch that does not apply (its error is the evidence of an *invalid* node, so the tree returns to the candidate it tried to repair), and is *off* by default until an interleaved A/B on the reference machine measures f (rule 3).

F. The code agent

`POST /v1/harness/code` runs a bounded loop on a workspace of flat files. Each reply holds one action: a Codex patch, a whole `solution.py`, a `run` block naming a Python file of the workspace (executed in the sandbox, reported in Codex's format), or `SUBMIT`. The exact verifier runs after every change and a passing workspace ends the run at once; `SUBMIT` alone never does (Open Math Harness's prover rule). Each step's prompt is the task, the current files, the last verdict and the last three exchanges, so the context is bounded by construction; the best-scoring workspace is returned when nothing passes. Model calls are bounded by `max_steps` (default 6, at most 12) and the run by a wall-clock cap, as Lemma 1 requires of every loop in Nafs-Aql. This follows the self-debugging and agent-interface results that concise, bounded execution feedback is what lets models repair code [158], [159], and the verifier-guided self-correction of formal provers [160]; best-first search over verified candidates is competitive with heavier tree search when the verifier is exact [161].

G. Verification

Real Lean. Lean 4.35.0 (core, no Mathlib) was installed with `elan` on the reference machine and the protocol was run against the real binary inside the macOS sandbox (Table XVIII). Lean resolves its own installation path, which

TABLE XVIII
THE LEAN PROTOCOL AGAINST REAL LEAN 4.35.0
(REAL_LEAN_ACCEPTS_PROOFS_AND_REJECTS_THE_KNOWN_ATTACKS).

Candidate for `a + b = b + a`	Verdict	Decided by
`by omega`	verified	Lean
`Nat.add_comm a b`	verified	Lean
helper lemma, then the theorem	verified	Lean
`by sorry`	partial (0.5)	Lean elaborates; `sorry`
`by rfl`	errors (0.05)	Lean error
`sorryAx _ false`	rejected	axioms report
`axiom cheat : False`	rejected	textual rule
shadowed `HAdd` instance, `rfl`	rejected	`pp.all` type

needs `stat` on the directories above the toolchain; the profile now allows metadata reads of exactly those directories, passed as `-D` parameters, and their contents stay unreadable. All eight cases behave as the protocol intends. The decisive one is the shadowed instance: a candidate that declares a high-priority `HAdd Nat Nat Nat` returning 0 keeps the statement byte for byte and closes it with `rfl`; only the comparison of the fully elaborated types rejects it. A hidden `sorryAx` passes the textual rules and is rejected on Lean's own axioms report. A check takes about 0.6 s once the statement's pristine type is cached (1.4 s for the first).

Tests. `nafs-harness` has 57 unit tests, including the ported upstream suites (Codex's streaming patch parser and fuzzy matcher, Open Math Harness's Lean textual rules, verify protocol and scoring). `nafs-verify` adds per-assert credit and two Lean tests that run the whole protocol through the real macOS sandbox with a stand-in `lean` executable (a pass, a partial proof graded 0.5, an elaboration error, a user axiom, `sorryAx` in the report, and fail-closed without a toolchain); the existing isolation tests (network, home reads, fork, escape attempts) pass with the parameterised Seatbelt profile. Four new orchestrator tests on the mock backends show the tree returning to the candidate with more credit, `repair_policy = "last"` reproducing the old loop, patch repair surviving a patch that does not apply, and the code agent running, patching and stopping on the verifier. All 22 earlier orchestrator tests pass unchanged, as Proposition 18 predicts; the workspace has 165 tests, clean under `clippy -D warnings` on macOS and, cross-checked, on the Linux target.

Live run (partial; shared machine). `scripts/harness_live_check.py` drives the running gateway with Qwen3-4B-Thinking as System 2 while the owner's other llama-server used about 270 % CPU (load average 10–30), so times are lower bounds of nothing and are not speed claims. On the first agent task (make a palindrome test ignore case and punctuation) the model's first patch used a unified-diff hunk header (`@@ -1,4 +1,12 @@`) that Codex's grammar reads as a context line, so it did not apply and the model was told why; its second patch applied and the hidden tests passed: *verified* after two model calls (about 500 s each). On the second task a single model call took 1,041 s, longer than the 900 s run cap, and ended inside the reasoning budget with no action, so the run stopped at the deadline with the original file returned *unverified*. Two changes follow. The patch parser now drops unified-diff line ranges from a hunk header (a leniency in the spirit of Codex's own heredoc stripping), with a test built from the live reply. And the run cap is checked between steps, so one step can overrun it by its own length; on this host a step budget of 512 reasoning and 768 answer tokens is too large for a 900 s cap. The third agent task (write run-length encoding from an empty workspace) failed the same way: its first patch updated a file that did not exist yet with lines lacking the + prefix, which the parser refused with the line number, and its second step ran out of budget inside the reasoning. The first patch-repair chat reached the gateway's 900 s request timeout, and the run was then stopped by hand (4 of 6 items; `benchmarks/harness_live_check.json`). On this host, under this load, System 2 agent steps are too slow for the default caps; the measurement of patch repair (Eq. 5) needs a quiet host.

XII. PHASE 11: A CODEX-STYLE AGENT HARNESS FOR SMALL MODELS ON A CPU

Phase 10 took leaves of Codex (the patch format, output truncation, the sandbox parameters) and kept a simple agent loop. Its live run (Section XI) showed where that loop loses time and accuracy on this machine: every step re-read almost its whole prompt, three of five actions were malformed, and one model call overran the run's cap. Phase 11 (25–26 October 2026) ports the *harness* mechanisms of Codex's agent core [155] (its turn loop, history, compaction, tool grammar and budget handling), adapted to a chat-completions server, a 1.7B and a 4B model, and four CPU cores. The source survey is in `docs/research/harness_integration.md` (Section 5) and the decision in `docs/decisions/006-codex-harness.md`.

A. Where an agent step spends its time

A step's wall time is, to first order,

$$T_{\text{step}} = \frac{n_{\text{prompt}} - n_{\text{cached}}}{r_p} + \frac{n_{\text{dec}}}{r_d} + T_{\text{tools}}, \qquad (6)$$

where n_{cached} prompt tokens are served from the server's cache, r_p is the prefill rate and r_d the decode rate. On the reference machine System 2 prefills about $r_p = 31$ tokens/s and decodes $r_d \approx 6$–8 tokens/s (measured 25 October 2026): a cold 1,969-token prompt took 64 s to prefill, and the same prompt with one more exchange appended took 3 s, because only the 21 new tokens were computed. Prefill is therefore not negligible next to decoding, and whether the cache is hit decides it.

B. What Codex does, and what carries over

Codex speaks only the Responses API and assumes frontier models, so nothing is embedded; Table XIX lists the mechanisms and their adaptation.

C. An append-only transcript

Proposition 19 (Prefix reuse)**.** *Let a server reuse the longest common prefix between a request and the token sequence cached in its slot, and let the slot hold the previous request of the run. In the Phase 10 layout, the files are part of the first*

TABLE XIX
CODEX AGENT-HARNESS MECHANISMS (`codex-rs` AT `970111f6`) AND THEIR ADAPTATION IN NAFS-AQL (CRATE `nafs-harness`).

Mechanism	In Codex	In Nafs-Aql
Append-only history	`context_manager`: items are only appended; a change of state is a new item; requests are sent incrementally when the new input extends the previous one	`context.rs`: the transcript only grows; the task message is never rewritten, so llama-server's slot cache reuses every earlier exchange (Proposition 19)
Compaction	at 90 % of the window, a model-written handoff summary plus the user messages ($\leq$20k tokens) replace the history	at `compact_at_tokens`, a *harness-written* state replaces it: the task, the current files, the last verdict and the attempts so far; same prefix as the first message, so the system prompt and task stay cached
Freeform `apply_patch`	the patch is a freeform tool constrained by a Lark grammar	`grammar.rs`: the grammar in GBNF, rebuilt per step (only existing files can be updated, no unified-diff ranges, the step's allowed actions), with the reasoning span kept free
Dispatch at item done	a tool call runs when its output item is complete, while the response still streams	the stream ends when the action is complete (unconstrained requests); under the grammar the server itself stops there
Truncation on insertion	tool output capped (10k tokens) as it enters history	observations and evidence middle-truncated to 2 kB, files to 6 kB (a 10k-token output would take minutes to prefill here)
Budget reminders	`rollout_budget`, `token_budget`: remaining-budget notes appended to history	"Steps left: n" appended to each failing observation; the last step offers no `run`
Turn diff	`turn_diff_tracker`: the turn's net change as a unified diff	the run's net diff returned with the files
Cancellation	a turn aborts in flight	the run's wall-clock cap holds *inside* a model call (the stream is dropped, which stops the server)
Not taken	parallel tool calls under a read/write lock; websocket reuse; remote and async compaction; MCP; approvals	one action per reply (the verifier runs after every change, so a second action would duplicate it); the rest needs server state llama.cpp does not have

user message, so after a step that changed a file the reusable prefix ends at the first changed byte of the file view: each such step prefills at least the rest of its prompt, and a run of K steps prefills $\Theta(K \cdot L)$ tokens for prompts of length L. In the append-only layout the reusable prefix covers the whole previous request, so step k prefills only the previous reply and its observation, and a run prefills each transcript token once: $\Theta(L_K)$ for a final transcript of length L_K, plus one task message per compaction.

Sketch. In the append layout request k is request $k-1$ followed by the assistant message and the observation of step $k-1$. The cached sequence is request $k-1$ followed by the generated tokens; the chat template renders the earlier assistant turn without its reasoning, so the common prefix is exactly request $k-1$. In the reset layout the first divergence is inside the first user message whenever a file changed. Summing over steps gives the two totals. □

Codex keeps its history append-only for the same reason (its incremental requests fail over to full ones on any mismatch), and prefix-sharing caches such as SGLang's RadixAttention rest on the same invariant [162]. One consequence is that the model must learn the new state from the observation rather than from a rewritten task message: after a patch the observation carries the edited files, and after a failing verdict the verifier's evidence. When the estimated transcript exceeds `compact_at_tokens` (2,600 tokens by default, which leaves room for a reply in System 1's 4,096-token slots) it is compacted into the task message with an "already tried" list in place of Codex's model-written summary: at 6–8 tokens/s a summary would cost minutes, and the facts it needs are already structured.

D. Actions under a grammar

Codex constrains `apply_patch` with a grammar; llama-server accepts a GBNF grammar in the request. The port specialises the grammar to each step: the `Update` and `Delete` targets are the workspace's files, an `@@` header cannot start with a range, a `run` block can name only a Python file that exists, and the last step offers only edits. When the grammar is complete only end-of-generation can be sampled, so nothing after the action is decoded. `SUBMIT` is no longer offered: the verifier ends the run, so `SUBMIT` only spent a step.

A reasoning model needs one more rule. llama-server applies a request grammar from the first sampled token; with a plain grammar Qwen3-4B-Thinking was forced to emit the answer in place of its reasoning (measured on llama.cpp `797de63`). The grammar therefore opens with the reasoning span, any text up to `</think>`:

```
root  ::= think action
think ::= ( [^<] | "<" [^/] | "</" [^t] | "</t" [^h] | "</th
     " [^i]
          | "</thi" [^n] | "</thin" [^k] | "</think" [^>] )*
      "</think>" [ \n]*
action ::= patch | whole | run
whole ::= "```python\n" codeline* "```"
ctx   ::= "@@" ( " " [^-\n] [^\n]* )? "\n"
```

With this grammar and a 60-token reasoning budget the server closed the reasoning at the budget and sampled a well-formed answer. Reasoning stays unconstrained, which matters because format restrictions on the reasoning itself can cost accuracy [163]; grammar-constrained decoding is otherwise cheap on the server side [164], [165].

E. Which edits to offer

Codex edits only through patches. Small models are worse at diff formats than at whole-file rewrites [157], [166], and a patch saves answer tokens only when the file is long (Eq. 5). The live runs agree: in Phase 10 Qwen3-4B wrote a unified-diff header, an update of a file that did not exist, and an update hunk without line prefixes; in the first Phase 11 smoke run Qwen3-1.7B wrote well-formed patches under the grammar that appended a second copy of the file. `edit_format = "auto"` therefore offers the whole `solution.py` while it is missing or at most 80 lines long and a patch or the whole

file above that; `"any"` offers both at every step, as Phase 10 did. Patches remain the edit for long files, where Eq. 5 favours them.

F. Smaller pieces

A workspace the verifier already judged is not judged again (a model that restores an earlier file is told so). Tracebacks name files relative to the run (`File "runner.py"`), not the sandbox's random directory. Each step records the server's token accounting (prompt tokens computed and served from the cache, decode tokens, finish reason), and each run returns the totals and its net diff. The Phase 10 loop remains selectable (`context = "reset"`), and the request may override `context`, `constrain`, `stream_cut` and `edit_format`, which is how the A/B below was run.

G. Results

Design of the A/B. `scripts/harness_ab.py` runs ten small Python tasks (six fixes of a buggy `solution.py`, four written from an empty workspace; four or five hidden asserts each) through the running gateway, once per arm, with the same model and a budget of six steps; the order of the arms is rotated from task to task so that a drift in machine load does not favour one arm. The arms are the Phase 10 loop (context rebuilt every step, no grammar, no stream cut, patches always offered), Phase 11 with `edit_format = "auto"`, and, on System 1, Phase 11 with a patch offered at every step, which isolates the edit policy. The host was shared with the owner's other model servers (load average 5–9), so wall times are reported but are not speed claims; token counts and outcomes do not depend on load.

System 1. Table XX gives the totals. The Phase 11 arm verified 4 of 10 tasks in 41 model calls; the Phase 10 loop verified 2 in 50. The difference in verified tasks lies on two discordant tasks (Phase 11 verified the palindrome fix and `flatten`, where Phase 10 did not; no task went the other way), so with ten tasks it is indicative and not significant (exact McNemar $p = 0.5$). The mechanism behind it is not in doubt: 23 of the Phase 10 loop's 50 steps held no action it could parse (the 1.7B model answered in prose or with `SUBMIT`), against 1 of 41 under the grammar, a reply that ran out of tokens before its fence closed. Offering a patch at every step was the worst arm: 0 of 10 verified, 16 of 60 steps malformed, and the most tokens decoded (10,706). Most of its patches *applied* and still broke the file: the 1.7B model prefixed added lines with extra plus signs (`++ def rle(s):`), left a mangled `End Patch` line inside the file, or replaced the whole file with `python check.py`; the grammar fixes the syntax of a patch, not its sense. Prefill was not where System 1 lost time: llama-server's chunked cache reuse (`--cache-reuse 256`) and its host-memory prompt cache served 75 % of the Phase 10 loop's prompt tokens as well, against 78 % for the append-only transcript. Phase 11 decoded more (4,146 against 2,793 tokens) because its steps are whole files rather than unparseable prose, and spent 504 s against 356 s in all: 126 s per verified task against 178 s.

System 2. System 2 is the code agent's default model, and three tasks were run per arm (a System 2 step takes minutes on this host). The Phase 11 arm verified all 3 in 3 model calls, one per task, in 451 s in all; the Phase 10 loop verified 0 of 3 in 15 calls and 2,695 s. Two of its runs ended at the 900 s cap, which now holds inside a model call (both ended at 900.0 s; in Phase 10 one call had run to 1,041 s), and the third spent its six steps. 13 of its 15 steps were malformed: update hunks whose lines lacked the +/−/space prefix or the `@@` marker, and replies with no action at all, two of them cut off by the token limit. Under the grammar no step was malformed. Two of the three Phase 11 steps are worth a note: the files were short, so `"auto"` offered only a whole file, and the model wrote a Codex patch *inside* the ```` ```python ```` block, which the grammar allows (a code line may start with ⋆⋆⋆); the parser applies a patch envelope wherever it appears, both applied, and both runs verified. Every Phase 11 call spent its full reasoning budget (511 tokens): decoding took 418 s of the 451 s and prefill 24 s, which is why round 2 measures the budget. With three tasks this is a demonstration rather than an estimate, but the direction agrees with System 1 and the mechanism is visible in every step.

Round 2: breaking repetition, and the reasoning budget. Round 1 showed a failure the grammar cannot touch: a small model that rewrites the same failing file step after step, anchored on its own previous reply (System 1 reproduced an already-judged file in four consecutive steps of one run). Two changes went into the build for round 2. Failing asserts now carry the value they got (`assert f(x) == y -> AssertionError: got 5`), the concise execution feedback that self-debugging relies on [158]; and `unstick`: after a step that reproduced files the verifier had already judged, the transcript is compacted before the next call (the model no longer sees, and copies, its repeated reply; the attempts list says what was tried) and the step is sampled 0.3 hotter per repeat. Table XXI gives the interleaved comparison on the same ten System 1 tasks: with `unstick` the agent verified 7 in 27 calls and 305 s, without it 5 in 35 calls and 523 s; the two discordant tasks both favour `unstick`, so it is on by default. The value-carrying evidence was in both arms and is not isolated; the Phase 11 arm verified 4 of 10 in round 1 and 5 without `unstick` in round 2, which is consistent with it helping but is not an interleaved comparison.

On System 2 every call of round 1 spent its full reasoning budget, so round 2 compared 512 and 256 reasoning tokens per step on six tasks. Both verified all six. Halving the budget halved the time of the four tasks it solved in one call (about 55 s against 100 s), but two tasks then needed four and six calls, 5 of its 14 steps were malformed, and the arm took 858 s against 718 s: a smaller budget is not faster overall. The malformed steps exposed a gap in the grammar. With a short file only a whole file is offered, and System 2 wrote a Codex patch *inside* the ```` ```python ```` block, which the grammar allowed (round 1's two such patches applied; round 2's four did not). A code line may no longer start with ⋆ (a top-level Python line never does), so the edit policy is now enforced by the grammar, not only by the prompt.

Round 3: a cheap first attempt. With patches kept out of

TABLE XX
CODE AGENT, INTERLEAVED LIVE A/B ON THE REFERENCE MACHINE (BENCHMARKS/HARNESS_AB.JSON). SAME TASKS, MODEL AND BUDGET (6 STEPS) PER ARM; ARM ORDER ROTATED PER TASK. MALFORMED: STEPS WHOSE REPLY HELD NO ACTION OR A PATCH THAT DID NOT APPLY. PREFILL: PROMPT TOKENS THE SERVER COMPUTED; CACHE: SHARE OF PROMPT TOKENS SERVED FROM ITS CACHE. WALL TIMES ARE FROM A SHARED HOST AND ARE NOT SPEED CLAIMS.

Arm	Verified	Calls	Malformed	Prefill tok.	Cache	Decoded tok.	Wall (s)
System 1 (Qwen3-1.7B), 10 tasks							
Phase 10 loop	2/10	50	23/50	6,415	75 %	2,793	356
Phase 11 (auto edits)	4/10	41	1/41	7,345	78 %	4,146	504
Phase 11, patch always	0/10	60	16/60	15,393	76 %	10,706	1,468
System 2 (Qwen3-4B-Thinking), 3 tasks							
Phase 10 loop	0/3	15	13/15	1,449	85 %	9,907	2,695
Phase 11 (auto edits)	3/3	3	0/3	741	23 %	1,818	451

TABLE XXI
ROUND 2, ON THE BUILD WITH VALUE-CARRYING EVIDENCE (BENCHMARKS/HARNESS_AB_ROUND2.JSON): UNSTICK ON SYSTEM 1 AND THE REASONING BUDGET PER STEP ON SYSTEM 2. COLUMNS AS IN TABLE XX.

Arm	Verified	Calls	Malformed	Prefill tok.	Cache	Decoded tok.	Wall (s)
System 1 (Qwen3-1.7B), 10 tasks							
Phase 11 without unstick	5/10	35	0/35	6,647	79 %	3,926	523
Phase 11 with unstick	7/10	27	0/27	3,283	80 %	2,575	305
System 2 (Qwen3-4B-Thinking), 6 tasks							
Phase 11, 512 reasoning tokens	6/6	7	0/7	276	89 %	4,175	718
Phase 11, 256 reasoning tokens	6/6	14	5/14	1,612	81 %	4,748	858

TABLE XXII
ROUND 3, ON THE BUILD WHOSE GRAMMAR KEEPS PATCHES OUT OF CODE BLOCKS (BENCHMARKS/HARNESS_AB_ROUND3.JSON): THE SYSTEM 2 REASONING BUDGET FIXED OR RAISED AFTER A FAILURE. COLUMNS AS IN TABLE XX.

Arm	Verified	Calls	Malformed	Prefill tok.	Cache	Decoded tok.	Wall (s)
System 2 (Qwen3-4B-Thinking), 6 tasks							
512 reasoning tokens every step	6/6	6	0/6	140	93 %	3,499	560
256 on the first step, then 512	6/6	6	0/6	6	100 %	1,961	282

code blocks, round 3 compared the 512-token budget on every step with 256 tokens on the first step and 512 after a failure, on the same six System 2 tasks (Table XXII). Both arms verified all six, each in one call per task, so the escalation was never needed; the cheap first attempt took 282 s in all against 560 s, half the time, and decoded 1,961 tokens against 3,499. It is the default (`first_step_thinking_budget = 256`). The same build primes llama-server's cache with the agent's system prompt at startup: the first call of round 3 found it cached (207 tokens) where round 1's first call found 5, and over the round 100 % of the prompt tokens of the faster arm came from the cache (partly because both arms send the same task message, so the second arm of a task finds it cached); prefill was 1 s of its 282 s. On this machine a verified System 2 fix of a small function now takes 33–59 s, against single calls of 500–1,041 s and no verified task in the Phase 10 live run; the hardware and the models are the same.

TABLE XXIII
FINAL VALIDATION RUN (29 OCTOBER 2026, REFERENCE MACHINE).

Suite	Result	Covers
Rust tests	190/190	nine crates, with real Lean 4.35.0
`fmt`, `clippy`	clean	`-D warnings`; Linux code via cross target
API contract	pass	records, SSE, live bounds
Security	26/26	sandbox escape, injection, auth, limits
End-to-end	65/65	every endpoint, SSE, every UI page, phone width
Rooms	59/59	presence, queue, delete, reconnect
Artefacts	pass	OpenAPI, schemas, TOML, requirements
Harness	40/40	Lean, code credit, validation, agent
MVP acceptance A1–A9	9/9	fast path to degraded mode

XIII. PHASE 12: END-TO-END VALIDATION OF THE WHOLE SYSTEM

Phase 12 (29 October 2026) ran every automated suite of Nafs-Aql against the live stack on the final code, on a quiet host (load average about 2 at the start), and added one suite for the harness of Sections XI and XII, which had no end-to-end coverage of its own. The sweep found three defects in the Phase 10 integration; all three are fixed and the suites were re-run in full afterwards (Table XXIII).

A. The harness sweep

`scripts/harness_sweep.py` has a deterministic part and a model part. The deterministic part (35 checks) drives the gateway with the real Lean toolchain: three accepted proofs, a partial proof (0.5), a Lean error with a position, and six attacks (a hidden `sorryAx`, a user axiom, `native_decide`, an `#eval`, a changed statement, a shadowed instance), each rejected by the layer the protocol assigns to it. It also checks per-assert credit, head-and-tail evidence, that evidence names `Probe.lean` and never the sandbox path, and 14 request-validation cases. The model part runs a Lean theorem through `/v1/chat`, the code agent on a fix task (verified in one call, 43 s) and a System 2 code chat whose candidate tree is recorded (verified, 275 s).

B. Defects found and fixed

1) **Inconsistent validation.** `/v1/chat` refused a Lean task with an unusable statement (400), but `/v1/verify` returned a *failed* verdict with HTTP 200, so a client could not tell a wrong proof from an ungradable task. Validation is now one function used by both.
2) **Lean tasks were classified as open questions.** The domain classifier knew tests, schemas and reference answers but not `verification.lean`, so a proof request received the open-question limits (no retries, 512 reasoning tokens) and made a single attempt. Lean tasks are now mathematics (up to three attempts, 1024 tokens); a unit test pins it.
3) **The prompt did not say that Mathlib is absent.** The model wrote `import Mathlib`, which fails on a core-Lean toolchain. The prompt now states which library is available, and adds a three-line Lean 4 syntax note (a proof is a `by` block, not `begin...end`; try `rfl`, `simp`, `omega`; the `induction ... with | zero | succ` form).

C. What the sweep says about the models

With these fixes the pipeline is correct end to end, which the deterministic integration test shows with the real binary: the System 1 draft fails Lean, System 2's first proof fails with a Lean error, the error appears in the next prompt, and the second proof is verified. The live models do not reach that point. For the theorem $n + 0 = n$, which `rfl` proves, Qwen3-4B-Thinking abstained after three attempts (826 s); streaming the rounds shows Lean 3 syntax from System 1 and, from System 2, `induction ... with` followed by an identifier, a stray inner `by`, and the same wrong proof on consecutive attempts. Every verdict was precise and no wrong proof was ever accepted: the system abstained with a notice, as Theorem 1 requires. The honest summary is that a 4B model quantised to 4 bits cannot yet write Lean 4 proofs on this CPU even for trivial statements; the verifier, the repair loop and the abstention are ready for a stronger prover, and the sweep's model checks therefore assert *honesty* (verified, or abstained with a notice, never an unchecked proof) rather than success. Code tasks, which the same models handle, verify.

TABLE XXIV
BATCHED DECODE, $N = 4$ STREAMS AGAINST ONE (REFERENCE MACHINE, IDLE).

Model, context	1 stream	4 streams	gain	$c(4)$
System 1, short	15.8	36.9	2.34×	1.71
System 2, short	5.12	13.78	2.69×	1.60
System 2, 800 tokens	5.03	9.17	1.82×	2.24
System 2, 1,600 tokens	4.09	6.14	1.50×	3.40

XIV. PHASE 13: PARALLEL AND CONCURRENT INFERENCE ON FOUR CPU CORES

Phase 13 (30–31 October 2026) asked how far the four cores can be pushed by running work in parallel. The assessment of the 100-prompt run (Section IX) is the starting point: the gate and the exact checks are not the cost (under 2.5 % of model time); decode is, with System 2's thinking at 77.6 % and drafts at 17.9 %, and six prompts (failed code and timeouts) used 52 %. The research note is `docs/research/parallel_inference.md` and the decision is `docs/decisions/007-parallel-inference.md`.

A. Literature and what transfers

Repeated sampling with an exact verifier raises coverage log-linearly with the number of samples [46]; hedged requests send a request to several replicas, keep the first answer and cancel the rest [167]; parallel streams over shared parameters behave like a larger model at a fraction of the cost of parameter scaling [168]; confidence can cut parallel traces early [153]; instances can attend to a shared cache [169]; model-free speculation drafts from earlier text [170]; table-lookup kernels speed low-bit models on CPUs [171]. The first two need only what Nafs-Aql has (an exact verifier and a batching server) and were implemented. The others need trained transformations, models that cooperate (small ones are reported not to), or a different runtime, and were not.

B. The measurement that decides: batching and context depth

llama-server decodes its slots in one batch. Table XXIV gives the throughput of N concurrent streams on Qwen3-4B-Thinking (decode only, prompt cached) at three context depths, and Qwen3-1.7B at short context.

One stream is bandwidth-bound (about 16 GB/s of weights per second at both sizes), so a few more streams are cheap, until compute saturates; but the gain falls with context, from 2.7× to 1.5× for System 2, and a repair prompt (the earlier candidate and the verifier's evidence) is about 1,600 tokens deep. A quantised KV cache with flash attention does not restore it (7.94 against 9.17 tok/s at 800 tokens, 6.36 against 6.14 at 1,600). Self-speculation does not help either: four llama.cpp n-gram modes gained at most 1.12× (code repair, 71 % of drafted tokens accepted) and lost up to 16 % (JSON), because verifying k drafted tokens costs about k steps on a compute-bound CPU.

C. One request with n choices

Candidates sent as separate requests make every slot prefill the whole prompt, and prefill (31 tok/s on System 2) is compute-bound and gains nothing from batching. With a 703-token prompt and a 24-token answer one request takes 31.7 s, four separate concurrent requests 124.5 s (3.9×) and one request with $n = 4$ 39.1 s (1.23×): llama-server prefills once and forks the KV cache. `Backend::stream_n` sends a round as one such request and delivers each choice when its own stream ends, so it can be verified while the others decode. The first live hedging runs, which used separate requests, were slow for exactly this reason.

D. Fusing the agreement samples with the draft

For an unverified maths draft the gate always draws $k = 2$ cheap samples (A-060), after the draft: $1 + c(2) = 2.33$ draft times. Drawn in the draft's batch they cost $c(3) = 1.78$, and they are dropped when the draft is stopped early (it is going to System 2 whatever they say). In an interleaved A/B (14 GSM8K prompts, two repetitions, the stream closed at the gate's decision so no System 2 run is started) the median time to the decision fell from 23.1 s to 17.9 s (−24 %), with the fused arm faster in 23 of 28 pairs (exact sign test $p = 0.0005$). Escalations were 7 and 5 of 28 (discordant pairs 4 to 2, McNemar $p = 0.69$), and the decision flips between two repeats of the same arm as often (6 of 28) as between arms, so the difference is sampling noise.

E. When does a concurrent round pay?

Take $R + 1$ rounds, one-attempt success probability p and batch cost factor $c(N)$, and let candidates be independent. A round of N succeeds with $1 - (1 - p)^N$, so

$$U(N) = \frac{1 - (1 - p)^{N(R+1)}}{c(N) \sum_{k=0}^{R} (1 - p)^{Nk}} \tag{7}$$

is the verified answers per single-attempt time. Widening to N pays iff $U(N) > U(1)$.

Proposition 20 (Limits of widening)**.** [exact] *As $p \to 0$, $U(N)/U(1) \to N/c(N)$: widening pays iff the batch is cheaper than the same streams in series. As $p \to 1$, $U(N)/U(1) \to 1/c(N) < 1$ for any $c(N) > 1$: it never pays.*

Proof. For small p, $1 - (1 - p)^{N(R+1)} \approx N(R + 1)p$ and $\sum_k (1 - p)^{Nk} \approx R + 1$, so $U(N) \approx Np/c(N)$ and $U(1) \approx p$. For $p \to 1$ the numerator tends to 1 for every N, and the sum to 1, so $U(N) \to 1/c(N)$. □ □

Consequence. With the short-context costs the policy widens for $p \leq 0.48$; with the 1,600-token costs $[1.0, 2.4, 3.1, 3.4]$ it widens only for $p \leq 0.07$ (hopeless tasks), because $c(4) = 3.4$ is close to 4. The 100-prompt run also shows that attempts are not independent: of the eight code items that reached System 2, four passed on the first attempt and four failed all three, and no repair rescued one.

TABLE XXV
EIGHT HARD CODE ITEMS, SYSTEM 2 FORCED (WALL TIME IN S; THE SEQUENTIAL LOOP WAS RUN TWICE).

Arm	Verified	Wall
sequential (1 + 2 serial repairs), run 1	4/8	4,052
sequential, run 2	3/8	4,523
four fresh at once, separate requests	4/8	3,308
first alone, then four, one $n = 4$ request	4/7	3,558

F. Hedging on System 2: implemented, measured, off

`nafs_core::parallel` and the System 2 loop implement it: one future per candidate that generates, applies a patch and verifies, so verification overlaps the other candidates' decoding; candidates consumed in completion order; the first that passes wins and dropping the rest cancels their streams; the width from Eq. (7) with an online Beta estimate of p; a `repair_rounds` override; `hedge_after_failure`, which runs the first attempt alone; and a round deadline 5 s inside the request cap. Table XXV is the live A/B on those eight items (System 2 forced, idle host).

Four concurrent fresh candidates cost the easy items +44 to +87 % and saved only on failing ones; the one-request variant was no faster on the failing items (872–930 s against 636–901 s), because a round at repair depth costs 2.2 to 3.4 single attempts (Table XXIV). The two sequential runs disagree with each other (4/8 and 3/8), so no accuracy difference can be claimed at this size. Hedging on System 2 is therefore *not* adopted: `deliberation.parallel.enabled = false`. Running it also exposed three defects, all fixed: separate requests prefilling W times; the next attempt's budget sized from the single-stream speed, which let a round overrun the request cap (HTTP 504); and an error in a round's last candidate discarding the best candidate so far.

G. Cheap concurrent samples before System 2

Where batching is cheap (System 1, short context) and the verifier is exact, the cores are better spent *before* the expensive model. When a draft fails its exact check, Nafs-Aql now draws $k = 4$ System 1 samples in one request, verifies each as it finishes and keeps the first that passes. Let f be the share of requests whose draft fails, r the share of those that a sample rescues, ρ the cost of the rescue stage (paid on every failed draft) and E the cost of an escalation.

Proposition 21 (Rescue break-even)**.** [exact] *The saving per request is $f\,(rE - \rho)$, positive iff $r > \rho/E$.*

Proof. Of the fraction f with a failed draft, a share r avoids the escalation ($-E$) and all pay ρ. □ □

Offline on 60 MBPP+ items the draft passes 46.7 % of the time; of 32 failed drafts, 0, 0, 4 and 6 are rescued by 1, 2, 3 and 4 samples (19 % with four, pass@5 56.7 %), and four samples in one request take 17.5 s against 10.4 s for the draft. Live, in an interleaved A/B of 40 items (rescue off against on, stream closed at the decision), escalations fell from 22 to

TABLE XXVI
PHASE 13 OUTCOMES.

Change	Default	Evidence
Fused agreement samples	on	$-24\,\%$, $p = 0.0005$
System 1 rescue (4 samples)	on	3 of 19 avoided; break-even 106 s
System 2 with 4 slots	on	2.7× to 1.5× decode
Concurrent hedging, System 2	off	no width pays at depth
n-choice requests	used	39.1 against 124.5 s
n-gram speculation	no	$\leq$ 1.12×, down to 0.84×
q8_0 KV cache	no	no gain at depth

TABLE XXVII
ENGINE FLAGS ON THE SHARED HOST: MEDIAN TOKENS/S OVER THREE INTERLEAVED ROUNDS, RELATIVE TO THE BASELINE (`-t 4`, FLASH ATTENTION AUTO, MICRO-BATCH 512, POLL 50). LOAD BEFORE A ROW: 8–67, MEDIAN 14.

Flag	S1 pp512	S1 tg64@1024	S2 pp512	S2 tg64@1024
baseline (tokens/s)	87.9	8.4	35.0	3.45
`-fa off`	0.79×	0.66×	0.88×	0.25×
`-fa on`	0.79×	0.31×	0.58×	0.18×
`-ub 256`	0.96×	0.69×	0.42×	0.26×
`-ub 1024`	0.96×	0.94×	0.55×	0.32×
`-t 3`	0.86×	1.02×	0.64×	0.51×
`-poll 0`	0.93×	0.56×	0.64×	0.50×
`-lm mlock`	1.18×	0.70×	0.73×	0.35×

20 (three avoided, one in the other direction, since the draft itself is random); the rescue added a median 14.5 s when it ran and 317 s in all. Break-even is $317/3 = 106$ s per avoided escalation, against 250–580 s measured for an escalation (the eight code items of the 100-prompt run: median 289 s, mean 580 s), so the rescue nets $+10.8$ to $+35.6$ s per code item and trades no accuracy: every rescued answer passed the exact check. The sample is small and the two studies agree on the rate; it is on by default (`gate.rescue_samples = 4`), and Lemma 1's call bound gains that term.

H. Serving topology and summary

System 2 now has four slots and 24,576 tokens of context (6,144 per slot) where it had two, since the 100-prompt run lost 476 s to waiting for a System 2 slot. Table XXVI summarises what was adopted and what was not. Every suite was re-run on the final code (31 October 2026): 206 Rust tests with real Lean, security 26/26, end-to-end 65/65, Rooms 59/59, the harness sweep 50/50 (it now covers the rescue, the parallel settings and their validation), and the nine acceptance tests, where Lemma 1's call bound for the maths request is $1 + k + \text{rescue} + N(R+1) = 10$ and 8 calls were used.

XV. PHASE 14: ENGINE KNOBS, THE MODEL SWITCH, AND A CASCADE IN THE HARNESS

Phase 14 (1 November 2026) asked three questions of the system as it stood after Phase 13: what the inference engine's own settings can still give on the target machine, how fast the request moves from one model to the other (and the service from one backend to another), and what of Codex's agent core the harness still lacks. The research note is `docs/research/engine_switching_harness.md` and the decision is `docs/decisions/008-engine-switching-harness.md`. The method is the one the project has used since Phase 7: measure first on the reference machine, interleave the arms, and keep only what the numbers justify. This phase ran while another workload of the owner's shared the four cores (load average 8–67 against about 2 when idle), which decides what can be concluded from the engine measurements (Section XV-B).

A. Literature and what transfers

Phases 7–13 covered what changes *what* the engine computes: speculative decoding [84] in three forms, weight and cache formats, batching across slots [172], and prefix caching [116], [162]. Three directions remained. (i) *How* the engine computes: flash attention on a CPU, the micro-batch size, the thread count, and the busy-wait level at which ggml's worker threads spin while waiting for each other, which matters when two servers' eight threads share four cores. (ii) The switch between models: with both models resident (1.2 and 2.8 GB of 32), a switch is routing plus a warm prefix, not a load, so the checkpoint-loading systems built for GPU memory pressure [173] do not apply; what applies is making the prefix warm before the switch. (iii) A *cascade* [74], [75] inside the harness: the quick model first, the thorough one only when needed, with the exact verifier as the cascade's scorer, which is the gate's logic applied to the agent. Ternary models and table-lookup kernels [171], [174] would change the model, not the engine, and no Qwen3 variant exists; they were not pursued.

B. The engine's knobs

`scripts/engine_tuning.py` runs `llama-bench` on the production GGUF files under each flag: flash attention on and off, micro-batches of 256 and 1,024 tokens, three threads, no polling (`-poll 0`), locked memory; prompt processing of 512 tokens and decode of 64 tokens at a depth of 1,024 (the depth of a repair prompt). A first, block-ordered run was discarded: the owner's other workload started midway and turned single blocks by a factor of 0.4. The second run cycles the arms in three rounds, one repetition each, so a burst of load hits every arm, and records the host load before every row. Table XXVII gives the medians. The verdict is *inconclusive, flags unchanged*: the per-round ratios of a flag against the same round's baseline swing from 0.07× to 3.8× (System 2 decode under flash attention: 0.15, 0.07, 1.04), which is the external load, not the flag, and the baseline itself sits 2–4 times below its idle values (31 tokens/s prefill and 5–6 tokens/s decode for System 2, Phase 13). No flag is adopted on this evidence. What remains is the method, an interleaved and load-labelled matrix to re-run on an idle host, and the rule in decision 008 for what counts as a win (a median over rounds above the baseline on both tests for the model the flag serves).

C. Co-scheduling the two servers

Both servers run four threads on four cores. `scripts/cosched_ab.py` starts a temporary pair

TABLE XXVIII
CO-SCHEDULING ON FOUR CORES (SHARED HOST, LOAD 6–11; MEDIANS OF TWO REPETITIONS). "KEEPS" IS THE RATE WITH THE OTHER SERVER BUSY OVER THE RATE ALONE.

Variant	S2 alone	S2+S1	keeps	S1 alone	S1+S2	keeps	both
base	3.62	2.29	63 %	9.87	5.21	53 %	7.50
`-poll 0`	3.67	2.37	65 %	10.02	5.45	54 %	7.82
S1 `-t 2`	4.33	2.02	47 %	6.88	5.49	80 %	7.52
S1 `-t 3`	4.11	2.00	49 %	9.40	5.42	58 %	7.42
`-poll 0`, S1 `-t 2`	6.37	1.69	27 %	9.55	5.40	57 %	7.09

per variant and measures System 2 decoding alone and with System 1 decoding beside it, and the reverse, two interleaved repetitions each (Table XXVIII). When both decode, each keeps about half to two thirds of its solo rate and the pair writes 7.1–7.8 tokens/s together in every variant: the cores are the bottleneck, and the flags only move the split. Polling off does not help and priorities are unavailable on macOS. Freezing the quick model while the thorough one works would cut the pair's throughput and the other user's time to first token for no gain in the thorough model's, so nothing was built; the serving script is unchanged.

D. The model switch

Inside a request the switch is the gate: draft, exact check, rescue batch, decision, System 2. `scripts/handoff_latency.py` times it on the live gateway from the SSE events and both servers' metrics, with the rescue on and off, interleaved. Four requests whose draft fails its exact check, two with the rescue on and two off, on the shared host (load 7–12): the gate's own work takes 0.8–0.9 s (26.9–34.5 s when the four-sample rescue batch runs, the batch Phase 13 measured to pay for itself); from the decision to System 2's first reasoning token 1.1–4.7 s (median 1.5 s), with 5–27 new prompt tokens against 262–1,148 cached; and System 1 reports no busy slot after the decision in any run. The switch inside a request is already as fast as the engine allows: there is nothing to freeze (an idle llama-server costs no CPU), and a prefetch of the System 2 prompt during the draft could save at most the 1.1–4.7 s above at the price of contending with the draft, so none was built.

Between backends, `POST /v1/system/backend` now warms the target pair before the swap: every constant prefix (the system prompt per domain on both models, the agent's prompt) is primed on the new servers, then the active pair is exchanged in one atomic pointer swap. Requests in flight hold the pair they started on and finish there; new requests find the new pair's caches warm (about 1 s to the first token against about 9.8 s cold, decision 005). The response reports the prefixes warmed, the warm-up time and the requests still running on the previous pair.

E. A cascade in the harness

Codex's agent core offers three things Phase 11 did not take: parallel tool calls, dispatch while the model still streams, and retries with backoff of a failed sampling request. The first two have nothing to overlap here (one action per reply; under the grammar the server stops at the end of the action). The third was missing and is now in: a model call that fails for a reason other than the run's cap is retried up to twice with Codex's backoff (200 ms, doubling, jitter 0.9–1.1).

TABLE XXIX
THE HARNESS CASCADE AGAINST THE PHASE 11 DEFAULT, TEN TASKS INTERLEAVED, SYSTEM 2 AS THE RUN'S MODEL (SHARED HOST).

Arm	Verified	Calls	Steps S1 / S2	Wall	Median
Phase 11 default	10/10	12	0 / 12	1,987 s	96 s
Cascade (4 candidates)	10/10	14	10 / 4	**1,467 s**	100 s

What the harness lacked was Nafs-Aql's own idea. The agent ran on System 2 from its first step, while System 1 verified 7 of the same 10 tasks at about 11 s a step (Phase 11, round 2) and four System 1 candidates in one request cost 1.7 times one (Phase 13). With `harness.cascade` the first step goes to System 1 as one request with `cascade_samples` choices (the prompt prefilled once, the choices decoded in one batch, at temperature 0.7 so they differ); each candidate's action is applied to its own copy of the files and judged by the exact verifier in order; the first that passes ends the run and the rest are not judged; when none passes, the best-scoring candidate (per-assert credit) becomes the state, its reply and observation stay in the transcript, and System 2 continues from step two as before. A candidate is offered no `run` (it is judged by its files), and a run asked for on System 1 itself gets no cascade. The A/B interleaves the cascade with the Phase 11 default on the same ten tasks (`scripts/harness_ab.py --arms s2,cascade`). Table XXIX gives the result on the shared host (load 5–16): both arms verify all ten tasks with no malformed step; the cascade takes 1,467 s against 1,987 s ($-26\,\%$) and is faster in 7 of 10 pairs (exact sign test $p = 0.17$: indicative at $n = 10$). Six of the ten tasks never reach System 2: a four-candidate System 1 request verifies them in 10–101 s (median 34 s against 85 s for the same tasks on System 2). The other four pay one System 1 step before System 2's; two still end faster, two slower, within the variance of System 2's own time (the sequential arm took 342 s and 322 s on two of them). The cascade decodes more tokens in total (5,958 against 4,534) but on the cheap model in one batch; what it saves is System 2 steps, 4 against 12. The default is now `harness.cascade = true` with four candidates; decision 008 says what re-opens it.

F. Summary

Table XXX summarises the phase. Three things changed: the cascade (on), retries with backoff (on), and the warm-first backend switch (on). Three were measured and left alone, with the method and the re-open rule recorded: the engine flags (inconclusive on a shared host), co-scheduling (nothing to arbitrate on four cores), and the switch inside a request (already about 1.5 s; nothing to freeze, no prefetch worth building). Every suite was re-run on the final code (Section XIII lists them); the counts are in the decision record.

TABLE XXX
PHASE 14 OUTCOMES.

Change	Default	Evidence
Harness cascade (FR-097)	on, 4	−26 % wall, 7 of 10 pairs
Retries with backoff (FR-098)	on, 2	unit tests
Warm-first backend switch (FR-099)	on	9.8 s against 1 s cold/warm
Engine flags	unchanged	inconclusive, shared host
Co-scheduling	unchanged	7.1–7.8 tok/s in every variant
Prompt prefetch for System 2	not built	switch costs 1.1–4.7 s

XVI. PHASE 15: SMILEY CORE AS THE THIRD INFERENCE BACKEND

Phase 15 (1 November 2026) integrates Smiley Core, the owner's own CPU inference engine from the Smiley project, as a third backend beside llama.cpp and vLLM. The research note is `docs/research/smiley_backend.md` and the decision is `docs/decisions/009-smiley-backend.md`.

A. The engine

Smiley Core is a Rust library of about 8,500 lines with one dependency: a GGUF loader, a byte-level BPE tokenizer checked against `llama-tokenize`, the `llama`, `qwen2` and `qwen3` forward passes with AVX2 and FMA kernels, an F16 or int8 KV cache, a chat session with the models' own templates (including Qwen3's `enable_thinking`), prompt-lookup and DSpark speculative decoding, and telemetry that reports the bytes moved per token beside the tokens per second, so a run can be checked against the machine's memory roofline rather than taken on faith. It decodes one stream per session: no batching across requests, no constrained sampling, no raw continuation. Both Nafs-Aql models pass its compatibility probe.

B. Integration

The gateway already speaks one OpenAI-compatible API to two kinds of server, so the integration is a server around the engine and a configured pair. The engine is vendored into the workspace (`rust/crates/smiley-core`, since `third_party/` is not under version control) with two marked additions: a continuation prefix on the session (what the assistant has already written, appended after the template's assistant header) and the sampled token's log-probability on each emitted token. `nafs-smiley-server` serves one model per process: `POST /v1/chat/completions` streamed or not, with `usage` and llama-style `timings` plus a `smiley` block carrying the bytes moved; `/v1/models` for the health check; `/metrics`. The system message becomes the session's system text, earlier exchanges are seeded as history, the last user message is the turn; the session is reset per request and reuses its cache prefix when the prompt shares it. `enable_thinking` maps to the session's thinking switch. The reasoning budget is enforced by counting reasoning tokens and, at the budget, replanning the turn with the reasoning so far plus `</think>` as the assistant's prefix, so the answer follows as under llama-server's budget forcing; the cache prefix is reused and only the forced tail is prefilled. `logprobs` are served for the gate's fixed rule. Requests with `n > 1` or a grammar are refused, so on this backend the gateway runs without the rescue batch, the harness grammar and the raw-continuation exit, as it does on vLLM. `[backends.smiley]` configures the pair (:8201, :8202, one stream each); `make up` builds and serves it; it is the default backend after the comparison below, and `make up-llama` switches the gateway to llama.cpp, warm-first.

C. Measurements

Two comparisons ran on the shared four-core host, the arms interleaved and the load recorded per request (`scripts/smiley_ab.py`, `scripts/backend_ab.py`).

The engines alone, with identical greedy requests (12 pairs per model). Decode speed was 14.3 tokens/s for Smiley Core against 12.6 for llama.cpp on System 1 and 7.4 against 6.6 on System 2; Smiley Core was faster in 10 of 12 and 11 of 12 pairs, by a median of $1.13\times$ and $1.14\times$. Its first token is later on System 1 (0.64 s against 0.11 s: the session is reset and the prompt replanned per request) and earlier on System 2 (1.60 s against 1.85 s). Answers are identical in 12 of 12 System 1 pairs and 9 of 12 System 2 pairs. The bytes moved per token (1.09 GB and 2.50 GB) put both models on the memory roofline at 15.6 and 18.5 GB/s, so the gain is the engine's own overhead and not a different algorithm.

The whole service, with the gateway switched between the backends before every request (warm-first) and the same twelve prompts sent in Auto mode (four maths with a reference answer, three code with hidden tests, two JSON schema, three open or factual):

	llama.cpp	Smiley Core
Checked answers (9 checkable)	9/9	9/9
Wall time, median per request	17.9 s	**9.7 s**
Wall time, 12 requests	303 s	**131 s**
First token, median	**0.97 s**	1.50 s

Smiley Core was faster in 10 of the 12 pairs (one-sided sign test $p = 0.02$), by a median of $1.59\times$. The code agent's first run on Smiley Core failed at the 900 s deadline and exposed a harness defect that is not the engine's: without a grammar the harness reads free text, and Qwen3-1.7B writes the patch without its `*** Begin Patch` line (the same on llama.cpp when unconstrained), which the parser read as no action. The patch finder and the stream cut now accept a patch that starts at its first file section (unit-tested). After the fix both backends verified the task: 263 s on Smiley Core, 53 s and 291 s in llama.cpp's two runs, which one run per backend cannot rank.

D. Sampling and the acceptance suite

llama-server samples with `top_k = 40` and `min_p = 0.05` by default; Smiley Core had temperature and top-p only, and System 2 on it wrote 1,025 reasoning tokens (the budget) where llama.cpp wrote 274 on the same question. The engine

gained both filters (off by default in the engine, on at llama-server's values in the server, unit-tested). Over three seeds of one GSM8K question Smiley Core then wrote about 12 % more reasoning tokens (1,037 against 925 on average) in about 16 % less wall time (132 s against 157 s). The nine MVP acceptance tests pass on Smiley Core (9/9); on a host still loaded by the preceding sweep two of them failed on time (first token 9.0 s against 2 s; slow path 309 s against 240 s) and passed on a quiet one (0.06–1.7 s; 193–195 s). The slow-path budget is within reach on Smiley Core only on a quiet host: System 2 decodes 7.4 tokens/s and a 1,024-token reasoning span is 138 s of it. The full sweeps on the final build: 309 Rust tests, contract, security 26/26, Rooms 59/59, end to end 65/65, harness sweep 53/53 (two checks assumed llama.cpp's four slots and now follow the active backend).

E. Summary

On the criterion fixed before the run, the time to a checked answer through the gateway with the answers equally checked, Smiley Core wins for one user at a time and is the default backend (`backend.active = "smiley"`; decision 009). What it gives up is what the engine does not have: one stream per model, so a second concurrent request waits, the rescue and the cascade have one candidate instead of four, there is no grammar (the agent relies on the hardened free-text parser) and no raw continuation (the adaptive exit is off). llama.cpp keeps all four and is one command away (`make up-llama` or the System page); it is the choice for a shared deployment or a workload dominated by the code agent.

XVII. Phase 16: The Code Agent Against Modern Harnesses

Phase 16 (2 November 2026) asks what the current agent harnesses and papers do that Nafs-Aql's code agent does not, which of those differences matter on four CPU cores with 1.7B and 4B models, and which can be closed and measured here. The research note is `docs/research/harness_gaps.md` and the decision is `docs/decisions/010-harness-gaps.md`.

A. What modern harnesses do

The sources fall into two groups. Four were read in full: the SWE-agent paper [159], the Agentless paper [175], the JetBrains study of observation masking [176], and Anthropic's article on context engineering [177]. The rest was read through search results (documentation pages, the DeepSeek-V3.2 report [178], Gemini CLI's documentation [179], the SWE-Search and SWE-Gym papers [180], [181]); a number from a vendor's own announcement is a vendor number.

- **An edit-time guardrail** (SWE-agent's agent–computer interface). After every edit the file is checked, a failing edit is reverted, and the agent sees the error and the lines around it. On SWE-bench Lite with GPT-4 Turbo the resolve rate was 18.0 % with the check, 15.0 % without it and 10.3 % with no edit command at all [159].
- **Context management.** Compaction, dropping old tool results, notes kept outside the window and sub-agents that return a condensed result [177]; masking the observations older than ten turns halves the cost with the solve rate of LLM summarization across five model configurations [176].
- **Selection by a verifier.** Majority voting resolved 25.67 % of SWE-bench Lite, regression tests added 27.00 % and reproduction tests 32.00 % [175]; trained critics rerank trajectories [181]; search backtracks to a better node [180].
- **Checkpoints and project instructions.** Gemini CLI snapshots the project before file-modifying actions and reads a hierarchy of `GEMINI.md` files; Codex and Claude Code read `AGENTS.md` and `CLAUDE.md`.
- **Stall detection.** Loop guards key on the tool name, its arguments and its result; repetition warns, repetition with stagnant results stops. This is engineering practice; no study was found.
- **Reasoning persistence.** DeepSeek-V3.2 keeps its reasoning across the tool calls of one turn and drops it when a new user message arrives [178].

B. The gaps

Nafs-Aql already had an append-only transcript that keeps the server's prompt cache valid, compaction written by the harness, a patch grammar, Codex's patch parser, an *exact* verifier after every change (a stronger selection signal than a critic model or generated tests), the cascade, retries and a wall-clock cap. Four gaps were real and are closed. A file that did not compile went to the verifier like any other, costing a sandbox run and a whole step, and the same `IndentationError` came back step after step. A step that made the tests do worse was kept as the base of the next one. Only an identical *workspace* was noticed as a loop, so the same failure from different code, or a reply with no action, repeated until the step budget (one run spent the whole 900 s cap that way). And the agent took the task text only.

Others are named and not closed, with the reason. Self-generated reproduction tests need a bug that fails them on the unpatched code; a new function has no such oracle, and a test written by a 1.7B model would raise the *label* without raising the trust. Reasoning persistence is not adopted now: the Qwen3-Thinking template (read from the model file) keeps an assistant turn's reasoning only after the last user query, and the harness sends observations as ordinary user messages, so each step is a new query; keeping the reasoning would mean sending tool responses in the template's form and about 25 s of prefill per kept 500-token span at the measured 20 tokens/s, and is the next experiment. Sub-agents, plan mode and hooks have no counterpart at the scale of one function in a flat workspace, where the cascade is the sub-agent idea and the sandbox is a hook that cannot be skipped. Observation masking changes nothing inside a ten-turn window, and the runs are at most twelve steps.

C. What was built

1) **A compile check before the verifier** (`harness.lint`). After a patch or a whole-file write every edited Python file is compiled in the sandbox (never executed). A file that does not compile is *not applied*: the files stay, and the

model is shown the error and the five lines around it with the offending one marked. No verifier run is spent, and a cascade candidate that does not compile is not judged. A failure of the sandbox itself passes the edit on to the verifier.

2) **Rollback to the best version** (`harness.rollback`). A change judged below the best version so far is dropped, the files are restored to that version, and the model is told what was tried; the failed attempt stays in the transcript.
3) **A stall guard** (`harness.stuck_after`, default 4). A failed step's signature is the hidden tests' evidence, or the action and its first line of feedback when the files were not judged. The second consecutive identical failure carries a nudge to change approach; the k-th ends the run with status `stuck` and the best files.
4) **Caller guidelines** (`guidance`, 2,000 characters), shown with the task and again after a compaction.

Each is a request field and a configuration key, on by default, and reported in the reply (`lint_rejections`, `rollbacks`, the step's `rolled_back`, the status `stuck`). Five unit tests drive the loop with a scripted host, and an integration test runs the compile check through the real sandbox.

D. Measurements

All runs were on the shared host (the owner's other workloads at load 5–15) with Smiley Core, over the same ten tasks (eight repairs and writes of one function with hidden tests), the arms interleaved and their order alternated per task: guards off (`lint`, `rollback`, `stuck_after=0`) against the production defaults.

Set	Arm	Verified	Model calls	Wall time
System 2, 6 steps	off	10/10	17	747 s
	on	10/10	17	772 s
System 1, 8 steps, round 1	off	4/10	58	307 s
	on	4/10	58	297 s
System 1, 8 steps, round 2	off	5/10	47	350 s
	on	5/10	47	347 s

The guards cost nothing: the verified counts and model calls are identical, the decoded tokens are equal except for one System 1 task whose sampling differed (2,418 against 2,362 in the first round), and the wall times agree within the host's noise. The compile check costs 37 ms per edit (35 ms of it Python's start-up) against steps of 5 to 230 s. They also changed no outcome here: neither model produced a file that does not compile, and rollback restored nothing, so the benefit lies in the failure modes the research names, which this set rarely produces. Those are pinned by tests and were seen once in practice, as a Smiley Core run that spent its whole 900 s cap on repeated `IndentationError`s. The A/B did expose a defect in the first version of the stall guard: System 1 asked six times in a row to run a file that does not exist, and the guard skipped `run` steps. It now counts them, hashes only the stable lines of the feedback, and the message lists the files that exist. The same ten System 1 tasks verified four times in the first round and five in the second in both arms, so a one-task difference is sampling noise.

E. Summary

The research found one capability the code agent lacked that matters at this scale, the edit-time guardrail, and three smaller ones (checkpoint and restore, stall detection, project instructions); all four are built, on by default, tested, and measured to cost nothing. Self-generated tests, reasoning persistence and sub-agents are named as gaps with the reason they were not closed. The verifier-based selection that other systems approximate with critic models is the one place Nafs-Aql is ahead: its verifier is exact.

XVIII. Phase 17: Smiley Core's Speed, the Model Switch and the Harness's Cost

Phase 17 (2 November 2026) assesses Smiley Core, the switch from System 1 to System 2, and whether the harness of Phase 16 costs speed. The research note is `docs/research/smiley_engine_perf.md` and the decision `docs/decisions/011-engine-switching-costs.md`. All numbers are from the shared reference machine with the arms interleaved.

A. How close the engine is to the machine

A multi-threaded streaming read (AVX2, 1 GiB) reaches 12.9, 17.2, 18.4 and 19.5 GB/s with one to four threads: the fourth core adds 6 %. Decode reads every weight once per token, so its speed is that bandwidth over the bytes per token (1,094 MB on System 1, 2,503 MB on System 2). The engine's own phase split, now reported by the server, puts 86–97 % of decode in the matmuls and the output head; sampling is 0.1–0.3 % and attention 1–3 % at these context lengths, so neither is worth optimising. The serial share (norms, rotary embedding, residuals, activation quantisation) grows from 2 % at one thread to 5 % at four.

Three threads beat four for decode in all six interleaved pairs (+4 % on System 1, +8 % on System 2 at the median) while cold prefill, which is compute-bound, is 16–17 % slower at three. On a shared host the fourth worker is the one the scheduler preempts, and every matmul waits for the slowest worker. Decode dominates a request and most prompts are now cache hits, so the default is three threads, which also leaves a core for the verifier sandbox and the gateway; a quiet host may prefer four. The engine's own tuning is already near its ceiling: with the software-prefetch distance of 1,024 bytes as the reference, no prefetch loses 11 % in two of three rounds and 512 loses 3 %, while 2048 and 4096 differ by about $+2$ % at the median, inside this host's noise; parking instead of spinning loses 2 %.

B. What the literature offers

Decode saturating memory with few threads is well supported and is confirmed above. Prompt-lookup and suffix decoding [182] draft tokens by matching recent output against

the prompt and past outputs and report large gains where output copies input, such as code editing; the engine's speculation is greedy-only while Nafs-Aql samples at 0.3–0.7, so using it needs speculative sampling (accept a deterministic draft token with probability $p(d)$, otherwise sample the residual), which is lossless but is engine work with a bounded payoff because System 2's time is reasoning text that does not copy the prompt. Prefix caching of the kind SGLang's RadixAttention [183] describes keeps the cache of earlier prompts and reuses the longest shared beginning; it applies directly and was built.

C. Switching

System 1 to System 2. Over six escalated requests the gate's own time was 0.08–0.12 s, the decision to System 2's first reasoning token 0.4–1.4 s (median 0.85 s), and System 1 was idle at the decision. Each model is its own process with its own mapped weights, so there is nothing to freeze, copy or reload; llama.cpp took a median 1.5 s in Phase 14.

Between kinds of request on one model. The session held a single cache, so a prompt benefited only when it followed a conversation with the same beginning. Alternating a chat prompt and the code agent's prompt prefilled the agent prompt in full each time: 266 tokens in 5.8 s on System 1 and 262 tokens in 14 s on System 2, where llama.cpp's four slots answered in 0.1–0.2 s. A *prefix store* in the engine keeps the cache of each finished conversation as a snapshot (114–147 KB per position, 768 MB per model, least recently used evicted) and restores the one that shares the longest beginning with the new prompt. Alternating or rotating three kinds of request now costs one token and 0.1 s on System 1 and 0.2 s on System 2, as on llama.cpp, and the answers are byte-identical (equal hashes) with the store on and off for both models. It also makes the gateway's warm-first backend switch effective: it primes 13 prefixes, of which a one-prefix session could keep only the last.

The reasoning budget's continuation. At the budget the server closes the reasoning and continues the turn with the reasoning so far as the assistant's prefix. The prefix was `<think>`, a newline, then the reasoning, which already begins with the newline the model wrote; the first reasoning token became two newlines instead of one, the cache stopped matching there, and *every reasoning token was prefilled again*. The prefix now repeats the generated text exactly, the match covers the whole conversation, and the continuation prefills 5 tokens in 0.2 s instead of 108 tokens and 2.2 s at a budget of 200 (7–30 s on System 2 at 128–512 reasoning tokens). On six escalated requests the cold prefill fell from 700–1,160 tokens (45–78 s) to 350–390 (22–24 s) and the mean time per request from 182 s to 133 s; the runs were not interleaved with the old build, but the prefill token counts are deterministic.

D. The harness's cost

The guards of Phase 16 cost nothing in the interleaved A/B over ten tasks, and the compile check takes 37 ms per edit against steps of 5 to 230 s; the agent's scripted loop (86 tests) runs in 0.03 s in total. The agent loop is instead a beneficiary of this phase: its steps share a long beginning (the append-only transcript), which the prefix store keeps and restores.

E. Summary

The engine is within 60–95 % of the machine's bandwidth, so more parallelism was not the lever. Three places repeated or waited on work and are fixed: the fourth decode thread (now three), the one-prefix cache (now a prefix store, 5.8 and 14 s down to 0.1 and 0.2 s for a switch of request kind) and the continuation's re-prefill of its own reasoning (now exact). Speculative sampling is named as the next engine item.

XIX. Phase 18: Several Users in One Pass over the Weights

Phase 18 (3 November 2026) turns Smiley Core, the default engine since Phase 15, from a one-stream engine into one that serves several users at once. The decision record is `docs/decisions/012-multiuser-engine.md` and the research note `docs/research/smiley_multiuser.md`. The measurements are from the shared reference machine, each engine alone, two rounds, long answers.

A. Why it pays on four cores

A decode step reads every weight once, so serving n users one after another reads the model n times where one pass would read it once. On this machine decode is bound by memory (19.5 GB/s measured) and not by arithmetic (about 460 GFLOP/s), and Q4_K needs 0.56 bytes per weight, so the roofline ridge sits near seven rows: up to about seven users, another row costs almost no extra time. The design follows vLLM's serving core [116] in what applies at this size: a scheduler that admits requests into slots and mixes work per step, chunked prefill so a long prompt does not stall the others, and a prefix cache shared by every slot.

B. What was built

The forward pass now takes a position and a cache *per row*, so rows belonging to different users share one read of the weights while each attends only to its own history. Attention runs one row at a time per user, so a user's logits are bit-identical to a solo run's; nothing is approximated to gain the sharing. A slot scheduler runs either one chunk of one prompt or one token for every ready user in a step, never both. A prefix store shared by all slots lets a finished request's cache serve the next user who begins the same way (the system prompt every request of a deployment sends, the transcript a code agent extends). The server owns one engine thread instead of a session behind a mutex.

C. Measured

TABLE XXXI
Decode throughput (tokens per second, all users together), each engine alone on the machine.

Users	1	2	4
Smiley Core	14.2	20.2	24.9
llama.cpp (slots)	14.4	14.4	32.4

At one user the engines are level; at two Smiley Core is $1.40\times$ ahead; at four llama.cpp is ahead ($1.30\times$). The engine primitive alone gives $1.56\times$, $2.57\times$ and $2.27\times$ at 2, 4 and 8 rows, the knee the roofline predicts. **Every user's text was identical to what they would have been given alone, at every concurrency** (4 of 4 at four users); llama.cpp's was not (0 of 4). Two defects were caught by the tests during the work: a slot whose restored prefix left nothing to prefill fed the wrong first token, and a finished cache was stored under the wrong token sequence.

D. What is not claimed

The gateway still sends one request at a time: it reads `max_in_flight` as the width of an n-choice request as well, and this server refuses $n > 1$, so the concurrency is available to the engine and not yet used by the gateway. Parity with vLLM is not claimed; vLLM does not run on this machine.

XX. Phase 19: The Engine Update and LittleBit

Phase 19 (5 November 2026) replaces the vendored Smiley Core (revision 7cb6028) by the owner's new one (06c2b88), re-applies Nafs-Aql's additions on it, integrates LittleBit [184], and re-validates the system. The decision record is `docs/decisions/013-smiley-core-littlebit.md` and the research note `docs/research/smiley_core_update.md`.

A. What the new engine brings

Grouped online-softmax attention, an int8 key–value cache rotated by a Hadamard transform, vectorised math, and sampling whose randomness is keyed by position, so a drafted token is accepted exactly when sequential sampling would have emitted it: speculation becomes lossless at any temperature, not only under argmax. LittleBit approximates a weight matrix as a product of sign matrices and small scale vectors, $\hat{W} = \mathrm{diag}(h)\, U_{\pm}\, \mathrm{diag}(\ell)\, V_{\pm}^{\top}\, \mathrm{diag}(g)$, summed over stages, stored beside the model as a sidecar file.

B. The merge

Nafs-Aql's vendored copy differs from upstream by a batching engine, a prefix store, a continuation prefix, log-probabilities and two sampler cuts, so the update was a three-way merge (base 7cb6028) after normalising both sides with the same formatter, which reduced 44 textual conflicts to 14. The additions were then re-applied on the new engine: a cache snapshot that records whether the cache is rotated; the per-row forward pass and attention without touching upstream's attention kernel (one row per user, so the bit-identity property of Phase 18 holds); top-k and min-p on the position-keyed sampler, in llama.cpp's order, with upstream's fast path when both are off. All 370 tests of the workspace pass in release mode, including the batching bit-identity tests, and formatting and the linter are clean.

C. Two defects the integration exposed

A crash. Served through the batching engine, a LittleBit model crashed on its first request: the single-session path sizes the sidecar's scratch buffers and the engine's constructor did not, so the first sidecar matmul wrote through an empty buffer. Fixed, and visible only because the engine was exercised with the new format.

A fourfold slowdown with no kernel change. After the merge Qwen3-1.7B decoded at 4.4–5.0 tokens per second, against 17–20 for the previous commit and 16–18 for upstream's own build, interleaved on the same machine. The dot kernels were byte-identical after formatting, so a code change could not be the cause directly. Bisecting by swapping files between a scratch copy of upstream and the crate showed that merely adding the batching engine (never called by the benchmark) was enough; a kernel micro-benchmark isolated it to the small-batch dot kernels (Q4_K, one activation, one thread: 679 ns per row against 96); disassembly showed a call to `core::array::try_from_fn` *inside* the kernel, once per super-block, in the slow build only. `std::array::from_fn` in two helpers marked `inline(always)` is inlined or not depending on how much unrelated code the crate contains. Spelling the eight-element arrays out restores 96 ns per row and 15–20 tokens per second. The lesson is general: a kernel whose speed depends on an optimiser decision about code elsewhere has to be guarded by a benchmark, and `kbench` is now part of the crate for that.

TABLE XXXII
DECODE SPEED OF QWEN3-1.7B Q4_K_M (TOKENS PER SECOND, MEDIAN OF FIVE INTERLEAVED ROUNDS OF 96 TOKENS, THREE THREADS, ONE SLOT, SHARED HOST).

Engine	tok/s
Previous (7cb6028 + Phase 18)	18.7
New (06c2b88), after the fix	19.2
New, before the fix	4.4–5.0
New with a LittleBit sidecar	20.1

D. LittleBit: integrated, tested, not useful yet

Qwen3-1.7B was converted at 1.0 bit per weight with the engine's post-training (Dual-SVID) converter: 196 projections, 172.7 MB of factors for 845.7 MB of weights ($4.9\times$), 0.980 bits per weight, 13 minutes on four cores. The mean relative reconstruction error of the weights is 0.752 (Q4_K itself is about 0.1). On twelve exact-answer prompts, greedy, interleaved on the same engine, **the dense model answered 9 correctly and the LittleBit model 0**; its text is incoherent. Decode was only 5 % faster (20.1 against 19.2 tok/s), because at this size the matmul is not the whole token. Upstream reports the same on a smaller model and attributes the paper's quality to distillation-based quantisation-aware training, which needs a GPU. The integration is therefore opt-in (`-littlebit`, `LITTLEBIT=1`); a missing or corrupt sidecar refuses to start, so a stale file beside a model cannot switch the weights silently, and the weights in use (dense or LittleBit) are reported in every response, in the backend health check and on the Status page. It is ready for trained factors; it is not a result.

E. Is anything else different

The old and the new engine give the same answers on the exact-answer prompts (9 of 12 on both, the same three misses). One of those misses, a time-of-day sum (3:40 pm plus 2 h 35 min answered 5:45 pm, the correct answer being 6:15 pm), was first seen in the web interface right after the merge and looked like a regression; the comparison shows it is the 1.7B model's own error, which the gate's confidence signal does not catch because both quick samples repeat it. Re-validation on the final code: contract tests, 26 of 26 security probes (0 open high-severity findings), 59 of 59 Rooms checks, 65 of 65 end-to-end checks, 53 of 53 harness checks and 9 of 9 acceptance tests pass.

XXI. Limitations and Ethical Considerations

Indicative results. Section IX reports measurements on one shared four-core machine with tens of items per comparison, far below the sample sizes of Section VIII; the quality claims remain hypotheses until the release-scale runs. The formal results hold under their stated assumptions, several of which (conditional independence of verifiers, a Beta prior on acceptance, homogeneous refinement dynamics, concave budget returns) are modelling choices to be checked.

Harness integration. The Lean verifier runs against core Lean 4.35.0; statements that import Mathlib need a Mathlib build and `lean_path`, which were not installed. On Linux, Lean runs with user and network namespaces only (A-070). Patch repair is off until the failure rate f of Eq. (5) is measured, and the code agent's live runs are a handful of tasks on a shared host. The Phase 11 A/B (Section XII) has ten System 1 tasks and three System 2 tasks per arm on a shared host: its verified counts are indicative, not significant, and its tasks are small single-file functions. Grammar constraints fix the syntax of an action, not its sense (a patch that applies can still break the file), and they need llama-server; on vLLM the harness relies on the stream cut alone.

Lean proofs need a stronger prover. The verifier and the repair loop are validated end to end with the real Lean binary, but Qwen3-4B-Thinking (4-bit, CPU) abstained on $n + 0 = n$ within its budget (Section XIII); Lean tasks therefore return *abstained* unless the model writes valid Lean 4. The verified proofs reported are the harness's own test cases, not model output.

Parallelism results are small-sample. The rescue (40 live pairs, 3 avoided escalations, 60 offline items) and the hedging comparison (8 items; the sequential loop itself gave 4/8 and 3/8) are indicative; the batch-scaling numbers are medians of two or three repetitions on one machine. Hedging on System 2 is off because of what was measured here; a host with a cheaper batch at depth could change that (Section XIV).

Shared weights. Fast and slow paths from one model family make correlated errors; the collision term is measured rather than assumed small, and the judge shares the same bias, which is why exact verifiers carry the safety case.

CPU constraint. On the reference machine the slow path takes minutes; the design is honest about this and makes interactive deliberation a GPU-profile feature. Published AVX2 throughput data are sparse; our own measurements cover two models and one machine.

Verifiers are partial. Only code, mathematics and structured output have exact verifiers; everything else is at best a consensus or judge assurance, which Theorem 6 shows has a floor. Test-based code verification can be gamed by overfitting to visible tests; hidden tests and mutation testing are required.

Security. The system executes model-written code in a sandbox and consumes untrusted text; prompt injection through retrieved memory or tool output is a first-class threat [121], [185]. The specification isolates the sandbox, strips instructions from retrieved content, and never lets observed content change controller parameters.

Religious content. The classical vocabulary is used as analogy and the components have plain names in code. We do not claim that the software has a soul, consciousness or moral standing; we do not model the *ruh*; and we cite scripture and hadith by number from named translations. The analogy section should be reviewed by scholars of the tradition, and users of the chat system who ask religious questions receive answers with the same assurance labelling as any other non-verifiable domain, with no claim of authority.

Data and privacy. Conversation traces feed the self-model and the distillation set; the specification requires consent, retention limits, redaction of personal data before training, and exclusion of any conversation from training on request.

Hallucination and faithfulness. Separating reasoning content from the answer does not make the reasoning faithful [119], [186]; we treat the trace as evidence for debugging and distillation, not as an explanation to the user.

XXII. Conclusion

Nafs-Aql is a buildable dual-process system: a small Qwen model answers fast, a calibrated gate decides when deliberation pays, a second model (or the same model in thinking mode) deliberates under a budget, exact verifiers decide what counts as known, and an audited ledger keeps the system honest about itself. The classical account of the self supplied the vocabulary and, more usefully, three disciplines that translate directly into engineering: the intellect arbitrates on cost and risk, the self is corrected under an external standard, and dispositions change slowly through verified practice. The mathematics turned those disciplines into a threshold gate, a verifier-bounded precision, a stable loop and a sized set of release gates. The feasibility study showed what October 2026 permits on a four-core machine: Qwen3 and Qwen3.5 below 9B parameters, llama-server or vLLM-CPU behind one API, Unsloth on a rented GPU, and a Rust gateway. The specification and roadmap that accompany this report give the team requirements with identifiers, machine-readable interfaces, and a phased plan with measurable gates. The first three actions are to run the Phase-0 baseline at the required sample size, to measure the verifiers' false-accept rates by mutation testing, and to benchmark the Qwen3.5 hybrid models against dense Qwen3 on the target CPU before choosing the upgrade path. The second and third are done and the first runs only at a scaled-down size on one CPU. With

the system built, the cascade runs end to end on a shared four-core CPU, the verifiers are measured, Profile C was evaluated and not adopted, and latency is attributed by stage, which points the next work at decode speed (EAGLE-3 and repacked weights) and at release-scale evaluation on a faster host.

Phase 11 adds one lesson about agent harnesses for small models. What made Codex's loop work here was not its tools but its discipline: an action language the server can enforce, a history that is never rewritten, and a hard budget. With the action constrained and the edit format chosen by file length, the 1.7B and 4B models stopped losing steps to malformed replies; that, more than any saving in prefill, is where the speed of a verified answer came from.

REFERENCES

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[2] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[3] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv preprint (Aug. 2024) titled "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters". [Online]. Available: https://arxiv.org/abs/2408.03314

[4] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[5] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu, "Do NOT think that much for 2+3=? On the overthinking of o1-like LLMs," *arXiv preprint arXiv:2412.21187*, 2024. [Online]. Available: https://arxiv.org/abs/2412.21187

[6] J. Zhang, N. Lin, L. Hou, L. Feng, and J. Li, "AdaptThink: Reasoning models can learn when to think," *arXiv preprint arXiv:2505.13417*, 2025. [Online]. Available: https://arxiv.org/abs/2505.13417

[7] D. Kahneman, *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux, 2011.

[8] J. S. B. T. Evans and K. E. Stanovich, "Dual-process theories of higher cognition: Advancing the debate," *Perspectives on Psychological Science*, vol. 8, no. 3, pp. 223–241, 2013.

[9] S. A. Sloman, "The empirical case for two systems of reasoning," *Psychological Bulletin*, vol. 119, no. 1, pp. 3–22, 1996.

[10] Abdel Haleem, M. A. S. (trans.), *The Qur'an*, ser. Oxford World's Classics. Oxford: Oxford University Press, 2008, paperback reissue of the Oxford World's Classics translation. Accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780199535958

[11] A. Ḥ. al Ghazālī, *Al-Ghazali: The Marvels of the Heart. Science of the Spirit. Book 21 of the Revival of the Religious Sciences (Iḥyā' 'Ulūm al-Dīn)*. Louisville, KY: Fons Vitae, 2010, translated by Walter James Skellie; introduction by T. J. Winter. Accessed 2026-10-18. [Online]. Available: https://www.fonsvitae.com/product/al-ghazali-the-marvels-of-the-heart/

[12] G. Picken, *Spiritual Purification in Islam: The Life and Works of al-Muhasibi*, ser. Routledge Sufi Series. London and New York: Routledge, 2011, accessed 2026-10-18. [Online]. Available: https://www.routledge.com/Spiritual-Purification-in-Islam-The-Life-and-Works-of-al-Muhasibi/Picken/p/book/9780415548229

[13] A. l.-Q. A. a.-K. al Qushayrī, *Al-Qushayri's Epistle on Sufism: Al-Risāla al-qushayriyya fī 'ilm al-taṣawwuf*, ser. Great Books of Islamic Civilization. Reading: Garnet Publishing, 2007, translated by Alexander D. Knysh. Accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9781859641866

[14] Dirac R&D Team, "Nafs-aql v0.1: Mathematical validation and technical report of a dual-process agent on Qwen (internal draft slide deck)," Oct. 2026, internal document, `nafs_aql_report.tex`, 18 October 2026.

[15] E. E. Calverley and I. R. Netton, "Nafs," in *Encyclopaedia of Islam, Second Edition*, P. Bearman, T. Bianquis, C. E. Bosworth, E. van Donzel, and W. P. Heinrichs, Eds. Leiden: Brill, 2012, first published online 2012; print edition 1960–2007. Title, container and DOI confirmed on the Crossref DOI record; Brill's pages returned 403 to direct access, so the author line was read from Brill's own citation string as surfaced by web search – confirm with library access before final submission. Accessed 2026-10-18. [Online]. Available: https://doi.org/10.1163/1573-3912_islam_COM_0833

[16] A. Schimmel, *Mystical Dimensions of Islam*. Chapel Hill, NC: University of North Carolina Press, 1975, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780807812716

[17] A. Rothman and A. Coyle, "Toward a framework for islamic psychology and psychotherapy: An islamic model of the soul," *Journal of Religion and Health*, vol. 57, no. 5, pp. 1731–1744, 2018, accessed 2026-10-18. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC6132620/

[18] A. Ḥ. al Ghazālī, *Al-Ghazālī on Disciplining the Soul (Kitāb Riyāḍat al-nafs) and on Breaking the Two Desires (Kitāb Kasr al-shahwatayn): Books XXII and XXIII of the Revival of the Religious Sciences (Iḥyā' 'Ulūm al-Dīn)*. Cambridge: Islamic Texts Society, 1995, translated with an introduction and notes by T. J. Winter. First edition 1995 (confirmed via the 1997 AJISS review essay, DOI 10.35632/ajis.v14i4.2218); ITS catalogue lists the 2016 reprint. Accessed 2026-10-18. [Online]. Available: https://its.org.uk/

[19] F. Griffel, *Al-Ghazālī's Philosophical Theology*. Oxford and New York: Oxford University Press, 2009, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780195331622

[20] ——, "al-ghazali," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds. Metaphysics Research Lab, Stanford University, 2026, first published 14 August 2007; substantive revision 15 January 2026. Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/al-ghazali/

[21] T. Kukkonen, "Receptive to reality: Al-ghazālī on the structure of the soul," *The Muslim World*, vol. 102, no. 3–4, pp. 541–561, 2012, accessed 2026-10-18. [Online]. Available: https://doi.org/10.1111/j.1478-1913.2012.01412.x

[22] ——, "Al-ghazālī on error," in *Islam and Rationality: The Impact of al-Ghazālī. Papers Collected on His 900th Anniversary*, F. Griffel, Ed. Leiden: Brill, 2016, vol. 2, pp. 3–31, editor and volume number from the Crossref container record; verify against the printed volume. Accessed 2026-10-18. [Online]. Available: https://doi.org/10.1163/9789004307490_002

[23] Ibn Sīnā (Avicenna), *Avicenna's Psychology: An English Translation of Kitāb al-Najāt, Book II, Chapter VI, with Historico-Philosophical Notes and Textual Improvements on the Cairo Edition*. London: Oxford University Press, 1952, translation by F. Rahman. Accessed 2026-10-18. [Online]. Available: https://catalogue.bnf.fr/ark:/12148/cb31745014f

[24] ——, *Avicenna's De Anima (Arabic Text): Being the Psychological Part of Kitāb al-Shifā'*, F. Rahman, Ed. London and New York: Oxford University Press, 1959, accessed 2026-10-18. [Online]. Available: https://wellcomecollection.org/works/g9jzxtmj

[25] J. McGinnis, *Avicenna*, ser. Great Medieval Thinkers. Oxford and New York: Oxford University Press, 2010, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780195331479

[26] D. Gutas, "Ibn sina [avicenna]," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds. Metaphysics Research Lab, Stanford University, 2025, first published 15 September 2016; substantive revision 31 October 2025. Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/ibn-sina/

[27] A. Ivry, "Arabic and islamic psychology and philosophy of mind," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta, Ed. Metaphysics Research Lab, Stanford University, 2012, summer 2012 Edition; first published 18 April 2008, substantive revision 29 May 2012. Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/arabic-islamic-mind/

[28] D. L. Black, "Estimation (wahm) in avicenna: The logical and psychological dimensions," *Dialogue: Canadian Philosophical Review*,

vol. 32, no. 2, pp. 219–258, 1993, accessed 2026-10-18. [Online]. Available: https://doi.org/10.1017/S0012217300014414

[29] D. Gutas, *Avicenna and the Aristotelian Tradition: Introduction to Reading Avicenna's Philosophical Works*, 2nd ed., ser. Islamic Philosophy, Theology and Science. Texts and Studies. Leiden: Brill, 2014, vol. 89, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9789004255807

[30] S. Rizvi, "Mulla sadra," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta, Ed. Metaphysics Research Lab, Stanford University, 2019, first published 9 June 2009; substantive revision 5 February 2019 (Spring 2021 archive edition). Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/mulla-sadra/

[31] M. E. Marmura, "Avicenna's "flying man" in context," *The Monist*, vol. 69, no. 3, pp. 383–395, 1986, accessed 2026-10-18. [Online]. Available: https://doi.org/10.5840/monist198669328

[32] J. Kaukua, *Self-Awareness in Islamic Philosophy: Avicenna and Beyond*. Cambridge: Cambridge University Press, 2015, accessed 2026-10-18. [Online]. Available: https://doi.org/10.1017/CBO9781316105238

[33] W. C. Chittick, *The Sufi Path of Knowledge: Ibn al-'Arabi's Metaphysics of Imagination*. Albany, NY: State University of New York Press, 1989, accessed 2026-10-18. [Online]. Available: https://sunypress.edu/Books/T/The-Sufi-Path-of-Knowledge2

[34] ——, "Ibn 'arabī," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds. Metaphysics Research Lab, Stanford University, 2025, first published 5 August 2008; substantive revision 5 December 2025. Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/ibn-arabi/

[35] R. Marcotte, "Suhrawardi," in *The Stanford Encyclopedia of Philosophy*, E. N. Zalta and U. Nodelman, Eds. Metaphysics Research Lab, Stanford University, 2019, first published 26 December 2007; substantive revision 29 March 2019 (Spring 2025 archive edition). Accessed 2026-10-18. [Online]. Available: https://plato.stanford.edu/entries/suhrawardi/

[36] M. Ha'iri Yazdi, *The Principles of Epistemology in Islamic Philosophy: Knowledge by Presence*. Albany, NY: State University of New York Press, 1992, foreword by Seyyed Hossein Nasr. Accessed 2026-10-18. [Online]. Available: https://sunypress.edu/Books/T/The-Principles-of-Epistemology-in-Islamic-Philosophy

[37] S. H. Rizvi, *Mulla Sadra and Metaphysics: Modulation of Being*, ser. Culture and Civilization in the Middle East. London and New York: Routledge, 2009, accessed 2026-10-18. [Online]. Available: https://www.routledge.com/Mulla-Sadra-and-Metaphysics-Modulation-of-Being/Rizvi/p/book/9780415490733

[38] I. Kalin, *Knowledge in Later Islamic Philosophy: Mullā Ṣadrā on Existence, Intellect, and Intuition*. Oxford and New York: Oxford University Press, 2010, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780199735242

[39] F. Rahman, *The Philosophy of Mullā Ṣadrā (Ṣadr al-Dīn al-Shīrāzī)*. Albany, NY: State University of New York Press, 1975, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9780873953009

[40] T. J. Gianotti, *Al-Ghazālī's Unspeakable Doctrine of the Soul: Unveiling the Esoteric Psychology and Eschatology of the Iḥyā'*, ser. Brill's Studies in Intellectual History. Leiden: Brill, 2001, vol. 104, accessed 2026-10-18. [Online]. Available: https://books.google.co.id/books?vid=ISBN9004120831

[41] A. Treiger, *Inspired Knowledge in Islamic Thought: Al-Ghazālī's Theory of Mystical Cognition and Its Avicennian Foundation*. London and New York: Routledge, 2011, crossref gives 2011 (e-book); Griffel's SEP bibliography cites it as 2012. Accessed 2026-10-18. [Online]. Available: https://doi.org/10.4324/9780203806487

[42] K. E. Stanovich and R. F. West, "Individual differences in reasoning: Implications for the rationality debate?" *Behavioral and Brain Sciences*, vol. 23, no. 5, pp. 645–665, 2000.

[43] A. Shenhav, M. M. Botvinick, and J. D. Cohen, "The expected value of control: An integrative theory of anterior cingulate cortex function," *Neuron*, vol. 79, no. 2, pp. 217–240, 2013.

[44] F. Lieder and T. L. Griffiths, "Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources," *Behavioral and Brain Sciences*, vol. 43, p. e1, 2020.

[45] S. Russell and E. Wefald, "Principles of metareasoning," *Artificial Intelligence*, vol. 49, no. 1-3, pp. 361–395, 1991.

[46] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, "Large language monkeys: Scaling inference compute with repeated sampling," *arXiv preprint arXiv:2407.21787*, 2024. [Online]. Available: https://arxiv.org/abs/2407.21787

[47] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021. [Online]. Available: https://arxiv.org/abs/2110.14168

[48] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2305.20050

[49] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025. [Online]. Available: https://doi.org/10.1038/s41586-025-09422-z

[50] OpenAI, A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky *et al.*, "OpenAI o1 system card," *arXiv preprint arXiv:2412.16720*, 2024. [Online]. Available: https://arxiv.org/abs/2412.16720

[51] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto, "s1: Simple test-time scaling," *arXiv preprint arXiv:2501.19393*, 2025. [Online]. Available: https://arxiv.org/abs/2501.19393

[52] A. Yang, A. Li, B. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[53] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[54] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, arXiv version (2303.11366) additionally lists Edward Berman as an author. [Online]. Available: https://arxiv.org/abs/2303.11366

[55] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[56] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1417–1440, 2024. [Online]. Available: https://arxiv.org/abs/2406.01297

[57] G. Fang, X. Ma, and X. Wang, "Thinkless: LLM learns when to think," *arXiv preprint arXiv:2505.13379*, 2025. [Online]. Available: https://arxiv.org/abs/2505.13379

[58] Y. He, X. Ding, B. Cai, Y. Zhang, K. Xiong, Z. Sun, B. Qin, and T. Liu, "Self-route: Automatic mode switching via capability estimation for efficient reasoning," *arXiv preprint arXiv:2505.20664*, 2025. [Online]. Available: https://arxiv.org/abs/2505.20664

[59] B. Hou, Y. Zhang, J. Ji, Y. Liu, K. Qian, J. Andreas, and S. Chang, "ThinkPrune: Pruning long chain-of-thought of LLMs via reinforcement learning," *arXiv preprint arXiv:2504.01296*, 2025. [Online]. Available: https://arxiv.org/abs/2504.01296

[60] C. Yang, Q. Si, Y. Duan, Z. Zhu, C. Zhu, Q. Li, M. Chen, Z. Lin, and W. Wang, "Dynamic early exit in reasoning models," *arXiv preprint arXiv:2504.15895*, 2025. [Online]. Available: https://arxiv.org/abs/2504.15895

[61] D. Arora and A. Zanette, "Training language models to reason efficiently," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.04463

[62] J. Lin, X. Zeng, J. Zhu, S. Wang, J. Shun, J. Wu, and D. Zhou, "Plan and budget: Effective and efficient test-time scaling on reasoning large language models," in *International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2505.16122

[63] Z. Zhai, B. Li, B. Xiao, M. Li, and X. Wang, "Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization," *arXiv preprint arXiv:2604.14853*, 2026. [Online]. Available: https://arxiv.org/abs/2604.14853

[64] C. Fan, Y. Cheng, M. Li, Y. Liang, T. Zhou, and S. Feizi, "Thinking hard, not smart: Reasoning models fail to ration test-time compute across questions," *arXiv preprint arXiv:2608.07968*, 2026. [Online]. Available: https://arxiv.org/abs/2608.07968

[65] Z. Zhou, L. Zhang, J. Wang, M. Yuan, S. Liu, T. Li, Y. Hu, and Y. Zhao, "Nice fold or hero call: Learning budget-efficient thinking

under policy-dependent solvability," *arXiv preprint arXiv:2605.11625*, 2026. [Online]. Available: https://arxiv.org/abs/2605.11625

[66] Z. Xu, H. Xie, Z. Miao, W. Gong, C. Qian, and L. Li, "Stable adaptive thinking via advantage shaping and length-aware gradient regulation," *arXiv preprint arXiv:2602.22556*, 2026. [Online]. Available: https://arxiv.org/abs/2602.22556

[67] H. Guo, T. Wei, D. Wang, Y. Du, Z. Wang, J. Zhang, and Z. Sun, "Think when needed: Model-aware reasoning routing for LLM-based ranking," in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2026, venue per arXiv comment "Accepted by SIGIR2026". [Online]. Available: https://arxiv.org/abs/2601.18146

[68] D. Su, S. Sukhbaatar, M. Rabbat, Y. Tian, and Q. Zheng, "Dualformer: Controllable fast and slow thinking by learning with randomized reasoning traces," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.09918

[69] S. Saha, A. Prasad, J. C.-Y. Chen, P. Hase, E. Stengel-Eskin, and M. Bansal, "System-1.x: Learning to balance fast and slow planning with language models," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2407.14414

[70] G. Booch, F. Fabiano, L. Horesh, K. Kate, J. Lenchner, N. Linck, A. Loreggia, K. Murugesan, N. Mattei, F. Rossi, and B. Srivastava, "Thinking fast and slow in AI," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 17, 2021, pp. 15 042–15 046. [Online]. Available: https://arxiv.org/abs/2010.06002

[71] Y. Bengio, "From System 1 deep learning to System 2 deep learning," Invited talk (Posner Lecture), Conference on Neural Information Processing Systems (NeurIPS), 2019. [Online]. Available: https://neurips.cc/virtual/2019/invited-talk/15488

[72] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, "Hybrid LLM: Cost-efficient and quality-aware query routing," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2404.14618

[73] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs from preference data," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv version titled "RouteLLM: Learning to Route LLMs with Preference Data". [Online]. Available: https://arxiv.org/abs/2406.18665

[74] M. Yue, J. Zhao, M. Zhang, L. Du, and Z. Yao, "Large language model cascades with mixture of thoughts representations for cost-efficient reasoning," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.03094

[75] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[76] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[77] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[78] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018. [Online]. Available: https://arxiv.org/abs/1711.06664

[79] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. [Online]. Available: https://arxiv.org/abs/2006.01862

[80] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. [Online]. Available: https://arxiv.org/abs/1706.04599

[81] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez *et al.*, "Language models (mostly) know what they know," *arXiv preprint arXiv:2207.05221*, 2022. [Online]. Available: https://arxiv.org/abs/2207.05221

[82] K. Tian, E. Mitchell, A. Zhou, A. Sharma, R. Rafailov, H. Yao, C. Finn, and C. D. Manning, "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14975

[83] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023, arXiv version (2107.07511, 2021) titled "A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification". [Online]. Available: https://arxiv.org/abs/2107.07511

[84] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. [Online]. Available: https://arxiv.org/abs/2211.17192

[85] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023. [Online]. Available: https://arxiv.org/abs/2302.01318

[86] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 5209–5235. [Online]. Available: https://arxiv.org/abs/2401.10774

[87] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE: Speculative sampling requires rethinking feature uncertainty," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 28 935–28 948. [Online]. Available: https://arxiv.org/abs/2401.15077

[88] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve, "Better & faster large language models via multi-token prediction," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 15 706–15 734. [Online]. Available: https://arxiv.org/abs/2404.19737

[89] A. Graves, "Adaptive computation time for recurrent neural networks," *arXiv preprint arXiv:1603.08983*, 2016. [Online]. Available: https://arxiv.org/abs/1603.08983

[90] D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. C. Humphreys, and A. Santoro, "Mixture-of-depths: Dynamically allocating compute in transformer-based language models," *arXiv preprint arXiv:2404.02258*, 2024. [Online]. Available: https://arxiv.org/abs/2404.02258

[91] J. Gu, J. Bradbury, C. Xiong, V. O. K. Li, and R. Socher, "Non-autoregressive neural machine translation," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1711.02281

[92] M. Ghazvininejad, O. Levy, Y. Liu, and L. Zettlemoyer, "Mask-predict: Parallel decoding of conditional masked language models," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. [Online]. Available: https://arxiv.org/abs/1904.09324

[93] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, "Structured denoising diffusion models in discrete state-spaces," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021. [Online]. Available: https://arxiv.org/abs/2107.03006

[94] A. Lou, C. Meng, and S. Ermon, "Discrete diffusion modeling by estimating the ratios of the data distribution," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.16834

[95] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov, "Simple and effective masked diffusion language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.07524

[96] S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li, "Large language diffusion models," *arXiv preprint arXiv:2502.09992*, 2025. [Online]. Available: https://arxiv.org/abs/2502.09992

[97] J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, "Dream 7b: Diffusion large language models," *arXiv preprint arXiv:2508.15487*, 2025. [Online]. Available: https://arxiv.org/abs/2508.15487

[98] S. Gong, R. Zhang, H. Zheng, J. Gu, N. Jaitly, L. Kong, and Y. Zhang, "DiffuCoder: Understanding and improving masked diffusion models for code generation," *arXiv preprint arXiv:2506.20639*, 2025. [Online]. Available: https://arxiv.org/abs/2506.20639

[99] M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov, "Block diffusion: Interpolating between autoregressive and diffusion language models," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2503.09573

[100] S. Cheng, Y. Bian, D. Liu, Y. Jiang, Y. Liu, L. Zhang, Q. Yao, Z. Tian, W. Wang, Q. Guo, K. Chen, B. Qi, and B. Zhou, "SDAR: A synergistic diffusion-autoregression paradigm for scalable sequence generation," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 22 058–22 075, arXiv version (2510.06303, Oct. 2025) lists 11 authors. [Online]. Available: https://arxiv.org/abs/2510.06303

[101] Y. Wang, L. Yang, B. Li, Y. Tian, K. Shen, and M. Wang, "Revolutionizing reinforcement learning framework for diffusion large language models," in *International Conference on Learning Representations (ICLR)*, 2026, introduces TraceRL and the TraDo model family. [Online]. Available: https://arxiv.org/abs/2509.06949

[102] C. Wu, H. Zhang, S. Xue, S. Diao, Y. Fu, Z. Liu, P. Molchanov, P. Luo, S. Han, and E. Xie, "Fast-dLLM v2: Efficient block-diffusion LLM," in *International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: https://arxiv.org/abs/2509.26328

[103] C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie, "Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding," *arXiv preprint arXiv:2505.22618*, 2025. [Online]. Available: https://arxiv.org/abs/2505.22618

[104] Inception Labs, S. Khanna, S. Kharbanda, S. Li, H. Varma, E. Wang, S. Birnbaum, Z. Luo, Y. Miraoui, A. Palrecha, S. Ermon, A. Grover, and V. Kuleshov, "Mercury: Ultra-fast language models based on diffusion," *arXiv preprint arXiv:2506.17298*, 2025. [Online]. Available: https://arxiv.org/abs/2506.17298

[105] T. Li, M. Chen, B. Guo, and Z. Shen, "A survey on diffusion language models," *arXiv preprint arXiv:2508.10875*, 2025, v3 revised June 2026. [Online]. Available: https://arxiv.org/abs/2508.10875

[106] A. Santilli, S. Severino, E. Postolache, V. Maiorca, M. Mancusi, R. Marin, and E. Rodolà, "Accelerating transformer inference for translation via parallel decoding," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.10427

[107] S. Kou, L. Hu, Z. He, Z. Deng, and H. Zhang, "CLLMs: Consistency large language models," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. [Online]. Available: https://arxiv.org/abs/2403.00835

[108] Qwen Team, "Qwen3.5: Towards native multimodal agents," https://qwen.ai/blog?id=qwen3.5, Feb. 2026, no standalone technical report as of 2026-10-18; model cards cite this post.

[109] ——, "Qwen3.6-27b: Flagship-level coding in a 27b dense model," https://qwen.ai/blog?id=qwen3.6-27b, Apr. 2026.

[110] ——, "Qwen3.6-35b-a3b: Agentic coding power, now open to all," https://qwen.ai/blog?id=qwen3.6-35b-a3b, Apr. 2026.

[111] ——, "Qwen3.8-max: A new bar for coding and cowork," https://qwen.ai/blog?id=qwen3.8, Aug. 2026, cited by the Qwen3.8-27B model card; no technical report.

[112] Y. S. Chow, H. Robbins, and D. Siegmund, *Great Expectations: The Theory of Optimal Stopping*. Boston, MA, USA: Houghton Mifflin, 1971, reissued by Dover, 1991, as *The Theory of Optimal Stopping*. Verified via HathiTrust record 000469250, accessed 2026-10-18.

[113] W. Hoeffding, "Probability inequalities for sums of bounded random variables," *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 13–30, 1963.

[114] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[115] P. Massart, "The tight constant in the Dvoretzky-Kiefer-Wolfowitz inequality," *The Annals of Probability*, vol. 18, no. 3, pp. 1269–1283, 1990.

[116] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. [Online]. Available: https://arxiv.org/abs/2309.06180

[117] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[118] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[119] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.04388

[120] T. Lanham, A. Chen, A. Radhakrishnan, B. Steiner, C. Denison, D. Hernandez *et al.*, "Measuring faithfulness in chain-of-thought reasoning," *arXiv preprint arXiv:2307.13702*, 2023. [Online]. Available: https://arxiv.org/abs/2307.13702

[121] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90. [Online]. Available: https://arxiv.org/abs/2302.12173

[122] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023. [Online]. Available: https://arxiv.org/abs/2307.15043

[123] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does LLM safety training fail?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2307.02483

[124] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, "Measuring mathematical problem solving with the MATH dataset," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021. [Online]. Available: https://arxiv.org/abs/2103.03874

[125] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[126] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021. [Online]. Available: https://arxiv.org/abs/2108.07732

[127] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://arxiv.org/abs/2009.03300

[128] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, "GPQA: A graduate-level Google-proof Q&A benchmark," in *First Conference on Language Modeling (COLM)*, 2024. [Online]. Available: https://arxiv.org/abs/2311.12022

[129] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. [Online]. Available: https://arxiv.org/abs/2109.07958

[130] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus, "Measuring short-form factuality in large language models," *arXiv preprint arXiv:2411.04368*, 2024. [Online]. Available: https://arxiv.org/abs/2411.04368

[131] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, and J. Wei, "Challenging BIG-Bench tasks and whether chain-of-thought can solve them," *arXiv preprint arXiv:2210.09261*, 2022. [Online]. Available: https://arxiv.org/abs/2210.09261

[132] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, "LiveCodeBench: Holistic and contamination free evaluation of large language models for code," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2403.07974

[133] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[134] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1-2, pp. 100–115, 1954.

[135] S. Hameroff and R. Penrose, "Consciousness in the universe: A review of the 'Orch OR' theory," *Physics of Life Reviews*, vol. 11, no. 1, pp. 39–78, 2014. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/24070914/

[136] M. Tegmark, "Importance of quantum decoherence in brain processes," *Physical Review E*, vol. 61, no. 4, pp. 4194–4206, 2000. [Online]. Available: https://arxiv.org/abs/quant-ph/9907009

[137] S. Hagan, S. R. Hameroff, and J. A. Tuszyński, "Quantum computation in brain microtubules: Decoherence and biological feasibility," *Physical Review E*, vol. 65, p. 061901, 2002.

[138] N. S. Babcock, G. Montes-Cabrera, K. E. Oberhofer, M. Chergui, G. L. Celardo, and P. Kurian, "Ultraviolet superradiance from mega-networks of tryptophan in biological architectures," *The Journal of Physical Chemistry B*, vol. 128, no. 17, pp. 4035–4046, 2024. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11075083/

[139] J. R. Busemeyer and P. D. Bruza, *Quantum Models of Cognition and Decision*. Cambridge University Press, 2012.

[140] E. M. Pothos and J. R. Busemeyer, "Can quantum probability provide a new direction for cognitive modeling?" *Behavioral and Brain Sciences*, vol. 36, no. 3, pp. 255–274, 2013.

[141] Z. Wang and J. R. Busemeyer, "A quantum question order model supported by empirical tests of an a priori and precise prediction," *Topics in Cognitive Science*, vol. 5, no. 4, pp. 689–710, 2013.

[142] Z. Wang, T. Solloway, R. M. Shiffrin, and J. R. Busemeyer, "Context effects produced by question orders reveal quantum nature of human judgments," *Proceedings of the National Academy of Sciences*, vol. 111, no. 26, pp. 9431–9436, 2014.

[143] "Auditing question-order effects in large language models with the QQ equality: Mechanism characterization and a saturation caveat," *arXiv preprint arXiv:2607.17219*, 2026. [Online]. Available: https://arxiv.org/pdf/2607.17219

[144] J. I. Gold and M. N. Shadlen, "The neural basis of decision making," *Annual Review of Neuroscience*, vol. 30, pp. 535–574, 2007.

[145] A. Wald, "Sequential tests of statistical hypotheses," *The Annals of Mathematical Statistics*, vol. 16, no. 2, pp. 117–186, 1945.

[146] A. Wald and J. Wolfowitz, "Optimum character of the sequential probability ratio test," *The Annals of Mathematical Statistics*, vol. 19, no. 3, pp. 326–339, 1948. [Online]. Available: https://www.projecteuclid.org/euclid.aoms/1177730197

[147] P. I. Frazier and A. J. Yu, "Sequential hypothesis testing under stochastic deadlines," in *Advances in Neural Information Processing Systems 20 (NIPS 2007)*, 2007, pp. 465–472. [Online]. Available: https://proceedings.neurips.cc/paper/2007/hash/9c82c7143c102b71c593d98d96093fde-Abstract.html

[148] J. Drugowitsch, R. Moreno-Bote, A. K. Churchland, M. N. Shadlen, and A. Pouget, "The cost of accumulating evidence in perceptual decision making," *Journal of Neuroscience*, vol. 32, no. 11, pp. 3612–3628, 2012.

[149] S. M. Fleming and H. C. Lau, "How to measure metacognition," *Frontiers in Human Neuroscience*, vol. 8, p. 443, 2014. [Online]. Available: https://www.frontiersin.org/articles/10.3389/fnhum.2014.00443

[150] S. Dehaene and J.-P. Changeux, "Experimental and theoretical approaches to conscious processing," *Neuron*, vol. 70, no. 2, pp. 200–227, 2011.

[151] P. Aggarwal, A. Madaan, Y. Yang, and Mausam, "Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs," in *Proceedings of EMNLP 2023*, 2023, pp. 12 375–12 396. [Online]. Available: https://aclanthology.org/2023.emnlp-main.761

[152] Y. Li, P. Yuan, S. Feng, B. Pan, X. Wang, B. Sun, H. Wang, and K. Li, "Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning," in *International Conference on Learning Representations (ICLR)*, 2024.

[153] Y. Fu, X. Wang, Y. Tian, and J. Zhao, "Deep think with confidence," *arXiv preprint arXiv:2508.15260*, 2025. [Online]. Available: https://arxiv.org/abs/2508.15260

[154] Open Math OS, "Open math harness (nucleus), release 0.41.0," https://github.com/openmathmodel/open-math-harness, revision 687f2a2, 2026, apache-2.0.

[155] OpenAI, "Codex CLI (codex-rs)," https://github.com/openai/codex, revision 970111f6, 2026, apache-2.0.

[156] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," Automated Deduction – CADE 28, LNCS 12699, pp. 625–635, 2021.

[157] E. Glukhov, M. Conti, E. Bogomolov, Y. Golubev, and A. Bezzubov, "Diff-XYZ: A benchmark for evaluating diff understanding," *arXiv preprint arXiv:2510.12487*, 2025.

[158] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[159] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[160] Y. Lin *et al.*, "Goedel-Prover-V2: Scaling formal theorem proving with scaffolded data synthesis and self-correction," *arXiv preprint arXiv:2508.03613*, 2025.

[161] R. Xin *et al.*, "BFS-Prover: Scalable best-first tree search for LLM-based automatic theorem proving," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, arXiv:2502.03438.

[162] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[163] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen, "Let me speak freely? A study on the impact of format restrictions on performance of large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, arXiv:2408.02442.

[164] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[165] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen, "XGrammar: Flexible and efficient structured generation engine for large language models," *arXiv preprint arXiv:2411.15100*, 2024.

[166] Aider, "Edit formats," https://aider.chat/docs/more/edit-formats.html, 2024, whole-file, diff and udiff formats and their per-model reliability.

[167] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[168] M. Chen, B. Hui *et al.*, "Parallel scaling law for language models," *arXiv preprint arXiv:2505.10475*, 2025.

[169] G. Rodionov, R. Garipov *et al.*, "Hogwild! inference: Parallel LLM generation via concurrent attention," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2504.06261.

[170] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme speculative decoding for emerging AI applications," *arXiv preprint arXiv:2411.04975*, 2024.

[171] J. Wei, S. Cao *et al.*, "T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge," *arXiv preprint arXiv:2407.00088*, 2024.

[172] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[173] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, "Serverlessllm: Low-latency serverless inference for large language models," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[174] S. Ma, H. Wang, S. Huang, X. Zhang, Y. Hu, T. Song, Y. Xia, and F. Wei, "Bitnet b1.58 2b4t technical report," *arXiv preprint arXiv:2504.12285*, 2025.

[175] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.

[176] T. Lindenbauer, I. Slinko, L. Felder, E. Bogomolov, and Y. Zharov, "The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management," *arXiv preprint arXiv:2508.21433*, 2025.

[177] Anthropic Engineering, "Effective context engineering for AI agents," https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025.

[178] DeepSeek-AI, "DeepSeek-V3.2: Pushing the frontier of open large language models," *arXiv preprint arXiv:2512.02556*, 2025.

[179] Google, "Gemini CLI documentation: checkpointing, context files (gemini.md), plan mode, subagents," https://geminicli.com/docs/, 2026.

[180] A. Antoniades, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang, "SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement," in *International Conference on Learning Representations (ICLR)*, 2025.

[181] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang, "Training software engineering agents and verifiers with SWE-Gym," *arXiv preprint arXiv:2412.21139*, 2024.

[182] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme speculative decoding for emerging AI applications," *arXiv preprint arXiv:2411.04975*, 2024.

[183] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," *arXiv preprint arXiv:2312.07104*, 2023.

[184] B. Lee, D. Kim, Y. You, and Y. Kim, "Littlebit: Ultra low-bit quantization via latent factorization," *arXiv preprint arXiv:2506.13771*, 2025.

[185] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” in *NeurIPS ML Safety Workshop*, 2022. [Online]. Available: https://arxiv.org/abs/2211.09527

[186] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: https://arxiv.org/abs/2202.03629